

INFORMATIQUE

2024 - 2025

Architectures Logicielles à Objet

Louis Sébastien SIMARD

SUP^F_C

Architectures Logicielles à Objets
Version 9

Contenu

Chapitre 1. Conception orientée objet.....	8
1.1. Le modèle objet.....	8
1.1.1. L'abstraction	8
1.1.2. L'encapsulation.....	8
1.1.3. Communication par messages	9
1.1.4. Le typage	9
1.1.4.1. La notion de type.....	9
1.1.4.2. Typage statique ou typage dynamique	9
1.1.5. La classification des objets	9
1.1.5.1. Héritage simple	9
1.1.5.2. Héritage multiple.....	10
1.1.6. La notion de classe	11
1.1.6.1. Définition d'une classe	11
1.1.6.2. Interface et implémentation	11
1.1.7. Classes et objets	12
1.1.7.1. L'état.....	12
1.1.7.2. Le comportement.....	12
1.1.7.3. L'identité.....	13
1.1.8. Les relations entre objets.....	13
1.1.8.1. Relations d'utilisation	14
1.1.8.2. Relations d'agrégation.....	15
1.1.8.3. Relations de composition	15
1.1.9. Relation entre classes.....	15
1.1.9.1. Relations d'héritage.....	15
1.1.10. Classes abstraites	16
1.1.11. Interfaces.....	17
1.2. Modélisation par objets	17
1.2.1. Identification des classes.....	17
1.2.2. Construire un modèle objet	18
1.3. Glossaire	19
1.4. Quelques principes de conception orientée objet.....	21
1.4.1. Le principe de Liskov	21
Chapitre 2. Conception orienté objet en Java.....	23
2.1 Concepts objet en Java.....	23
2.1.1 Introduction.....	23

2.1.2. La classe	23
2.1.3. Les objets.....	24
2.1.3.1. Le constructeur.....	25
2.1.3.2. this	26
2.1.3.3. Le destructeur	27
2.1.3.4. Les attributs de classe	27
2.1.3.5. Le polymorphisme	28
2.1.3.6. Le passage des arguments à une méthode	30
2.1.3.7. Les variables locales	31
2.1.3.8. Les méthodes de classe	31
2.1.3.9. L'héritage.....	32
2.1.3.10. Le transtypage	37
2.1.3.11. Les classes abstraites.....	38
2.1.3.12. Les interfaces.....	39
2.1.3.13. Les relations entre les classes	42
2.1.3.14. Les packages	44
2.1.3.15. Contrôle d'accès en Java	45
2.1.3.16. Les exceptions	46
2.1.4. Exercices	51
2.1.4.1. Exercice : Héritage.....	51
2.1.4.2. Exercice : Classe abstraite	52
2.1.4.3. Exercice : Expression mathématique	52
2.1.4.4. Exercice : Exceptions	52
2.1.4.5. Exercice : Bloc try/catch	53
2.1.5. Solutions	53
2.1.5.1. Héritage	53
2.1.5.2. Classe abstraite.....	55
2.1.5.3. Expression mathématique.....	57
2.1.5.4. Exceptions	58
2.1.5.5. Bloc try/catch	59
2.2 Les collections.....	59
2.2.1. Introduction.....	59
2.2.1.1. Les collections supportées par le framework Java	59
2.2.1.2. L'interface List	60
2.2.1.3. Les algorithmes du framework collections	62
2.2.1.4. L'interface Map.....	64

2.3 Les entrées-sorties	66
2.3.1. Introduction.....	66
2.3.2. Lire des fichiers textes	66
2.3.3. Writer	68
2.3.4. Formater une sortie à l'écran	69
2.3.5. Saisie de valeurs au clavier	70
2.3.6. La sérialisation des objets	71
2.4 JDBC.....	73
2.4.1. Accéder aux bases de données	73
2.4.2. Principe d'accès à une base	74
2.4.3. Exemple	75
2.4.3.1. Chargement du pilote de la base	75
2.4.3.2. Création d'une requête	75
2.4.3.3. Récupération d'un résultat.....	75
2.4.3.4. Exemple complet.....	76
2.4.4. Établissement d'une connexion	77
2.4.4.1. Chargement et déclaration du pilote	77
2.4.4.2. Connexion à la base.....	78
2.4.5. Exécution de requêtes SQL.....	79
2.4.5.1. Création d'un objet <code>Statement</code>	79
2.4.5.2. Exécution d'un <code>Statement</code>	80
2.4.5.3. Fermeture d'un objet <code>Statement</code>	81
2.4.5.4. Création d'un objet <code>PreparedStatement</code>	81
2.4.5.5. Exécution d'un <code>PreparedStatement</code>	83
2.4.5.6. Exécution de mises à jour en batch.....	84
2.4.6. Exploitation des résultats	86
2.4.6.1. Types de <code>ResultSet</code>	86
2.4.6.2. Mise à jour au travers d'un <code>ResultSet</code> : <i>concurrency</i>	87
2.4.6.3. Comportement lors de la fermeture du <code>Statement</code> : <i>holdability</i>	87
2.4.6.4. Lecture du résultat	88
2.4.6.5. Mise à jour du résultat	90
2.4.6.6. Fermeture d'un <code>ResultSet</code>	92
Chapitre 3. Design Patterns.....	93
3.1. Introduction.....	93
3.2. Les patterns	93
3.2.1 Composite.....	94

3.2.1.2. Application du modèle composite pour un calcul.....	95
3.2.2. Décorateur.....	96
3.2.2.1. Description	96
3.2.2.2. Diagramme de classes	97
3.2.2.3. Classes participantes	97
3.2.2.4. Domaine d'application	97
3.2.2. 5. Exemple	97
3.2.2.6. Conclusion	98
3.2.3. Etat	99
3.2.4. Builder	101
3.2.4.1 Le builder fluent : beaucoup de méthodes chaînées, un seul build	102
3.2.4.2 Le fluent interface	102
3.2.4.3 Le builder Command : beaucoup de setter, un seul build() (mais qui fait beaucoup).....	103
3.2.4.4 Le builder officiel : beaucoup de build, une seule class Director	103
3.2.4.5 Pourquoi tant d'implémentations différentes du builder?	106
3.2.4.6 Choisir le bon patron de conception	107
3.2.5. Observateur.....	107
3.2.5.1. Motivation	107
3.2.5.2. Diagramme de classes	108
3.2.5.3. Exemple	108
3.2.5.4. Cas d'utilisation	110
3.2.5.5. Exercice - 1.....	110
3.2.5.6. Exercice - 2.....	110
3.2.6. Singleton.....	111
3.2.6.1. Motivation	111
3.2.6.2. Exemple	112
3.2.6.3. Cas d'utilisation	112
3.2.7. Stratégie	113
3.2.7.1. Description	113
3.2.7.2. Diagramme de classes	113
3.2.7.3. Classes participantes	113
3.2.7.4. Domaines d'utilisation.....	113
3.2.7.5. Exemple	114
3.2.7.6. Conclusion	115
3.2.8. Visiteur	116
3.2.8.1 Définition.....	116

3.2.8.2 Implémentation.....	117
3.2.8.3 Intérêt.....	117
3.2.8.4 Exemple	118
3.2.9. MVC	121
3.2.9.1 Définition.....	121
3.2.9.2 Objectif	121
3.2.9.3 La couche modèle.....	121
3.2.9.4 La couche vue	121
3.2.9.5 La couche contrôleur	122
3.2.9.6 Exemple de MVC : les Swing Java.....	122
3.2.9.7 Exemple de MVC : un programme Java.....	122
Annexe A. Solutions Des Exercices	124
1. Observateur - Exercice 1.....	124
2. Observateur - Exercice 2.....	126

Chapitre 1. Conception orientée objet

1.1. Le modèle objet

Le modèle objet utilise une terminologie particulière pour exprimer certains concepts. Cette section liste ces concepts et tente de les expliquer de manière simple.

1.1.1. L'abstraction

L'abstraction est un moyen qui permet à un être humain d'affronter la complexité du monde qui l'entoure. Voici quelques définitions :

« Abstraction : principe consistant à ignorer les aspects d'un sujet qui ne sont pas significatifs pour l'objectif en cours de manière à se concentrer complètement sur ceux qui le sont. » [COAD 91]

« Une abstraction dénote les caractéristiques essentielles d'un objet qui le distinguent des autres sortes d'objets en lui donnant des limites conceptuelles bien définies et ce relativement à la vision d'un observateur. » [BOOCH 91]

Une abstraction consiste donc à se concentrer sur la vue externe d'un objet de manière à séparer le comportement essentiel d'un objet de son implémentation. Les objets communiquent les uns avec les autres à travers l'envoi de messages. Afin de pouvoir recevoir des messages, les objets exposent une interface qui contient l'ensemble des messages qu'il peut traiter. Nous pouvons considérer deux types d'abstractions :

- l'abstraction procédurale
- l'abstraction de données

« Abstraction procédurale : principe par lequel toute opération qui aboutit à un effet bien déterminé peut être considérée par ses utilisateurs comme une seule entité en dépit du fait que cette opération peut se décomposer en un ensemble d'opérations de plus bas niveau. » [COAD 91]

« Abstraction de données : principe qui consiste à définir un type de données en termes d'opérations s'appliquant aux objets de ce type, avec la contrainte que les données propres de tels objets ne puissent être modifiées ou consultées que par le biais de ces opérations. » [COAD 91]

A travers cette définition, il faut comprendre que si l'on définit un objet à travers ses attributs et les services qu'il peut rendre (à l'aide de ses méthodes), le seul moyen d'accéder aux attributs consiste à utiliser les méthodes qu'il propose.

1.1.2. L'encapsulation

L'encapsulation consiste à intégrer les données (attributs) et services (méthodes) permettant de les manipuler au sein d'une même entité : l'objet. L'encapsulation masque les détails de l'implémentation à l'utilisateur de cet objet en ne fournissant qu'une vue externe : l'interface.

« L'encapsulation est le processus qui consiste à empêcher d'accéder aux détails d'un objet qui ne contribuent pas à ses caractéristiques essentielles. » [BOOCH 91]

En n'exposant qu'une l'interface, les opérations sur l'implémentation, comme la modification de code ou une optimisation d'une méthode interne, n'affecteront pas les utilisateurs de l'objet. L'encapsulation améliore donc les opérations de maintenance et favorise l'évolution tant que l'interface de l'objet n'est pas modifiée.

1.1.3. Communication par messages

Les objets ne peuvent communiquer entre eux qu'à travers l'envoi de messages. Dans la plupart des langages à objets, cela se traduit par l'appel d'une méthode de l'objet cible. Cette méthode doit faire partie de l'interface de l'objet. La communication par messages est la conséquence directe du principe d'encapsulation. Un objet est en quelque sorte une boîte noire qui ne fournit qu'un ensemble limité des méthodes publiques afin de préserver son intégrité.

1.1.4. Le typage

1.1.4.1. La notion de type

La notion de type est liée au concept de type de données abstrait. Nous considérerons que la notion de classe d'objets et type de données sont équivalentes¹. Le typage permet de concevoir des logiciels de meilleure qualité, car le compilateur peut effectuer des contrôles supplémentaires. Notons toutefois que de nombreux langages supportant la notion d'objet n'imposent pas le typage des variables : Perl, Python, etc.

Voici quelques arguments en faveur du typage fort :

- Avec la vérification de type, la plupart des erreurs sont détectées à la compilation
- La déclaration de variables typées peut servir à produire de la documentation
- Connaissant le type d'une variable, le compilateur peut effectuer des optimisations

Le typage dynamique possède néanmoins certains avantages. Il permet de créer des composants logiciels hautement réutilisables au détriment de la détection de certaines des erreurs à l'exécution.

1.1.4.2. Typage statique ou typage dynamique

Dans le typage statique (typage fort), ce sont les références (identificateurs) aux objets qui sont typées. Lorsque nous déclarons une variable d'objet de type *Personne*, la variable ne pourra pas contenir autre chose qu'une instance de *Personne* ou d'une classe dérivée. Le compilateur peut donc effectuer des tests de cohérence à la compilation.

Dans les langages dynamiques, ce n'est pas la variable qui est typée mais son contenu. Nous pouvons donc « changer » le type d'une variable ² à l'exécution, ce qui procure une grande souplesse, mais peut tout de même poser certains problèmes³.

Le choix d'un langage à typage statique ou à typage dynamique dépend des objectifs à atteindre. Dans les développements lourds, le typage statique est à privilégier. D'ailleurs, dans les secteurs stratégiques, le langage Ada est souvent privilégié car il est intransigeant quant au typage des variables et ne supporte aucune ambiguïté dans les déclarations et les affectations.

1.1.5. La classification des objets

1.1.5.1. Héritage simple

« Héritage : mécanisme destiné à exprimer les similitudes entre classes, et ce en simplifiant la définition de classes possédant des similitudes avec celles précédemment définies. L'héritage met en œuvre des

¹ Il peut exister des nuances dans certains langages : Smalltalk par exemple.

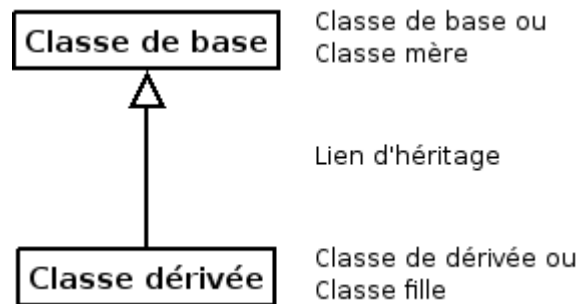
² Le contenu référencé par la variable pour être exact.

³ Les nombreux utilisateurs des langages de scripts comme PHP, Perl, ... comprendront certainement.

principes de généralisation et de spécialisation en partageant explicitement les attributs et les services communs au moyen d'une hiérarchie de classes. » [COAD 91]

Dans l'héritage simple, nous avons une classe « mère » et une classe « fille ». La classe « fille » hérite de tous les attributs et de toutes les méthodes de sa « mère ». Les termes « classe de base » et « classe dérivée » peuvent également être employés. La classe dérivée enrichie et/ou spécialise la classe de base en ajoutant des attributs et/ou des méthodes. Elle peut aussi redéfinir certaines méthodes pour les adapter au service attendu. La classe mère peut être une classe abstraite. Une classe abstraite (appelée généralement « interface ») est une classe dont les méthodes sont déclarées mais dont l'implémentation (le code) n'est pas fournie. Ce type de classe n'est pas instanciable : nous ne pouvons créer d'objet à partir d'une classe abstraite. La classe fille devra donc implémenter toutes les méthodes de sa mère.

La notation UML pour représenter une relation d'héritage est la suivante :



UML : relation d'héritage simple

Les avantages de l'héritage simple sont :

- Réduction du code source d'une application car il y a une factorisation importante
- Maintenance facilitée car la correction d'une imperfection ou d'une erreur dans une méthode d'une classe de base est aussitôt répercutée dans la hiérarchie des classes filles
- Si une hiérarchie de classes est bien conçue et déjà testée en production (donc sans bugs importants), la création de nouvelles classes plus spécialisées sera rapide et fiable car nous réutiliserons un ensemble de classes solide comme fondation pour les nouvelles
- La réutilisabilité des composants existants (créés dans le cadre d'un autre développement)
- Un partage de code, car l'héritage permet d'éviter la redondance de code

Le développement d'une application utilisant correctement l'héritage permettrait de réduire la taille du code source de 50% d'après [BOOCH 91]. Le nombre de bugs serait considérablement réduit, car il est proportionnel au nombre de lignes de code source d'une application. Il y a quelques années, une étude de Microsoft annonçait le nombre d'une erreur toutes les quinze lignes de code (et les développeurs de chez Microsoft ne sont pas des débutants).

1.1.5.2. Héritage multiple

Certains langages offrent l'héritage multiple, d'autres non. C'est un sujet épineux et les discussions entre les partisans et les opposants ne sont pas prêtes de s'éteindre.

Pour les opposants : « Comme dans 99% des cas, l'héritage multiple est mal utilisé, il faut le bannir des langages objets dignes de ce nom ». Cette manière de penser est un peu rapide. Souvenons-nous que beaucoup de langages sont nés en prenant le langage C++ comme modèle, puis ont supprimé toutes les constructions dites complexes. Par exemple : les « templates » et la « surcharge des opérateurs ». Force est de constater que les langages « simples » se complexifient à chaque version et se réapproprient les

concepts qu'ils avaient éliminés dans un premier temps. Java a repris la notion de templates (en allégé), C# a intégré les modèles et la surcharge des opérateurs.

Les partisans : « Pourquoi supprimer une notion qui peut rendre des services. C'est au développeur de prendre ses responsabilités et d'utiliser l'héritage multiple que lorsque cela s'avère nécessaire ».

Soyons pragmatique : l'héritage multiple est en général à éviter, car il est souvent mal employé et une solution alternative peut être utilisée. Néanmoins, en certaines circonstances, il est bien pratique.

Avec l'héritage multiple, une classe fille peut avoir plusieurs classes mères. Elle hérite donc des attributs et des méthodes de plusieurs sources différentes, d'où des conflits potentiels : attributs et méthodes de mêmes noms, etc. Chaque langage apporte une solution différente à ce type de problème de collisions.

1.1.6. La notion de classe

Dans cette section, nous allons nous attarder plus longuement sur les notions de classe, d'objet, d'interface, etc.

1.1.6.1. Définition d'une classe

Une classe implique les notions d'abstraction et de généralisation (héritage). Elle permet de représenter un concept du monde réel ou une invention du développeur sans matérialiser quoi que ce soit. Les concepts véhiculés par la notion de classe ne sont pas étrangers à notre manière de penser. Tous les jours, nous utilisons l'abstraction ou la généralisation pour parler du monde qui nous entoure. Nous parlons des « gens », des « voitures », etc. La généralisation est une faculté humaine qui permet de s'affranchir des détails inutiles. Quand nous constatons que les voitures roulent, nous ne nous sentons pas obligés de préciser que notre voiture roule, que celle du voisin roule également, etc. Si Patrick, l'enfant d'à côté dit à son copain : « La Ferrari de mon papa roule sur la route... », celui-ci répondra certainement avec un sourire en coin : « Ben oui. C'est une voiture ! » Un troisième larron qui écoutait discrètement ajoutera : « Comme c'est une voiture de sport, en plus elle roule très vite ! »

Cette petite discussion nous montre que l'abstraction est une faculté humaine et que nous la pratiquons instinctivement. Une voiture normale roule, une voiture de sport roule parce que c'est une voiture, mais en plus, elle va vite car c'est une voiture sportive.

En informatique, une classe est un moule qui permet de regrouper des caractéristiques communes et des comportements similaires pour créer un type de données. A partir d'une classe, nous pouvons créer des objets (aussi appelés instances de classes). Ces objets matérialiseront quelque chose de concret, directement utilisable dans une application. La classe et l'objet sont donc deux concepts différents : un objet est créé (instancié) en prenant comme modèle une classe. La classe étant un type de données, un objet est donc typé.

1.1.6.2. Interface et implémentation

Une classe expose vers l'extérieur une partie publique aussi appelée « interface » ou « vue externe ». Pour le développeur d'applications, c'est la partie la plus difficile à définir correctement car toute modification de cette interface aura des répercussions sur les clients (les utilisateurs de la classe). Une interface est en quelque sorte un contrat avec les utilisateurs et une classe doit, si possible, respecter ce contrat sans jamais le remettre en cause.

L'implémentation de la classe est par contre cachée, invisible au client. La modification de l'implémentation n'aura aucune répercussion sur le code de l'utilisateur de la classe. Cette partie peut donc être sujette à amélioration et à optimisation.

Une classe est donc découpée (en fonction des langages) en plusieurs parties dont les deux principales sont :

- une partie publique visible de tous les clients : c'est à travers l'interface que l'on communique avec l'objet,
- une partie privée inaccessible depuis l'extérieur : l'état (les propriétés) est mémorisé dans cette section.

1.1.7. Classes et objets

Qu'est-ce qu'un objet ? Humainement parlant, un objet peut être défini comme :

- une chose palpable et/ou visible,
- quelque chose qui peut être appréhendé intellectuellement,
- quelque chose vers qui la pensée ou l'action est dirigée

En informatique, la définition d'un objet est souvent plus abstraite. Certains objets existeront uniquement pour « rendre » une architecture logicielle plus souple et plus évolutive, d'autres représenteront des choses éphémères comme des algorithmes, etc.

Dans la conception orientée objet, les objets ne représenteront donc pas uniquement des choses du monde réel, mais également des éléments abstraits, pour peu que ceux-ci soient pertinents dans le domaine de l'application en cours d'étude. Ainsi, dans une application de traitement d'images numériques, une source de lumière serait certainement vue comme un objet. Pour citer un autre exemple, l'état d'un capteur de chaleur dans un système de chauffage d'une centrale nucléaire serait également considéré comme un objet informatique.

Pour résumer, certains objets peuvent avoir des limites conceptuelles un peu floues. Ce type d'objets apparaît en général lorsque la conception d'une application est bien avancée ou plus généralement en observant le modèle objet d'une application similaire.

Nous adopterons la définition d'un objet donnée par Grady Booch : « Un objet a un état, un comportement et une identité ; la structure et le comportement d'objets similaires sont définis dans leur classe commune ; les termes instance et objet sont interchangeables. »

1.1.7.1. L'état

Un objet est toujours dans un état particulier. Prenons l'exemple d'un véhicule. Il peut être à l'arrêt, en mouvement, freiner, etc. En fonction de son état, ses réactions peuvent changer. Par exemple, appuyer sur la pédale d'accélérateur ne provoquera pas le même comportement si le moteur est à l'arrêt ou en marche.

L'état d'un objet est généralement matérialisé par la valeur des propriétés qui le caractérise. Dans l'exemple ci-dessus, une variable pourrait mémoriser si le moteur est en fonctionnement ou non. Dans certains cas complexes, une propriété peut être représentée par un autre objet. Dans les cas simples, une valeur scalaire comme une chaîne de caractères ou un entier peuvent suffire.

1.1.7.2. Le comportement

Le comportement d'un objet est sa capacité à répondre à des sollicitations externes en fonction de l'état de ses propriétés. Nous agissons sur un objet en lui envoyant un message. Clarifions immédiatement ce qu'est un message : dans la plupart des langages orientés objets, envoyer un message à un objet revient à appeler l'une de ces méthodes publiques.

Dans l'exemple ci-dessous, nous créons un objet *Personne* représenté par la variable *p*. Nous lui envoyons ensuite le message « Modifie ton nom en "DUPONT" puis ton âge en 25 ». En C++, comme en Java ou en C#, nous invoquons les méthodes `setNom()` et `setAge()` pour simuler l'envoi de ces messages.

```
Personne p = new Personne();  
p.setNom( "DUPONT" );  
p.setAge( 25 );
```

Le comportement d'un objet peut donc être défini par la réaction de celui-ci en réponse à la réception d'un message qui lui est destiné. Un objet étant une sorte de boîte noire ; il expose uniquement son interface publique aux clients. Le client d'un objet ne peut donc « voir » que le résultat d'un message émis à un objet cible, mais n'a pas la possibilité de savoir comment le résultat a été obtenu. Le comportement d'un objet est donc également encapsulé.



Rappel : la notion de client

Dans notre terminologie, le client et l'utilisateur d'un objet sont interchangeables.

1.1.7.3. L'identité

L'identité est souvent source de confusion. Par défaut, nous considérons que deux objets en mémoire (donc à des adresses machines différentes) sont deux objets distincts. Dans l'absolu, nous avons parfaitement raison, mais ce n'est pas toujours correct. Considérons la nouvelle définition de la classe *Personne* :

```
class Personne {  
    public Personne( String numss ) { }  
    public String getNumSS() { }  
  
    public void setNom(String nom) { }  
    public String getNom() { }  
  
    public void setAge(Integer age) { }  
    public Integer getAge() { }  
  
    private String numss;  
    private Integer age;  
    private String nom;  
};
```

Créons deux objets *Personne* *p1* et *p2* à partir de cette classe :

```
Personne p1 = new Personne( "NUMSS_1" );  
Personne p2 = new Personne( "NUMSS_1" );
```

Maintenant, posons-nous la question suivante : *p1* et *p2* sont-ils deux objets différents ?

Dans la mémoire de l'ordinateur, certainement. Les variables *p1* et *p2* représentent deux emplacements différents. Conceptuellement parlant, c'est moins évident. Nous voyons bien que *p1* et *p2* représentent la même personne, puisque le numéro de sécurité sociale est identique. Nous avons donc deux objets informatiques différents qui représentent le même objet du monde réel. La définition de l'identité d'un objet donnée par Khoshafian et Copeland est : « L'identité est cette propriété d'un objet qui le distingue de tous les autres objets ». Dans notre exemple, le numéro de sécurité sociale permet de différencier des objets de type *Personne*. Dans certains cas, aucune propriété ne permettra de faire cette distinction. Nous utiliserons l'adresse mémoire de l'objet comme identité.

1.1.8. Les relations entre objets

Dans une application informatique, les objets ne sont pas isolés. Ils évoluent entourés d'autres objets et doivent collaborer ou composer avec les autres. C'est l'agencement de tous ces objets qui forme une architecture dite orientée objets.

Les objets sont donc en relation avec d'autres. Nous pouvons rapidement lister trois types de relations :

- les relations d'utilisation,
- les relations de composition,
- les relations d'agrégation.

1.1.8.1. Relations d'utilisation

La relation d'utilisation est simple. Pour pouvoir utiliser l'objet *objetB* de type B, l'objet *objetA* de type A doit pouvoir accéder à l'interface publique d'*objetB*. En général, il nous suffira « d'inclure » ou « d'importer » la définition de la classe de B avant l'appel d'une méthode d'*objetB* par *objetA*.

La relation d'utilisation est tellement courante et intuitive que nous ne nous attarderons pas trop longtemps dessus. Nous donnons seulement un exemple simple de relation d'utilisation en C++.



[...]

La chaîne de caractères [...] signifie que le code est incomplet et que les lignes non pertinentes ont été supprimées.

Soit les classes *Salarie* et *CalculateurDePaie* : la définition de la classe *Salarie* est située dans le fichier *salarie.h*. L'implémentation de la classe *CalculateurDePaie* est quant à elle située dans le fichier *calculateurdepaie.cpp*.

Voici un extrait du fichier *salarie.h*

```
[...]
class Salarie {
public:
    Salarie( unsigned int num );

    unsigned int getTypeContrat() const;

private:
    [...]
};
[...]
```

Maintenant le fichier *calculateurdepaie.cpp* :

```
#include "salarie.h"

[...]

int CalculateurDePaie::calculerPaie( const Salarie& p )
{
    if( p.getTypeContrat() == CDD ) {
        calculerPaieContratCDD( p );
    }
    else if ( p.getTypeContrat() == CDI ) {
        calculerPaieContratCDI( p );
    }
}
```

```

    else {
        // autres traitements
    }

    [...]
}

[...]
```

Nous constatons que la méthode *calculerPaie* de la classe *CalculateurDePaie* utilise un objet de type *Salarie*. Pour pouvoir utiliser un objet de ce type, nous devons donc inclure le fichier *salarie.h* dans le fichier *calculateurdepaie.cpp*. En cas d'oubli, une erreur de compilation surviendra.

1.1.8.2. Relations d'agrégation

Dans le monde réel, les relations d'agrégation sont nombreuses. Un véhicule est composé d'un moteur, de roues, etc. Nous sommes dans une relation de contenance. Un objet en contient donc d'autres et l'ensemble forme un tout. Il faut remarquer que la durée de vie des objets agrégés n'est pas forcément liée à celle du conteneur. Pour reprendre l'exemple du véhicule, la destruction du véhicule n'implique pas forcément celle des composants. C'est la relation de contenance la plus commune.

1.1.8.3. Relations de composition

Les relations de composition diffèrent des relations d'agrégation dans le fait que la durée de vie des composants est liée à celle du conteneur. Prenons l'exemple d'une interface utilisateur. Les interfaces graphiques utilisent en général des relations de composition : un formulaire contient des contrôles comme les zones de saisie, les boutons poussoirs, etc. Lorsque le formulaire est détruit, tous les contrôles le sont également, car ils ne peuvent pas exister en dehors de celui-ci.

1.1.9. Relation entre classes

Considérons les énumérations suivantes :

- Un capteur infrarouge est une sorte de capteur,
- Un capteur volumétrique est une sorte de capteur,
- Un capteur infrarouge avec connexion WIFI et un capteur infrarouge avec connexion satellite sont tous les deux des capteurs infrarouges,
- Le processeur Motorola 6809 est un composant des capteurs infrarouges avec connexion WIFI ou connexion satellite.

Cette énumération très simple montre qu'il existe des relations entre les classes comme il existe des relations entre les objets.

1.1.9.1. Relations d'héritage

Dans le langage courant, nous pourrions dire « Un capteur infrarouge est un genre ou une sorte de capteur ». En somme, nous généralisons et nous faisons abstraction du fait que notre capteur infrarouge fonctionne justement en infrarouge. L'important, au moment où l'on parle, c'est que ce soit un capteur.

Dans la POO ⁴, la généralisation se traduira par une relation d'héritage. L'héritage impose une relation forte et statique entre les classes. Cette relation est définie et figée à la compilation. Il sera donc impossible de la modifier dynamiquement. Elle permet de factoriser des propriétés et des comportements communs et nous évite de dupliquer inutilement du code, tout en créant une hiérarchie de classes reliées entre elles par la relation « est une sorte de ». Si une classe *ClasseB* hérite de la classe *ClasseA*, *ClasseB* est de type *ClasseB* mais également de type *ClasseA*. Cela a pour conséquence qu'un tableau paramétré pour contenir des objets de type *ClasseA* pourra contenir également des objets de type *ClasseB* ⁵.

L'héritage dénotant une relation de généralisation/spécialisation, on peut traduire toute relation d'héritage par : la classe dérivée est une version spécialisée de sa classe de base. De nombreux langages ont supprimé la possibilité d'utiliser l'héritage multiple. Dans ces langages, une classe ne peut avoir qu'une classe de base.

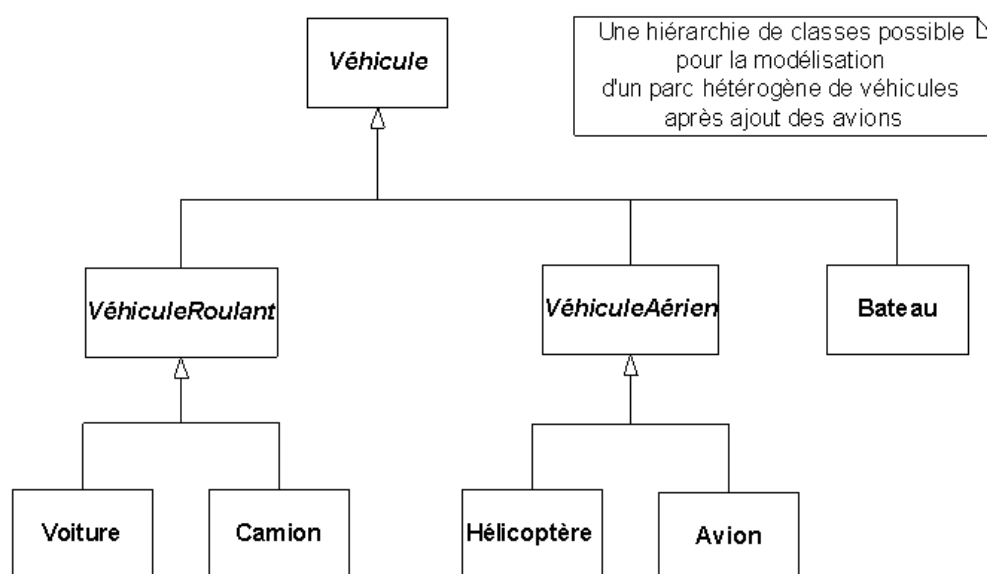
L'héritage permet donc à une classe de récupérer les variables membres et les fonctions membres de la classe mère mais quid du comportement ?

1.1.9.1.1. Le polymorphisme

Si l'héritage des variables membres est quelque chose de positif, l'héritage du comportement sans possibilité de le redéfinir lorsque cela est nécessaire rendrait l'héritage en général sans grande valeur. C'est là qu'intervient la notion de polymorphisme dynamique. Une classe fille peut redéfinir l'implémentation d'une fonction membre héritée de la classe mère. Elle peut changer complètement le contenu de cette fonction ou bien récupérer le comportement de base et ajouter quelques modifications afin de la spécialiser. Ce type de polymorphisme est dit *dynamique*, car la fonction membre à appeler ne sera connue qu'à l'exécution et non pas à la compilation. Il faut faire remarquer que cela induit une petite surcharge à l'exécution.

Il existe un autre type de polymorphisme : le polymorphisme statique. Cette fois, le but est de pouvoir avoir des fonctions membres ayant le même nom. Une contrainte toutefois présente dans la plupart des langages : les arguments de ce type de fonction doivent impérativement être différents. Notez que dans la majorité des langages à objets, le type de la valeur retournée ne permet pas au compilateur de choisir une fonction plutôt qu'une autre.

1.1.10. Classes abstraites



⁴ Programmation orientée objets

⁵ Attention ! L'inverse n'est pas vrai mais nous y reviendrons en temps voulu.

Certains noms de classes - *Véhicule*, *VéhiculeRoulant* et *VéhiculeAérien* - sont en italique dans la figure ci-dessus : cela signifie que ces classes sont dites abstraites. Une classe est dite abstraite si elle ne peut posséder d'instance. C'est le cas si elle ne fournit pas d'implémentation pour certaines de ces méthodes qui sont dites méthodes abstraites. Il appartient donc à ces classes dérivées de définir du code pour chacune des méthodes abstraites. Par opposition, on pourra alors parler de classes concrètes ; les classes concrètes sont les seules susceptibles d'être instanciées.

Les classes abstraites permettent de définir un cadre de travail pour les classes dérivées en proposant un ensemble de méthodes que l'on retrouvera tout au long de la hiérarchie. Ce mécanisme est fondamental pour la mise en place du polymorphisme. En outre, si l'on considère la classe *Véhicule*, il est tout à fait naturel qu'elle ne puisse avoir d'instance : un objet instance de *Véhicule* ne correspond à aucun objet concret, mais plutôt au concept d'un objet capable de démarrer, ralentir... que ce soit une voiture, un camion ou un avion.

1.1.11. Interfaces

Les problèmes liés à l'héritage multiple ont conduit certains concepteurs de langages à le bannir des fonctionnalités proposées. On a vu apparaître d'autres mécanismes telles les interfaces. Une interface est semblable à une classe sans attribut (mais pouvant contenir des constantes) dont toutes les méthodes sont abstraites.

En plus de ses caractéristiques d'héritage, une classe implémente une interface si elle propose une implémentation pour chacune des méthodes décrites en interface. Ce mécanisme est particulièrement puissant, car il crée des relations de polymorphisme fortes entre différentes classes implémentant les mêmes interfaces sans que ces dernières aient une relation d'héritage. En particulier, les méthodes décrites dans les interfaces sont, par définition, polymorphes puisqu'elles sont implémentées de façon indépendante dans chaque classe implémentant une même interface.

Faire la différence entre héritage et utilisation d'interfaces n'est pas toujours aisé et dépend du point de vue du concepteur, mais également de l'environnement du logiciel à construire.

1.2. Modélisation par objets

1.2.1. Identification des classes

Soyons clair : il n'existe aucune méthode formelle pour identifier les classes constituant une application. Il s'agit d'un travail difficile. Seule des discussions avec les experts du domaine⁶ peuvent aider l'analyste à identifier les classes pertinentes, surtout si le domaine étudié lui est inconnu. L'étude d'applications similaires peut fortement aider et réduire les erreurs de conception.

Dans un premier temps, nous devons découvrir les abstractions et les objets utilisés par les experts du domaine. Il y a donc une forte interaction entre les analystes et les utilisateurs. Les utilisateurs décrivent leur travail au quotidien en employant des termes qui leur sont familiers. Il s'agit d'une description exhaustive qui va nous permettre d'isoler les classes potentielles. Voici une liste qui peut faciliter la découverte des classes et objets [COAD 91] et [BOOCH 91] :

- Choses tangibles : Voiture, Commande, Facture, ...
- Les personnes et rôles : Secrétaire, Opérateur, ...
- Événements : Alarmes, Clic de souris, ...
- Interactions entre objets : Location de voitures, ...
- Emplacements physiques : Bureau, Atelier...
- Concepts, principes ou idées non tangibles : Connaissance, Métier, ...

⁶ Un expert du domaine est simplement un utilisateur ayant une bonne compréhension du domaine étudié.

- Systèmes extérieurs avec lesquels le système que l'on étudie interagit

1.2.2. Construire un modèle objet

Nous vous donnons les conseils de certains experts en programmation orientée objet en ce qui concerne la création d'un modèle objet :

- Ne pas commencer à construire un modèle objet en jetant pêle-mêle classes, associations et héritage. Il faut avant tout comprendre le modèle à résoudre. Le modèle objet est conditionné par la pertinence de la solution
- Identifier les classes d'objets
- Choisir les noms avec soin. Ils doivent être descriptifs et sans ambiguïté
- Commencer à remplir un dictionnaire de données contenant les descriptions des classes, des attributs et des associations
- Ajouter les associations entre classes
- Ajouter les attributs et les liens
- Organiser et simplifier les classes d'objets en utilisant l'héritage.

1.3. Glossaire

Le tableau ci-dessous récapitule les principaux termes utilisés en conception orientée objet.

Tableau 1.1. Glossaire des concepts du modèle objet

Concept	Définition
Abstraction de données	Principe considérant que la structure interne des données est cachée à leur utilisateur qui ne connaît d'elles que les procédures permettant de les traiter. Du point de vue de la programmation objet, on considérera toujours un objet comme une boîte noire que l'on manipule en lui envoyant des messages.
Abstraction procédurale	Principe considérant que toute action est atomique du point de vue de celui qui la déclenche. En un mot, il n'a aucune idée des mécanismes mis en jeu dans l'implémentation de l'action. Du point de vue de la programmation orientée objet, cela signifie que l'appel d'un message est toujours vu comme atomique. Dans la pratique cela revient à masquer à l'utilisateur le code d'implémentation des sous-programmes qu'il utilise.
Classe	Composant logiciel décrivant des objets. On dit que la classe est la métadonnée des objets.
Sous-classe	On dit également <i>Classe fille</i> ou <i>Classe dérivée</i> . Se dit d'une classe qui dérive d'une autre classe (on parle de classe mère ou de super-classe). C'est une forme spécialisée de sa super-classe et à ce titre, elle redéfinit souvent certaines de ses méthodes pour les adapter à ses fonctions.
Super-classe	Classe générale qui a été dérivée en une ou plusieurs sous-classes. Une super-classe définit un cadre de travail et propose des méthodes d'ordre général à toute une famille. Certaines d'entre elles sont destinées à être redéfinies par les sous-classes afin d'adapter leur comportement dans une classe spécialisée. Une super-classe peut même être abstraite, c'est à dire ne pas fournir d'implémentation pour certaines méthodes qui devront alors être obligatoirement redéfinies dans les classes concrètes.
Classe abstraite	Classe modélisant un concept de haut niveau et dont certaines méthodes (elles-mêmes dites abstraites) ne peuvent être implémentées. Cette classe est destinée à être dérivée au cours du processus d'héritage.
Classe concrète	Classe dont toutes les méthodes sont implémentées et qui, de ce fait et par opposition à une classe abstraite (voir ci-dessus), est susceptible d'être instanciée.
Encapsulation	L'un des trois principes fondamentaux du paradigme objet. Il prône, d'une part le rassemblement des données et du code les utilisant dans une entité unique nommée objet, d'autre part, la séparation nette entre la partie publique d'un objet (ou interface) seule connue de l'utilisateur de la partie privée ou implémentation qui doit rester masquée.
Héritage	L'un des trois principes fondamentaux du paradigme objet. Possibilité offerte par les langages orientés de définir des arborescences de classes traduisant le principe de généralisation / spécialisation. Une relation d'héritage peut se traduire par la phrase : « La classe dérivée est une version spécialisée de sa classe de base. »
Interface	Partie visible d'un objet ou d'une classe. C'est donc la liste des messages auxquels un objet (une classe) est capable de répondre. Sorte de classe sans attribut et ne déclarant que des méthodes abstraites (i.e. sans implémentation).

Concept	Définition
	• Sorte de classe sans attribut et ne déclarant que des méthodes abstraites (i.e. sans implémentation).
Message	Unique moyen de communication fourni par les objets. Une invocation de message se traduit habituellement par l'activation d'une méthode. D'ailleurs, dans la plupart des langages orientés objet modernes, il n'y a pas de distinction entre les notions de message et de méthode.
Objet	Entité fondamentale rassemblant des données, ainsi que le code, opérant sur ces données. Un objet communique avec son environnement par messages et répond aux principes d'abstraction de données et d'abstraction procédurale. Un ensemble d'objets présentant les mêmes caractéristiques forme une classe.
Polymorphisme	L'un des trois principes fondamentaux du paradigme objet. Faculté présentée par certaines méthodes dont le comportement est différent (malgré une signature identique) selon l'objet auquel elles s'appliquent. Dans la plupart des langages orientés objet, ce comportement n'est disponible que pour des classes appartenant au même graphe d'héritage.
Signature d'une fonction	Liste de paramètres et type de retour d'une fonction.

1.4. Quelques principes de conception orientée objet

1.4.1. Le principe de Liskov

Le principe de Liskov permet de vérifier la validité d'une relation d'héritage entre deux classes. Ce principe est également connu sous le terme du « principe des 100% ». Pour valider le principe de Liskov, nous devons pouvoir, entre deux classes A et B, B héritant de A :

- Dire la phrase B est un A.
- Remplacer A par B dans toutes les situations sans que cela ne mène à une incohérence sémantique.

Afin de montrer ce principe dans un cas simple mais bien réel, utilisons une hiérarchie d'objets graphiques : *ObjetGraphique*, *Rectangle* et *Carré* :

- *ObjetGraphique* est la classe de base
- *Rectangle* hérite de la classe *ObjetGraphique*
- *Carré* hérite de la classe *Rectangle*

Appliquons le principe de Liskov entre les classes *Rectangle* et *ObjetGraphique* :

- Nous pouvons dire sans problème que *Rectangle* est un *ObjetGraphique*.
- Pouvons-nous remplacer *Rectangle* par *ObjetGraphique* dans le code si la substitution concerne une variable qui ne fait pas appel à des méthodes spécifiques à *Rectangle* ? Oui. La sémantique des méthodes d'*ObjetGraphique* ne change pas pour *Rectangle*.

Maintenant, appliquons le principe de Liskov entre les classes *Carré* et *Rectangle* :

- Nous pouvons dire sans problème que *Carré* est un *Rectangle*.
- Pouvons-nous remplacer *Carré* par *Rectangle* dans le code si la substitution concerne une variable qui ne fait pas appel à des méthodes spécifiques à *Rectangle* ? Non.

Pourquoi ?

Si nous faisons hériter *Carré* de *Rectangle*, la sémantique de certaines méthodes va changer pour la classe *Carré* par rapport à celles de la classe *Rectangle*.

Nous allons avoir très certainement les méthodes *setLargeur(...)* et *setHauteur(...)* dans la classe *Rectangle*. *Carré* va redéfinir le comportement de ces deux méthodes pour faire en sorte de modifier les attributs largeur et hauteur à l'appel de chacune des méthodes. Ceci afin de conserver sa propriété d'être un *Carré*.

Sans pointer du doigt qu'une méthode nommée *setLargeur(...)* ne devrait pas modifier un attribut concernant la hauteur d'une instance, le principe des 100% est cassé. Les deux méthodes n'ont plus la même sémantique en fonction du type de l'instance. Un développeur qui appelle une méthode *setLargeur(...)* ne s'attend sans doute pas à voir la hauteur de l'instance modifiée à son insu. *Carré* ne doit donc pas hériter de *Rectangle*.

Dans le cas présent, la classe *Carré* est un *Rectangle* dont la particularité est d'avoir la largeur égale à la hauteur. Il est donc plus judicieux d'ajouter une méthode *estCarre()* dans l'interface de *Rectangle*.

En conception orientée objet, il faut en permanence se poser la question suivante : le principe de Liskov est-il respecté ? C'est très important, car il permet de contrôler si une relation d'héritage est valide ou non.

2.1 Concepts objet en Java

2.1.1 Introduction

Le langage Java a été construit en pensant à la programmation orientée objet. C'est d'ailleurs un langage différent du C++ sur cet aspect. C++ permet l'utilisation de plusieurs types de programmation dont l'objet. En Java, tout est objet⁷. Même le programme est un objet : nous devons déclarer une classe possédant une méthode statique nommée *main* pour pouvoir définir le point d'entrée d'un programme. Le modèle objet de Java est propre et bien pensé. Il s'est inspiré des langages qui l'ont précédé pour utiliser les concepts intéressants et laisser de côté les choses inutiles ou trop complexes.

Au cours de ce chapitre, nous allons passer en revue les éléments fondamentaux permettant de programmer en utilisant les concepts orientés objet.

2.1.2. La classe

En programmation orientée objet, nous manipulons des entités que nous appelons des objets. Nous communiquons avec un objet en utilisant une interface que cet objet met à notre disposition.

Une application orientée objet est tout simplement un assemblage d'objets qui vont communiquer entre eux pour obtenir au final le résultat attendu.

Les objets sont donc des petits programmes théoriquement autonomes. Ils possèdent leurs propres variables et leurs propres méthodes. Nous pouvons considérer un objet comme une couche ou une unité d'abstraction, tout comme une méthode ou une fonction dans la programmation structurée. La programmation orientée objet permet donc de découper une application en utilisant un plus haut niveau d'abstraction que ne le permettait la programmation structurée. La POO permet de se rapprocher encore plus du monde réel, car nous définissons des objets qui représentent en général des choses tangibles, très proches du monde réel. Nous pouvons considérer que la programmation orientée objet est une évolution de la programmation structurée.

Un objet est une instance d'une classe d'objet. Une classe est une sorte de moule qui décrit ce que contiendra un objet de cette classe. Une classe définit les attributs et les méthodes que chacune des instances d'objets contiendront.

Une visibilité est associée à chacun de ces éléments. En schématisant, un attribut ou une méthode peut être *public*, *protected* ou *private*. Les éléments marqués comme *public* définissent l'interface publique de la classe. L'interface publique doit être minimale et ne pas être sujette à modification. Les attributs sont toujours encapsulés et par conséquent ne doivent jamais être marqués comme *public*. Cette remarque concernant les attributs n'est pas imposée par le langage, mais par une convention tout à fait justifiée.

Les attributs mémorisent l'état d'un objet à un instant donné. Modifier la valeur d'un attribut peut déclencher une succession d'actions afin de maintenir l'objet dans un état cohérent. Les attributs doivent donc être modifiés à travers des méthodes qui l'on nomme des accesseurs. La convention en Java est de préfixer ces méthodes avec *get* ou *set*.

⁷ Ce n'est pas tout à fait vrai puisqu'il existe encore les types de base.

Soit la classe *Compte* définie par le diagramme de classe UML suivant :

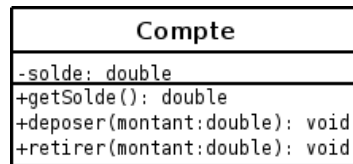


Diagramme UML - Classe Compte

Nous pouvons constater que la classe *Compte* possède un attribut *solde* de type *double*. Cet attribut est privé, donc inaccessible depuis l'extérieur de l'objet. Trois méthodes publiques sont également présentes. Elles définissent l'interface publique de la classe et sont visibles de tous les autres objets.

Observons maintenant l'implémentation de la classe en Java :

Exemple 3.1. La classe *Compte*

```
public class Compte {
    private double solde = 0.0;

    public double getSolde() { return solde; }

    public void deposer(double montant) {
        solde += montant;
    }

    public void retirer(double montant) {
        if (solde == 0) {
            System.out.println("Le compte est vide !");
            return;
        }
        else if (montant > solde) {
            System.out.println("Montant trop élevé !");
            return;
        }
        else
            solde -= montant;
    }
}
```

La méthode *getSolde* est un accesseur. Elle permet de lire l'attribut *solde*. Nous constatons qu'il n'est pas possible de modifier directement l'attribut *solde*. Par contre, les méthodes *deposer* et *retirer* agissent sur cet attribut. *deposer* ne fait rien d'extraordinaire, mais *retirer* effectue des contrôles avant d'agir sur l'attribut.

2.1.3. Les objets

Comme annoncé dans la section précédente, les objets sont créés à partir des classes. Un objet est une instance d'une classe. Tous les objets instanciés à partir d'une même classe ont une structure et un comportement similaire. Un objet est un composant logiciel qui encapsule :

- Une identité : l'identité d'un objet est la caractéristique qui permet de le différencier des autres objets du même type.
- Un état : l'état d'un objet est représenté par la valeur de ses attributs.

- Un comportement : le comportement d'un objet est défini par ses méthodes.

2.1.3.1. Le constructeur

Un constructeur est une méthode particulière dont le nom est identique au nom de la classe dans laquelle il est défini. Le constructeur est utilisé par l'opérateur *new* lors de la création d'un objet :

```
Compte compteAlain, compteSylvie;  
  
compteAlain = new Compte("ALAIN001");  
compteSylvie = new Compte("SYLVIE001", 1000.00);
```

L'exemple montre la création de deux instances d'objet à partir de la classe *Compte*. Nous voyons des appels à des constructeurs différents pour les variables *compteAlain* et *compteSylvie*. En effet, nous pouvons déclarer plusieurs constructeurs différents pour une classe. Il suffit que ces constructeurs aient des arguments qui permettent de les différencier. Il existe également un constructeur particulier : c'est le constructeur par défaut. Il ne prend aucun argument. Si le développeur ne définit aucun constructeur pour une classe, le compilateur Java crée automatiquement un constructeur par défaut.

L'opérateur *new* permet de :

- de réserver de la mémoire auprès de la JVM,
- de créer un objet dont tous les attributs sont initialisés avec la valeur 0 pour les types numériques et avec la valeur *null* pour les références.

Le rôle du constructeur est d'initialiser les attributs de manière appropriée lorsque l'opérateur *new* a fini son travail. Nous pouvons remarquer qu'un constructeur ne retourne pas de valeur.

Modifions la classe *Compte* pour y ajouter les constructeurs et un attribut dont le rôle est de mémoriser le numéro du compte :

Exemple 3.2. Compte.java

```
// Compte.java  
public class Compte {  
    private double solde = 0.0;  
    private String numero = "";  
  
    public Compte(String numeroCompte) {  
        numero = numeroCompte;  
    }  
  
    public Compte(String numeroCompte, double soldeInitial) {  
        numero = numeroCompte;  
        solde = soldeInitial;  
    }  
  
    public String getNumero() { return numero; }  
    public double getSolde() { return solde; }  
  
    public void deposer(double montant) {  
        solde += montant;  
    }  
  
    public void retirer(double montant) {  
        if (solde == 0) {  
            System.out.println("Le compte est vide !");  
            return;  
        }  
    }  
}
```

```

    }
    else if (montant > solde) {
        System.out.println("Montant trop élevé !");
        return;
    }
    else
        solde -= montant;
}
}

```

Testons notre classe avec un petit programme. Rappelons que la classe *Compte* doit être dans un fichier nommé *Compte.java*. Le programme qui est lui-même un objet est défini dans le fichier *TestCompte1.java* et la classe représentant le programme est nommée *TestCompte1* :

```

// TestCompte1.java
public class TestCompte1 {
    public static void main (String Arguments []) {
        Compte compteAlain, compteSylvie;

        compteAlain = new Compte("ALAIN001");
        compteSylvie = new Compte("SYLVIE001", 1000.00);

        System.out.println ("Compte Alain  : " + compteAlain.getNumero() + "\n"
+
                                "                        " + compteAlain.getSolde());
        System.out.println ("Compte Sylvie : " + compteSylvie.getNumero() +
"\n" +
                                "                        " + compteSylvie.getSolde());
    }
}

```

Les fichiers *Compte.java* et *TestCompte1.java* sont dans le même dossier. Nous compilons le programme et nous l'exécutons :

```

~$ javac TestCompte1.java
~$ java TestCompte1
Compte Alain  : ALAIN001
                0.0
Compte Sylvie : SYLVIE001
                1000.0
~$

```

L'accès aux membres (attributs ou méthodes) se fait à l'aide de l'opérateur « . » .

2.1.3.2. this

Dans l'implémentation des méthodes, il nous est possible d'utiliser l'opérateur *this*. Cet opérateur est une référence sur l'objet en cours. Un cas d'utilisation de cet opérateur est, par exemple, de lever l'ambiguïté entre le nom des paramètres et le nom des attributs internes à l'objet. Imaginons que nous ayons une méthode permettant de réinitialiser le solde d'un compte, cette méthode est nommée *setSolde* :

```

public void setSolde(double solde) {
    this.solde = solde;
}

```

L'opérateur *this* indique au compilateur que l'attribut *solde* reçoit la valeur de l'argument *solde*. En l'absence de *this*, le compilateur ne sait pas à quoi correspond la variable *solde* : attribut ou argument ?

2.1.3.3. Le destructeur

Le destructeur au sens C++ n'a pas de sens en Java, ni dans aucun langage ayant un ramasse-miettes. Une méthode particulière est appelée lorsque le « Garbage Collector » s'occupe de supprimer une instance. Cette méthode a la signature suivante :

```
public void finalise();
```

Nous pouvons redéfinir cette méthode si besoin. Néanmoins, son utilité est discutable, car nous ne savons pas quand le GC va récupérer notre objet. Prévoir de fermer une connexion réseau ou un fichier dans la méthode *finalise* n'est pas une bonne idée. Nous verrons plus tard comment résoudre ce problème. Force est de constater que C++ peut faire les choses beaucoup mieux dans ce domaine avec le principe RAII.

2.1.3.4. Les attributs de classe

Les attributs de classe sont des attributs dont la particularité est d'être partagés par toutes les instances d'une classe. Un attribut de ce type n'est donc pas dupliqué dans chaque instance, mais commun à toutes les instances. Ajoutons un attribut de classe à notre classe *Compte*. Cet attribut nous est utile pour « compter » toutes les instances que nous avons créé. Grâce à un attribut de classe, chaque objet pourra nous donner cette information.

```
// Compte.java
public class Compte {
    private double solde = 0.0;
    private String numero = "";
    private static int nombreInstances = 0;

    public Compte(String numeroCompte) {
        numero = numeroCompte;
        nombreInstances++;
    }

    public Compte(String numeroCompte, double soldeInitial) {
        numero = numeroCompte;
        solde = soldeInitial;
        nombreInstances++;
    }

    public String getNumero() { return numero; }
    public double getSolde() { return solde; }

    public void deposer(double montant) {
        solde += montant;
    }

    public void retirer(double montant) {
        if (solde == 0) {
            System.out.println("Le compte est vide !");
            return;
        }
        else if (montant > solde) {
            System.out.println("Montant trop élevé !");
            return;
        }
        else
            solde -= montant;
    }

    public int getNombreInstances() {
        return nombreInstances;
    }
}
```

```
}  
}
```

Le programme permettant de tester notre attribut de classe :

```
public class TestCompte1 {  
    public static void main (String Arguments []) {  
        Compte compteAlain, compteSylvie;  
  
        compteAlain = new Compte2("ALAIN001");  
        compteSylvie = new Compte2("SYLVIE001", 1000.00);  
  
        System.out.println ("Compte Alain   : " + compteAlain.getNumero() + ", \n" +  
                             "                               " + compteAlain.getSolde());  
        System.out.println ("Compte Sylvie  : " + compteSylvie.getNumero() + ", \n"  
+  
                             "                               " + compteSylvie.getSolde());  
  
        System.out.println (  
            "Nombres d'instances : " + "\n" +  
            "    depuis Alain      : " + compteAlain.getNombreInstances() + "\n" +  
            "    depuis Sylvie     : " + compteSylvie.getNombreInstances());  
    }  
}
```

Il serait possible d'accéder à la variable statique *nombreInstances* directement depuis la classe sans passer par une instance si la variable était marquée comme *public* :

```
System.out.println(Compte.nombreInstances);
```

Cela démontre que les variables statiques sont complètement indépendantes des instances. Nous verrons fréquemment ce type de notation dans le framework Java.

2.1.3.5. Le polymorphisme

Le polymorphisme est la capacité de définir plusieurs méthodes ayant le même nom, du moment que les arguments sont différents. Modifions la classe *Compte* pour lui ajouter deux méthodes de débogage. Ces méthodes sont toutes deux nommées *debug*. La méthode sans argument affiche des informations à l'écran, la seconde prend un argument qui est le nom d'un fichier de logs où seront enregistrées les informations.

```
import java.io.*;  
  
public class Compte {  
    private double solde = 0.0;  
    private String numero = "";  
    private static int nombreInstances = 0;  
  
    public Compte(String numeroCompte) {  
        numero = numeroCompte;  
        nombreInstances++;  
    }  
  
    public Compte(String numeroCompte, double soldeInitial) {  
        numero = numeroCompte;  
        solde = soldeInitial;  
        nombreInstances++;  
    }  
  
    public String getNumero() { return numero; }
```

```

    public double getSolde() { return solde; }

    public void deposter(double montant) {
        solde += montant;
    }

    public void retirer(double montant) {
        if (solde == 0) {
            System.out.println("Le compte est vide !");
            return;
        }
        else if (montant > solde) {
            System.out.println("Montant trop élevé !");
            return;
        }
        else
            solde -= montant;
    }

    public int getNombreInstances() {
        return nombreInstances;
    }

    void debug() {
        System.out.println("Compte " + numero + " : " + solde);
    }

    void debug(String nomFichier) {
        try {
            FileWriter fw = new FileWriter(nomFichier);
            BufferedWriter bw = new BufferedWriter(fw);
            PrintWriter fichier = new PrintWriter(bw);

            fichier.println("Compte " + numero + " : " + solde);
            fichier.close();
        }
        catch(IOException e) {
            System.out.println("Exception : " + e.getMessage());
        }
    }
}

```

Maintenant, le programme permettant de tester les méthodes *debug* :

```

public class TestCompte {
    public static void main (String Arguments []) {
        Compte2 compteAlain = new Compte2("ALAIN001", 1000.50);
        compteAlain.debug();
        compteAlain.debug("/tmp/alain.log");
    }
}

```

Nous trouvons une ligne particulière au début du fichier *Compte.java* :

```
import java.io.*;
```

Cette ligne est nécessaire pour inclure les classes *FileWriter*, *BufferedWriter* et *PrintWriter*. En effet, ces classes ne font pas partie des éléments de base du langage comme la classe *String*, mais appartiennent au framework fourni avec Java.

Nous trouverions aussi le traitement d'une exception si le fichier ne pouvait pas être ouvert. Nous reviendrons sur les exceptions plus tard.

2.1.3.6. Le passage des arguments à une méthode

En Java, le passage des arguments est toujours réalisé par valeur. Une méthode reçoit donc une copie de l'argument. Voici un exemple montrant le phénomène sur des types de base :

```
public class TestPassageArguments {
    public static void main (String Arguments []) {
        int a = 10;
        int b = 20;

        System.out.println("a: " + a + ", b: " + b);
        echangeAetB(a, b);
        System.out.println("a: " + a + ", b: " + b);
    }

    public static void echangeAetB(int a, int b) {
        int tmp = a;
        a = b;
        b = tmp;
        System.out.println("a: " + a + ", b: " + b);
    }
}
```

L'affichage écran de ce programme montre que l'échange entre *a* et *b* n'a pas eu lieu hors de la méthode *echangeAetB*. Par contre, si nous passons des objets comme arguments, la méthode travaillera sur les valeurs réelles et non pas une copie, car les objets sont passés par référence, c'est-à-dire par adresse. La méthode recevra une copie de la référence, mais cela n'aura aucune importance, car la valeur de l'adresse pointée par la référence sera toujours la même. Le même exemple que le précédent :

```
class Entier {
    public int entier = 0;
    public String toString() { return Integer.toString(entier); }
}

public class TestPassageArguments {
    public static void main (String Arguments []) {
        Entier a = new Entier();
        a.entier = 10;

        Entier b = new Entier();
        b.entier = 20;

        System.out.println("a: " + a + ", b: " + b);
        echangeAetB(a, b);
        System.out.println("a: " + a + ", b: " + b);
    }

    public static void echangeAetB(Entier a, Entier b) {
        int tmp = a.entier;
        a.entier = b.entier;
        b.entier = tmp;
        System.out.println("a: " + a + ", b: " + b);
    }
}
```

Cette fois, les valeurs ont bien été échangées, car la méthode a travaillé sur une copie des adresses mémoires des objets *a* et *b*. Il nous est facile de comprendre que cela revient au même.

Nous pouvons rencontrer un problème avec un certain type de classe : les classes immutables. Ces classes ne peuvent pas être modifiées. Pour les connaître, nous devons parcourir la documentation du framework Java. Par exemple, les classes *Integer*, *String*, etc. sont immutables.

L'exemple suivant met en évidence le problème :

```
public class TestPassageArguments {
    public static void main (String Arguments []) {
        Integer a = new Integer(10);
        Integer b = new Integer(20);

        System.out.println("a: " + a + ", b: " + b);
        echangeAetB(a, b);
        System.out.println("a: " + a + ", b: " + b);
    }

    public static void echangeAetB(Integer a, Integer b) {
        int tmp = a;
        a = b;
        b = tmp;
        System.out.println("a: " + a + ", b: " + b);
    }
}
```

2.1.3.7. Les variables locales

Une variable locale est une variable déclarée dans une méthode. Le corps de la méthode définit ce qu'on appelle la « portée » de la variable. Lorsque le programme sort de la méthode, la variable devient inaccessible. Cela entraîne les conséquences suivantes :

- La variable est un type de base : elle est immédiatement détruite et le contenu est perdu.
- La variable est une référence sur un objet : la variable locale est détruite, mais l'objet qu'elle pointait continue d'exister si cet objet est atteignable à un autre endroit du programme : c'est-à-dire qu'il est encore référencé par une variable valide. Si cet objet n'est plus référencé, le ramasse-miettes le supprimera dans un moment plus ou moins lointain.

Initialisation des variables locales

Les variables locales ne sont pas initialisées à 0 pour les types de base numériques ou à *null* pour les références. Une erreur sera générée par le compilateur lors de la première instruction qui essaiera de lire le contenu de la variable. Nous devons donc initialiser correctement toutes les variables locales lors de leur déclaration.

2.1.3.8. Les méthodes de classe

Tout comme les attributs de classe, il existe des méthodes de classe. Elles sont marquées comme *static*. Elles peuvent être utilisées par une instance de la classe ou directement par la classe, donc sans passer par une instance. Certaines limitations sont toutefois à noter :

- elles ne peuvent référencer que des attributs de classe,
- elles ne peuvent qu'appeler des méthodes de classe.

Beaucoup de classes spécialisées n'offrent que des méthodes de classe. Pour ce type de classes, une instantiation n'aurait guère de sens. Parmi ces classes, nous trouvons la classe *java.lang.Math*. Cette classe possède, parmi d'autres, une méthode statique nommée *int max(int, int)* qui retourne l'argument le plus grand entre les deux valeurs passées. Voici un exemple d'utilisation de cette méthode :

```

public class TestMathMax {
    public static void main (String Arguments []) {
        System.out.println("Valeur max entre 10 et 20: " + Math.max(10, 20));
    }
}

```

2.1.3.9. L'héritage

L'héritage est une grande avancée dans la production de code de qualité. Des études tendent à montrer que l'héritage permet de réduire de 30 à 40% la quantité de code à produire lors de la conception d'une application. L'héritage permet de concevoir des classes apparentées et de les organiser en une structure arborescente. L'héritage permet à une classe dérivée de récupérer les attributs et les méthodes de sa classe de base et d'en adapter le comportement pour une spécialisation. La relation entre une classe de base et une classe dérivée est « est une spécialisation de ». Le terme spécialisation est très important : nous ne dérivons pas une classe *Voiture* d'une classe *Avion* par convenance personnelle. La classe dérivée doit pouvoir être perçue comme une spécialisation de sa classe de base. Par exemple, une classe *Cercle* peut être dérivée d'une classe *ObjetGraphique* dans une application de dessin assisté par ordinateur. Cet héritage est correct, car un cercle est bien un objet graphique.

Java ne permet que l'héritage simple. Cela signifie qu'une classe dérivée ne peut posséder qu'une seule classe de base. Certains langages comme C++ ou Eiffel autorise l'héritage multiple.

La notion d'héritage ne se limite pas à un seul niveau. Une classe *A* peut être la classe de base d'une classe *B* qui elle-même peut être la classe de base d'une classe *C*. En général, la classe *A* est la plus généraliste. La classe *B* va apporter des informations supplémentaires à la classe *A* sous forme d'attributs supplémentaires, ajouter de nouveaux comportements sous la forme de nouvelles méthodes et peut-être modifier l'implémentation de certaines méthodes existantes. La même chose va être possible entre *B* et *C*. *C* sera donc la classe la plus spécialisée des trois.

La figure ci-dessous donne le diagramme UML des principes énoncés ci-dessous. *C* possède donc les attributs *a* et *d* et les méthodes *methode1* et *methode2*. Elle redéfinit toutefois la méthode *methode2* pour en adapter le comportement. Les méthodes sont toutes publiques, les attributs privés.

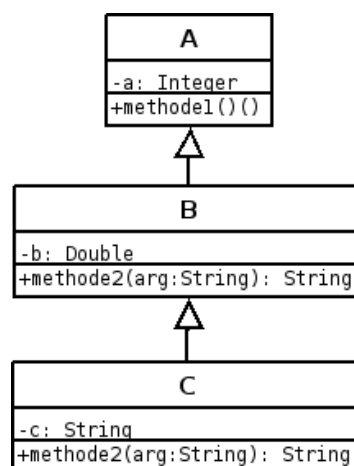


Figure 3.1. Héritage de classes

Voici le code Java correspondant :

Exemple

```
class A {
    Integer a = new Integer(5);

    public void methode1() {
        System.out.println( a );
    }
}

class B extends A {
    Double b = new Double(10.0);

    public String methode2(String arg) {
        return arg;
    }
}

class C extends B {
    public String methode2(String arg) {
        return "Methode redefinie dans la classe C : " + arg;
    }
}

public class TestHeritage {
    public static void main(String[] args) {
        A a = new A();
        B b = new B();
        C c = new C();

        System.out.print( "a.methode1() : " ); a.methode1();
        System.out.print( "b.methode1() : " ); b.methode1();
        System.out.println( "b.methode2() : " + b.methode2( "Un texte" ) );
        System.out.print( "c.methode1() : " ); c.methode1();
        System.out.println( "c.methode2() : " + c.methode2( "Un autre texte" ) );
    }
}
```

Nous pouvons constater que l'héritage est matérialisé par le mot clé « extends ». La classe *C* redéfinit la méthode *methode2* pour lui donner une implémentation différente. Le programme montre bien que le code exécuté pour la méthode *methode2* est différent selon le type d'instance. Ce concept est celui du polymorphisme dynamique.

2.1.3.9.1. Le polymorphisme dynamique

L'héritage en tant que tel ne servirait pas à grand-chose si nous n'avions pas la possibilité de modifier le comportement des objets en plus de leur ajouter de nouveaux attributs. L'héritage permet de jouer avec le polymorphisme dynamique. Nous avons déjà vu le polymorphisme statique qui donne la possibilité de créer des méthodes ayant le même nom. Le polymorphisme dynamique s'applique sur les méthodes ayant la même signature. Il permet d'appeler la bonne méthode en fonction du type de l'instance. Ce choix est réalisé lors de l'exécution du programme. Afin de bien comprendre le concept, observons ce morceau de code basé sur les classes *A*, *B* et *C* :

```
B o = new C();
o.methode2( "Une chaîne" );
```

La méthode appelée est la méthode de la classe *C*. Pourquoi ? Car le polymorphisme dynamique intervient dans l'appel. La variable est bien de type *B*, mais le compilateur sait que l'objet réel est de type *C*. Il va donc appeler la méthode de la classe *C*.

L'affectation d'une instance de type *C* à une variable de type *B* est possible, car *C* est également de type *B*. Il en possède toutes les caractéristiques. L'inverse n'est pas vrai. La classe *B* n'est pas de type *C*. Le code suivant produit une erreur de compilation avec le message « incompatible types » :

```
C o = new B();
```

Maintenant, examinons ce morceau de programme :

```
A o = new C();  
o.methode2( "Une chaîne" );
```

Nous obtenons une erreur de compilation lors de l'appel de la méthode *methode2* : « cannot find symbol ». La classe *C* est bien de *A* mais le type *A* n'a pas connaissance de l'existence de la méthode *methode2*. Ni d'ailleurs de l'existence de l'attribut *b*. Bien que l'instance référencée par *o* soit réellement un objet de type *C*, le compilateur la voit comme une instance de type *A*.

Modifions ce morceau de code :

```
A o = new C();  
B m = (C) o  
m.methode2( "Une chaîne" );
```

Cette fois, le code compile et fonctionne. Nous avons néanmoins été dans l'obligation de transtyper l'instance *o* pour indiquer au compilateur « de voir » *o* comme un type *B*. Le transtypage est souvent nécessaire en Java. Il n'est toutefois pas aussi dangereux que dans le langage C++, car la compilation vérifie si le transtypage est possible.

2.1.3.9.2. La classe *Object*

En Java, toutes les classes héritent au moins d'une classe : *Object*. La classe *Object* est située au sommet de la hiérarchie. La classe *A* hérite donc de la classe *Object*. *Object* fournit le minimum vital au système objet du langage Java. *Object* offre une interface publique contenant les méthodes polymorphes suivantes qui fournissent une implémentation par défaut :

- *public String toString()* : permet de représenter un objet sous la forme d'une chaîne de caractères. C'est cette méthode qui est appelée lorsque nous donnons une référence sur un objet dans la méthode *println*.
- *public boolean equals(Object)* : permet de comparer l'égalité de deux objets. La manière de déclarer que deux objets sont égaux dépend généralement de la représentation interne de chaque objet.
- *public int hashCode()* : permet de fournir un code de hachage pour l'objet.
- *public Object clone()* : permet de cloner l'objet courant. La méthode doit construire un nouvel objet puis le retourner à l'appelant.

Opérateur d'égalité sur des instances

En Java, nous ne devons pas comparer l'égalité entre deux objets à l'aide de l'opérateur `==`. Cet opérateur va comparer l'égalité sur la valeur des variables : l'adresse mémoire des objets. Deux objets *String* différents possédant tous les deux le texte « Texte » seront considérés comme

différents. La méthode *equals* héritée de la classe *Object* est présente pour fournir ce service. Elle doit être redéfinie pour la classe dont nous aurions à tester l'égalité.

L'exemple [TestObject.java](#) redéfinit la plupart des méthodes de la classe *Object*. Il met en application les explications sur les méthodes données ci-dessus.

Exemple : TestObject.java

```
class Objet implements Cloneable {
    int valeur;

    public Objet( int arg ) {
        valeur = arg;
    }

    public String toString() {
        return "Valeur : " + valeur;
    }

    public boolean equals( Object o ) {
        if ( o instanceof Object )
            if ( ( ( Objet ) o ).valeur == valeur ) return true;

        return false;
    }

    public Object clone() {
        return new Objet( valeur );
    }
}

class TestObject {
    public static void main( String[] args ) {
        Objet a = new Objet( 5 );
        Objet b = new Objet( 10 );

        if ( a.equals( b ) )
            System.out.println( "a est égal à b" );
        else
            System.out.println( "a est différent de b" );

        Objet c = (Objet) a.clone();

        if ( a.equals( c ) )
            System.out.println( "a est égal à c" );
        else
            System.out.println( "a est différent de c" );

        if ( a == c )
            System.out.println( "a == c" );
        else
            System.out.println( "a == c ne teste pas l'égalité des objets" );

        System.out.println( a ); // appel de a.toString()
    }
}
```

2.1.3.9.3. Le mot clé « super »

Nous aurons quelquefois le besoin d'accéder à la classe de base depuis une classe dérivée, notamment dans les constructeurs ou pour appeler une méthode spécifique à la classe de base, alors qu'elle existe également dans la classe dérivée.

super représente un accès à la classe mère. [TestHeritage.java](#) utilise le mot clé *super* dans les constructeurs des classes *B* et *C* et dans une méthode de la classe *C* :

Exemple 3.5. TestHeritage.java

```
class A {
    Integer a;

    public A( Integer arg ) {
        a = arg;
    }

    public void method1() {
        System.out.println( a );
    }
}

class B extends A {
    Double b;

    public B( Double arg1, Integer arg2 ) {
        super( arg2 ); // appel du constructeur de A
        b = arg1;
    }

    public String methode2(String arg) {
        return arg;
    }
}

class C extends B {
    public C( Double arg1, Integer arg2 ) {
        super( arg1, arg2 );
    }

    public void method1() {
        System.out.println( "Methode 'method1' redefinie dans la classe C" );
    }

    public String methode2(String arg) {
        return "Methode redefinie dans la classe C : " + arg;
    }

    public void testMethod1() {
        super.method1();
    }
}

public class TestHeritage {
    public static void main(String[] args) {
        A a = new A( new Integer( 5 ) );
        B b = new B( new Double( 10. ), new Integer( 5 ) );
        C c = new C( new Double( 20. ), new Integer( 8 ) );

        System.out.print( "c.testMethod1() : " ); c.testMethod1();
    }
}
```

2.1.3.10. Le transtypage

Le transtypage consiste à convertir un type en un autre type. Le transtypage n'est possible que si le type à obtenir est compatible avec le type à convertir. Java vérifie scrupuleusement cette compatibilité et lancera une exception en cas de problème.

2.1.3.10.1. Le transtypage implicite

Ce transtypage est effectué automatiquement par le compilateur. Par exemple, lorsque nous écrivons :

```
Object o = new A();
```

le compilateur comprend :

```
Object o = (Object) new A();
```

Le transtypage est automatique car la classe *A* possède *Object* comme classe de base. Un objet instancié à partir de la classe *A* peut être vu comme une instance de la classe *Object*.

2.1.3.10.2. Le transtypage explicite

En utilisant les classes *A*, *B* et *C*, expliquons le transtypage explicite :

```
A a = new C();  
B b = a; // erreur
```

Le compilateur génère une erreur car il ne sait pas que *a* référence un type *C*. Pour lui, le transtypage de *a* vers *b* est illégal. Pour compiler, nous devons indiquer au compilateur le transtypage à effectuer explicitement à l'aide de l'opérateur (*type*).

```
A a = new C();  
B b = (C) a; // correct
```

Le transtypage explicite n'autorise pas tout. Le compilateur vérifie à la compilation si la conversion est possible. Ecrire ceci générera une erreur :

```
A a = new C();  
B b = (String) a; // erreur
```

2.1.3.10.3. L'opérateur instanceof

Il peut être nécessaire de tester si une instance est bien d'un type particulier avant d'effectuer une opération. En effet, appeler une méthode sur une instance qui ne la possède pas provoquera une erreur à l'exécution. Ceci est particulièrement vrai lorsque des objets sont stockés dans un tableau. Soit un tableau qui contient des objets de type *A*. Nous voulons appeler dans une boucle la méthode *methode2*. L'appel de cette méthode ne sera possible que sur les types *B* et *C*. L'opérateur *instanceof* permet de vérifier le type d'un objet :

```
A [] tab = new A[5];  
tab[0] = new C();  
tab[1] = new A();  
tab[2] = new B();
```

```

tab[3] = new A();
tab[4] = new C();

for ( int i = 0; i < tab.length; i++ ) {
    if ( tab[i] instanceof B )
        tab[i].methode2();
}

```

Nous appelons la méthode *methode2* seulement si l'objet est de type *B*. Rappelons que les objets de type *C* sont également de type *B*, *A* et *Object*.

2.1.3.11. Les classes abstraites

Une classe abstraite est une classe qui ne peut pas être instanciée, car son implémentation n'est pas complètement terminée. Ces classes contiennent des concepts ou des mécanismes génériques que les classes dérivées devront compléter. Une classe qui hérite d'une classe abstraite est également abstraite si elle n'implémente pas toutes les méthodes de la classe de base. Un exemple de classe abstraite serait une classe *ObjetGraphique*. De cette classe seront dérivées les classes *Ellipse*, *Rectangle*, etc. La méthode *dessiner* de la classe *ObjetGraphique* ne peut pas être implémentée, car le code sera différent dans chaque classe dérivée. Fournir une implémentation pour la méthode *dessiner* dans la classe *ObjetGraphique* n'a pas de sens, car nous pourrions alors instancier un objet à partir de cette classe. *ObjetGraphique* est un concept permettant de manipuler tous les objets graphiques de manière générique. Une instance *ObjetGraphique* ne peut pas exister en tant que telle. *ObjetGraphique* sera donc une classe abstraite. Toutes les classes dérivées devront implémenter la méthode *dessiner* pour pouvoir être instanciables.

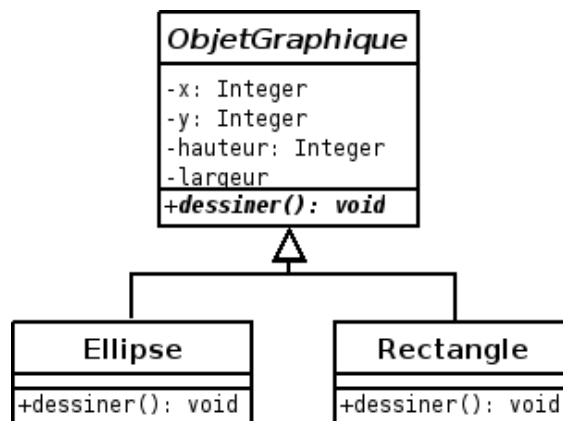


Diagramme UML - Classe abstraite

```

public abstract class ObjetGraphique {
    Integer x, y, hauteur, largeur;
    Color couleur;

    public ObjetGraphique( int x, int y, int hauteur, int largeur ) {
        this.x = new Integer( x );
        this.y = new Integer( y );
        this.hauteur = new Integer( hauteur );
        this.largeur = new Integer( largeur );
        couleur = Color.blue;
    }

    // méthode abstraite. Elle ne fournit pas d'implémentation.
    public abstract void dessiner();

    public Color getCouleur() {
        return couleur;
    }
}

```

```
}  
}
```

La classe *ObjetGraphique* est marquée comme *abstract*. Sa méthode *dessiner* est vide. Il est donc impossible d'écrire la ligne suivante sans erreur de compilation :

```
ObjetGraphique o = new ObjetGraphique( 10, 20, 100, 100 );
```

La classe *Ellipse* va hériter de la classe *ObjetGraphique*. Elle donne une implémentation pour la méthode *dessiner*. La classe *Ellipse* peut donc être instanciée :

```
public class Ellipse extends ObjetGraphique {  
  
    public Ellipse( int x, int y, int hauteur, int largeur ) {  
        super( x, y, hauteur, largeur );  
    }  
  
    public void dessiner() {  
        System.out.println( "Appel de la méthode Ellipse.dessiner()" );  
    }  
}
```

Le compilateur accepte de compiler le code :

```
ObjetGraphique o = new Ellipse( 10, 20, 100, 100 );  
o.dessiner();
```

Grace au polymorphisme dynamique, c'est bien la méthode *dessiner* de la classe *Ellipse* qui est appelée. Bien que la classe *ObjetGraphique* soit abstraite, c'est un type à part entière. Nous pouvons donc écrire :

```
ObjetGraphique [] tab = new ObjetGraphique[2];  
tab[0] = new Ellipse( 5, 30, 50, 100 );  
tab[1] = new Ellipse( 20, 40, 150, 300 )  
  
for ( int i = 0; i < tab.length; i++ ) {  
    tab[i].dessiner();  
}
```

Nous ne testons pas le type des objets dans ce morceau de code, car ils sont nécessairement du type *ObjetGraphique*.

2.1.3.12. Les interfaces

Le langage Java ne permet pas l'héritage multiple, mais uniquement l'héritage simple pour des raisons de simplicité d'implémentation. Néanmoins, le concept d'héritage multiple, souvent nécessaire, peut être approché avec les interfaces.

Le terme « interface » est porteur de plusieurs significations dans le langage Java. La première concerne l'ensemble des méthodes publiques d'une classe. Chaque classe possède une interface sans quoi son utilité serait discutable. La seconde est le concept d'interface représenté par le mot clé *interface*.

Une interface est une sorte de classe abstraite sans aucune implémentation. Toutes les méthodes d'une interface sont publiques. Une interface peut contenir des variables, mais avec les limitations suivantes :

- elles doivent être marquées comme *static*.
- elles doivent être constantes, donc non modifiables : elles sont également marquées comme *final*.

Une classe peut hériter d'une seule classe mais peut « implémenter » plusieurs interfaces. Implémenter une interface impose de fournir une définition pour chacune des méthodes déclarées dans l'interface. Une interface représente également un type au même titre qu'une classe concrète ou abstraite. L'utilisation des interfaces permet de développer avec une grande souplesse, car elles nous permettent de séparer proprement les concepts mis en œuvre de l'implémentation proprement dite. D'ailleurs, en programmation orientée objet, le principe de base est de programmer en pensant interface et non pas implémentation.

Concrètement, voici à quoi ressemble une interface en code Java :

```
interface Parler {  
    public void parler();  
}
```

Nous venons de déclarer une interface qui contient une seule méthode : *parler*. La déclaration utilise le mot clé *interface*. Maintenant, nous déclarons deux classes : *Humain* et *Animal*. Ces deux classes ne font pas partie de la même structure de classe⁸, mais elles implémentent l'interface *Parler*.

```
class Humain implements Parler {  
    String nom;  
    String prenom;  
  
    Humain( String nom, String prenom ) {  
        this.nom = nom;  
        this.prenom = prenom;  
    }  
  
    public String getNom() {  
        return nom;  
    }  
  
    public String getPrenom() {  
        return prenom;  
    }  
  
    public void parler() {  
        System.out.println( "Humain utilise la parole pour s'exprimer" );  
    }  
}  
  
class Animal implements Parler {  
    public Animal() {  
    }  
  
    public void parler() {  
        System.out.println( "Un animal n'utilise pas la parole pour s'exprimer"  
);  
    }  
}
```

Les classes *Humain* et *Animal* sont bien du type *Parler* :

```
Parler [] tab = new Parler[2];  
tab[0] = new Animal();  
tab[1] = new Humain("Caricand", "Jean-Michel");  
  
tab[0].parler();  
tab[1].parler();
```

⁸ Mis à part qu'elles héritent toutes les deux de la classe *Object*.

Par contre, en déclarant une variable sur un type *Parler*, des méthodes spécifiques au type *Humain* ne pourront pas être appelées sans un transtypage adéquat.

```
Parler jmc = new Humain( "Caricand", "Jean-Michel");  
//String nom = jmc.getNom(); // erreur  
String nom = ((Humain) jmc).getNom(); // correct
```

Une variable de type *Humain* ou de type *Animal* sait « parler » sans transtypage :

```
Humain jmc = new Humain( "Caricand", "Jean-Michel");  
Animal chien = new Animal();  
  
jmc.parler(); // correct  
chien.parler(); // correct
```

L'utilisation d'une interface est similaire à celle d'une classe abstraite avec quelques différences fondamentales concernant l'implémentation. Dans certains langages, C++ par exemple, une interface à la Java est simplement une classe abstraite pure, c'est-à-dire une classe dans laquelle toutes les méthodes sont abstraites. En C++, le mot clé *interface* n'est pas utile sachant que l'héritage multiple de classes est possible. En Java, c'est différent. Nous n'avons que l'héritage simple à disposition. Les classes abstraites sont utiles pour mettre en place des concepts, mais pas suffisantes pour implémenter des hiérarchies complexes. La notion d'interface à la Java est donc là pour combler cette insuffisance. Il faut également reconnaître que le mot clé *interface* a pour mérite d'être clair.

Les interfaces ne sont pas la panacée non plus, car elles forcent seulement une classe à respecter une interface publique, mais ne fournissent pas de facilités dans l'implémentation. Il est souvent nécessaire de dupliquer du code similaire dans des classes différentes, alors que nous aurions pu le factoriser en C++ dans une classe abstraite sans la limitation de l'héritage simple.

Nous pouvons donc implémenter plusieurs interfaces dans une classe. Voici un programme qui tire parti de cette fonctionnalité :

```
interface A {  
    public void methodeA();  
}  
  
interface B {  
    public void methodeB();  
}  
  
class MaClasse implements A, B {  
    public void methodeA() {  
        System.out.println( "methodeA" );  
    }  
  
    public void methodeB() {  
        System.out.println( "methodeB" );  
    }  
}  
  
public class TestInterface2 {  
    public static void main( String [] args ) {  
        MaClasse mc = new MaClasse();  
  
        mc.methodeA();  
        mc.methodeB();  
    }  
}
```

2.1.3.13. Les relations entre les classes

Une relation est un lien qui lie deux classes entre elles. De même que dans le monde réel, les objets d'une application interagissent les uns avec les autres, certains objets en contiennent d'autres, etc. L'héritage est une sorte de relation, mais une relation statique, figée à la compilation. Dans ce chapitre, nous allons parler des relations d'associations, de compositions et d'agrégations.

2.1.3.13.1. Les agrégations

La relation d'agrégation lie deux objets sous la forme composition/composant. Un objet est donc composé d'autres objets. L'agrégation est néanmoins une composition faible, car la durée de vie d'un composant ne dépend pas de celle du composé. Prenons par exemple une voiture. Une voiture « normale » possède 4 roues et un moteur. La voiture est donc un assemblage d'autres objets. Par contre, la destruction de l'objet voiture n'entraîne pas forcément la destruction des roues ou du moteur. La durée de vie des composants n'est donc pas dépendante de l'objet qui les possède.

Voici le diagramme UML décrivant une agrégation :

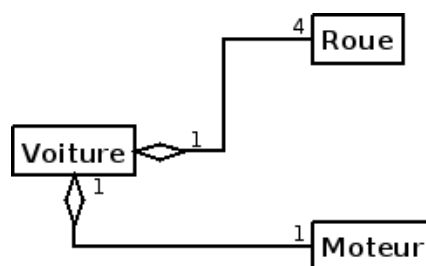


Diagramme UML - Relation d'agrégation

Le losange qui décrit une relation d'agrégation est vide. Nous pouvons observer avec les cardinalités qu'une voiture doit posséder 4 roues et 1 seul moteur.

Exemple : Agrégation - Implémentation en Java

```
public class Voiture {
    Moteur moteur;
    Roue [] roues = new Roue[4];
}
```

En fonction de la cardinalité de la relation, nous traduirons la relation par une référence ou par un tableau sur un ensemble de références. Dans le cas d'une agrégation, les objets entrants dans la composition du composé seront également référencés à un autre endroit du programme. La suppression du composé ne donnera donc pas au ramasse-miettes la possibilité de détruire les composants.

2.1.3.13.2. Les compositions

Une relation de composition est une agrégation forte. Dans ce type de relation, la durée de vie d'un composant est directement liée à celle du composé. Prenons par exemple un système de fichiers : un répertoire contient des fichiers. Un fichier ne peut appartenir qu'à un seul répertoire. Evidemment, il peut exister des liens symboliques dans d'autres répertoires qui pointent vers ce fichier, mais physiquement, le fichier est bien dans un seul répertoire. La destruction du répertoire entraînera la destruction de son contenu, donc du fichier également. Nous sommes bien dans une relation de composition.

Voici le diagramme UML décrivant une composition :

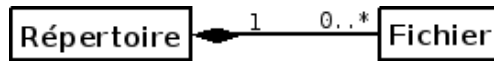


Diagramme UML - Relation de composition

Le losange qui décrit une relation de composition est plein. Comme dans la relation d'agrégation, nous pouvons indiquer les cardinalités de la relation. Dans le diagramme, nous comprenons qu'un fichier ne peut appartenir qu'à un seul répertoire et que ce répertoire peut contenir 0 ou un nombre infini de fichiers.

Exemple : Composition - Implémentation en Java

```
public class Répertoire {
    Vector<Fichier> fichiers;
}
```

Comme pour la relation d'agrégation, en fonction de la cardinalité de la relation, nous traduirons la composition par une référence ou par un tableau sur un ensemble de références. Dans le cas d'une composition, les objets entrants dans la composition du composé ne seront référencés que par le composé. La suppression du composé donnera donc au ramasse-miettes la possibilité de détruire les composants.

2.1.3.13.3. Les associations

Une relation d'association est une relation structurelle. Elle permet de mémoriser un lien d'utilisation entre deux classes.

Voici le diagramme UML décrivant une association :

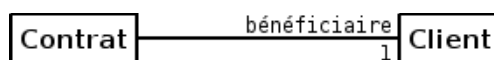


Diagramme UML - Relation d'association

La relation est décrite en UML par un trait simple. Une relation d'association ne décrit pas une relation de composé/composant. Elle indique seulement qu'il existe une relation d'utilisation entre une ou plusieurs classes. Il est souvent utile d'indiquer au moins le rôle d'une des classes : le client est le bénéficiaire du contrat. Nous pouvons également indiquer les cardinalités.

Exemple : Association - Implémentation en Java

```
public class Contrat {
    Client beneficiaire;
}
```

Tout comme les relations d'agrégation et de composition, la relation d'association sera traduite en Java par l'ajout d'une référence ou d'un tableau de références.

2.1.3.13.4. Conclusion

La notation UML permet de distinguer de nombreux types de relations entre les classes. Le Java n'intègre pas les concepts nécessaires pour formaliser les catégories de relations afin de les différencier. Si un code est difficile à comprendre, il faudra en général lire les diagrammes de classes UML.

2.1.3.14. Les packages

Au cours d'un développement, le nombre de classes peut s'accroître rapidement. Il est rapidement nécessaire de les organiser en packages pour :

- Organiser les classes par catégorie afin de les localiser plus rapidement.
- Gérer les conflits de noms : deux classes peuvent avoir le même nom si elles sont situées dans deux packages différents.
- Etablir des contrôles d'accès aux classes, à leurs méthodes et leurs attributs entre les packages.

Le framework Java contient de nombreux packages, comme par exemple le package *java.io* pour les classes destinées aux entrées/sorties. Toutes les classes relatives aux entrées/sorties ont été regroupées dans ce package. Pour pouvoir utiliser une classe située dans ce package, nous utilisons le mot clé *import*.

```
import java.io.*;
```

Note Le package *java.lang*

Avant de pouvoir utiliser une classe située dans un package, nous devons utiliser le mot clé *import*. Pourtant, nous avons déjà utilisé certaines classes comme les classes *Integer* et *String* sans utiliser ce mot clé. La raison est que les classes situées dans le package *java.lang* sont directement accessibles sans que nous ayons besoin de les importer.

Les packages Java sont organisés en utilisant le système de fichiers. L'arborescence d'un package est donc constituée de répertoires dans lesquels sont stockés les fichiers *.java* contenant les classes.

La création d'un package consiste à créer un répertoire. Le chemin vers ce répertoire devra être ajouté à la variable d'environnement *CLASSPATH* pour que le package soit accessible par le compilateur et la JVM.

Sous Unix, si le répertoire qui contient nos packages est */home/jmcaricand/alao/packages*, nous modifions la variable *CLASSPATH* de cette manière :

```
export CLASSPATH=/home/jmcaricand/alao/packages:$CLASSPATH
```

Sous Windows, si le répertoire est *C:\Documents and settings\jmcaricand\alao\packages* :

```
set CLASSPATH=c:\Documents and settings\jmcaricand\alao\packages;%CLASSPATH%
```

Si nous voulons créer le package outils, nous créons le répertoire :

```
# cd /home/jmcaricand/alao/packages
# mkdir outils
```

Dans le répertoire `/home/jmcaricand/alao/packages/ouils`, nous créons la classe *Test*. La première ligne du fichier *Test.java* sera *package ouils;* .

```
package ouils;

public class Test {
    public void message() {
        System.out.println( "Test.message" );
    }
}
```

Afin de conserver une compatibilité entre les systèmes d'exploitation, il est conseillé de toujours nommer les packages en minuscules.

Un package peut faire partie d'un autre package :

```
# mkdir ouils/patterns
```

Dans le répertoire *ouils/patterns*, nous créons une classe *Singleton* :

```
package ouils.patterns;

public class Singleton {
    ...
}
```



Nommer les packages

Un conseil de nommer les packages de manière à ne pas entrer en conflit avec d'autres packages. Une solution consiste à ajouter au nom du package le nom de domaine DNS de notre société. Par exemple, un package Graphique créé par une société possédant le nom de domaine *societe.com* pourrait être *com.societe.graphique*. L'utilisation du nom de domaine devrait être suffisante pour éviter les conflits, car il est théoriquement unique.

Le framework Java possède de nombreux packages. Les plus utilisés sont :

Tableau 3.1. Les packages Java

Package	Description
java.lang	Classes de base du langage
java.util	Divers outils (dates, collections...)
java.io	Classes concernant les entrées sorties
java.awt	Interface graphique
java.awt.event	Gestion des événements de l'interface graphique

2.1.3.15. Contrôle d'accès en Java

Le contrôle d'accès permet d'appliquer le concept d'encapsulation cher à la programmation orientée objet. Plusieurs types de contrôles peuvent être réalisés par Java en fonction des spécificateurs utilisés et de l'emplacement des classes dans les packages.

Nous avons déjà utilisé des spécificateurs d'accès aux attributs et aux méthodes dans l'écriture de certaines classes. Les spécificateurs d'accès sont les suivants :

- *public* : le membre est accessible sans restrictions.
- *protected* : le membre est accessible par toutes les classes de son package et toutes les classes dérivées.
- *private* : le membre n'est accessible par aucune autre classe.
- aucun spécificateur (« *friendly* ») : le membre est accessible uniquement des classes situées dans le même package.

Les classes possèdent également des spécificateurs d'accès :

- *public* : la classe est visible de partout. Elle doit être située dans un fichier qui porte son nom.
- aucun spécificateur (« *friendly* ») : la classe est visible uniquement dans son package.

Il n'est pas possible de redéfinir le spécificateur d'accès à une variable lors d'une relation d'héritage. Il est possible de redéfinir le spécificateur d'accès à une méthode lors d'une relation d'héritage. Toutefois, la redéfinition doit aller dans le sens de l'ouverture et non de la restriction.

Exemple :

```
package com.test;
public class ClasseA {
    protected String getDescription() {
        return "classe A";
    }
}
package com.test2;
import com.test.ClasseA;
public class ClasseB extends ClasseA {
    @Override
    public String getDescription() {
        return "classe B";
    }
}
```

2.1.3.16. Les exceptions

Certaines erreurs surviennent dans le déroulement d'un programme. Ces erreurs ne sont pas nécessairement le résultat du codage. Néanmoins, l'interception des erreurs non prévues est un plus qui est possible en Java grâce au mécanisme des exceptions. L'exemple le plus courant est la division d'un nombre par zéro. Dans les langages procéduraux traditionnels, ce type d'erreur doit être prévu par le développeur sous peine d'un arrêt brutal du programme. Dans la plupart des cas, le contrôle est fait à la saisie du diviseur. Les langages à objets modernes intègrent des outils qui nous permettent de contrôler de manière propre ces erreurs. C'est le rôle des exceptions.

Une exception est un événement anormal qui survient dans le déroulement du programme. Nous avons alors les possibilités suivantes :

- ne rien faire : le programme s'arrête avec un message d'erreur précis sur la cause de l'exception.
- gérer l'exception dès qu'elle survient pour en corriger la cause.
- laisser l'exception se propager pour être traitée ailleurs dans le programme.

```

public class Division
{
    public static void main (String[] args)
    {
        int numerator = 4;
        int denominator = 0;

        int quotient = numerator / denominator;

        System.out.printf ("%d / %d = %d\n", numerator, denominator, quotient);

        System.out.println ("Fin du programme");
    }
}

```

Dans le programme précédent, nous allons avoir un problème : le dénominateur à la valeur zéro. La ligne

```
int quotient = numerator / denominator;
```

va provoquer une erreur qui va interrompre le programme, car la division par zéro va générer une exception. Nous ne verrons donc pas l'affichage du texte « Fin du programme ».

La JVM va quand même nous aider dans la correction de l'erreur en nous indiquant la cause de celle-ci. Le programme s'arrête sur une exception de type *ArithmeticException* en nous indiquant également le numéro de ligne.

Nous pouvons prévoir l'erreur avec un traitement procédural :

```

public static void main (String[] args)
{
    int numerator = 4;
    int denominator = 0;

    if (denominator != 0)
    {
        int quotient = numerator / denominator;
        System.out.printf ("%d / %d = %d\n", numerator, denominator, quotient);
    }
    else
    {
        System.out.println ("Vous ne pouvez pas faire une division par zéro
!");
    }

    System.out.println ("Fin du programme");
}

```

Cette manière de procéder ne permet pas d'intercepter toutes les erreurs possibles. Certaines erreurs systèmes ne pourront pas de plus être traitées avec un *if-else*. Heureusement, les langages à objets intègrent tous un traitement des erreurs plus générique : une gestion des erreurs à travers des exceptions.

Une exception est un objet qui transporte des informations sur un type d'erreur particulier, jusqu'à un point du programme appelé un gestionnaire d'exceptions. Nous devons seulement indiquer au compilateur quelle partie du programme nous désirons surveiller. Le bloc d'instructions à surveiller est défini dans un *try { ... }*. Juste après ce bloc *try*, nous trouvons les gestionnaires d'exceptions. Ces gestionnaires sont matérialisés par des blocs *catch(type d'exception) { ... }*. Nous écrivons « les gestionnaires », car nous pouvons traiter différemment les exceptions que nous aurions pu rencontrer dans le *try { ... }*.

Une exception est un objet qui hérite de la classe *java.lang.Throwable*. Seules les classes qui héritent de cette classe pourront être envoyées par une instruction en erreur à l'aide de l'instruction *throw*, puis être capturées par l'instruction *catch*. L'ensemble du framework Java est basé sur l'utilisation des exceptions et la connaissance de celles-ci est donc un impératif dans ce langage.

Reprenons le programme précédent en utilisant les exceptions :

```
public static void main (String[] args)
{
    int numerator = 4;
    int denominator = 0;

    try
    {
        int quotient = numerator / denominator;
        System.out.printf ("%d / %d = %d\n", numerator, denominator, quotient);
    }
    catch (ArithmeticException exception)
    {
        System.out.println ("Vous ne pouvez pas faire une division par zéro
!");
    }

    System.out.println ("Fin du programme");
}
```

Nous mettons les instructions à surveiller dans un bloc *try { ... }* puis indiquons au compilateur qu'il faut traiter les exceptions du type *ArithmeticException* d'une manière spécifique. La ligne

```
System.out.println ("Fin du programme");
```

ne fait pas partie du bloc *catch* car elle devra toujours être exécutée.

Nous pouvons observer que la gestion des erreurs est plus lisible qu'un *if-else* et surtout plus sûre. Pour être certains de capturer toutes les exceptions possibles, nous pouvons ajouter un *catch* supplémentaire :

```
public static void main (String[] args)
{
    int numerator = 4;
    int denominator = 0;

    try
    {
        int quotient = numerator / denominator;
        System.out.printf ("%d / %d = %d\n", numerator, denominator, quotient);
    }
    catch (ArithmeticException exception)
    {
        System.out.println ("Vous ne pouvez pas faire une division par zéro
!");
    }
    catch (Exception exception)
    {
        System.out.println ("Une exception imprévue !");
    }

    System.out.println ("Fin du programme");
}
```


En ajoutant des *catch* successifs, nous devons prendre soin de traiter les exceptions les plus spécialisées en premier pour terminer avec les plus génériques. En effet, l'instruction *catch* se base sur la hiérarchie des classes d'exceptions pour appliquer les traitements. Un seul bloc *catch* est exécuté à la suite d'un *try*.

Observons la hiérarchie des classes d'exceptions de Java :

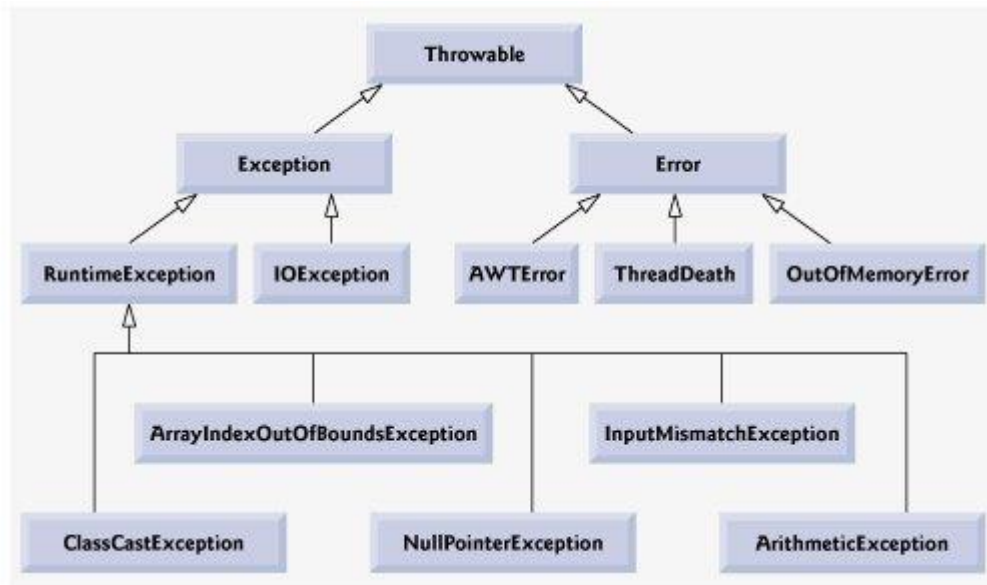


Diagramme UML - Relation d'association

Toutes les classes d'exceptions héritent de *Throwable*. Nous avons ensuite deux branches distinctes :

- *Error* : les exceptions de type *Error* sont graves et ne peuvent pas être traitées par l'application. Le programme est stoppé immédiatement.
- *Exception* : les exceptions de type *Exception* doivent être capturées par un gestionnaire d'exceptions. D'ailleurs, le compilateur impose que les méthodes pouvant envoyer ces exceptions soient appelées dans un bloc *try*. Il génère une erreur de compilation si ce n'est pas le cas.

Située dans la branche *Exception*, nous avons également une classe *RuntimeException*. Les méthodes envoyant ce type d'exception n'ont pas besoin d'être dans un bloc *try*. Ces exceptions sont générées, par exemple, lors d'un accès à un index non valide d'un tableau, etc.

Les exceptions métiers

Les exceptions métiers, c'est-à-dire les exceptions que nous créons pour nos besoins, sont en général dérivées de la classe *Exception*.

2.1.3.16.1. Les exceptions métiers

Une exception métier une classe d'exception qui doit transporter des informations sur des erreurs survenant dans une application que nous développons. Avant de pouvoir utiliser une exception métier, nous devons créer une classe spécialisée comme suit :

- Elle doit hériter de la classe *Exception*. De cette manière, elle devra être capturée.
- Le constructeur de notre classe doit prendre au minimum un argument de type *String* qui décrit le type d'erreur rencontrée.
- Elle doit, si nécessaire, surcharger la méthode *getMessage* qui permet de récupérer le message d'erreur transporté.

- Elle doit contenir les méthodes et les attributs pouvant être utiles au traitement de l'erreur.

Une fois définie, la classe d'exception doit être « lancée » lorsque nous rencontrons une situation exceptionnelle dans le déroulement de notre application. Pour lancer une exception, nous utilisons l'instruction *throws*. Cette instruction attend un objet du type *Throwable*. Le compilateur refusera de compiler si la classe passée à *throws* ne respecte pas cette condition.

Une autre remarque s'impose au sujet de l'instruction *throws* : une méthode susceptible de lancer une exception doit posséder un spécificateur *throws* suivi de la liste des exceptions qui peuvent être lancées.

2.1.3.16.2. L'instruction *finally*

Le modèle objet de C++ permet l'utilisation des destructeurs. Un destructeur est une méthode spéciale qui est appelée lors de la destruction d'un objet. En général, nous libérons toutes les ressources allouées par l'objet dans cette méthode. Cette manière de faire n'est pas possible en Java, car la notion de destructeur n'existe pas. Il existe bien une méthode *finalize* mais elle est peu utile. D'ailleurs, le framework Java y fait rarement appel. Le problème avec Java, c'est que la destruction des objets est de la responsabilité du ramasse-miettes. Nous ne pouvons pas savoir à quel moment un objet sera réellement libéré⁹. Impossible donc de restituer les ressources systèmes dans la méthode *finalize*.

Le problème devient épineux lorsqu'un objet ouvre un fichier dans un bloc *try* et qu'une exception est levée. Le point d'exécution du programme peut ne jamais exécuter les instructions qui devraient fermer le fichier ouvert par l'objet. Heureusement, nous avons à notre disposition l'instruction *finally*. Le bloc *finally* est placé après le ou les blocs *catch* et surtout, il est toujours exécuté.

finally sert donc principalement à remettre en état la cohérence d'un traitement qui aura été perturbé par la levée d'une exception.

2.1.3.16.3. Exemple complet

Mettons en pratique l'ensemble des remarques à travers un petit exemple qui reprend les principales remarques vues précédemment.

```
class AgeNonValideException extends Exception {
    public AgeNonValideException( String message ) {
        super( message ) ;
    }
}

class Personne {
    Integer age = new Integer( 0 );

    public Personne() {
    }

    public void setAge( Integer age ) throws AgeNonValideException {
        if ( age < 0 || age > 130 ) {
            throw new AgeNonValideException( "L'age \"" + age + "\" est en " +
                                                "dehors des limites autorisées" );
        }

        this.age = age;
    }
}

public class UtiliserException {
```

⁹ Il peut ne jamais être libéré si la mémoire est encore vaste.

```

public static void main( String [] args ) {
    Personne p = new Personne();

    try {
        p.setAge( new Integer( 150 ) );
        System.out.println( "Age modifié pour p" );
    }
    catch ( AgeNonValideException e ) {
        System.out.println( "Une exception a été levée pour la modification
de l'age de p :\n" +
                                "Le message transporté par l'exception est : "
+ e.getMessage() );
    }
    finally {
        System.out.println( "Le bloc finally est toujours exécuté !" );
    }
}

```

Nous pouvons remarquer que la méthode *setAge* annonce qu'elle est susceptible de lever une exception. Dans cette méthode, l'instruction *throw* lance une exception créée in-situ avec un message qui sera transporté jusqu'au gestionnaire chargé de la traiter. La méthode *setAge* doit être dans un bloc *try*, car l'exception *AgeNonValideException* dérive de la classe *Exception*. Le compilateur refusera de compiler le programme si ce n'est pas le cas. Dans le bloc *catch*, nous affichons la raison de l'erreur. Le bloc *finally* est exécuté, qu'une exception soit levée ou non.

2.1.4. Exercices

2.1.4.1. Exercice : Héritage

Créez une classe *Date* qui permettra de stocker le jour, le mois et l'année. Définissez les constructeurs qui semblent intéressants, ainsi que les méthodes permettant d'accéder aux valeurs encapsulées dans la classe.

Cette classe dispose des propriétés suivantes :

- jour
- mois
- année

Implémentez une méthode *afficher* dont le prototype est :

```
public void afficher();
```

Cette méthode affiche la date sous la forme « jour/mois/année ».

Implémenter une méthode *controlerValiderDate()* permettant de vérifier la validité de la date à la création et à la modification de l'une de ses propriétés. Le contrôle est réalisé sur les jours et les mois. Pour les années, la plage valide va de 1970 à 2050 inclus. Les années bissextiles sont prises en compte pour le contrôle du nombre de jour pour le mois de février.

A partir d'une instance de la classe *Date*, nous devons pouvoir :

- Tester l'égalité avec une autre instance de type *Date*,

- Comparer cette instance avec une instance de type *Date*,
- Créer une nouvelle instance de type *Date*.

Ecrivez le code nécessaire.

Enfin, créez une classe *Evenement* qui hérite de *Date*. Cette classe possède une propriété supplémentaire qui décrit l'événement sous la forme d'un texte. Toutes les remarques concernant la classe *Date* sont valides pour cette nouvelle classe : égalité, comparaison, etc.

2.1.4.2. Exercice : Classe abstraite

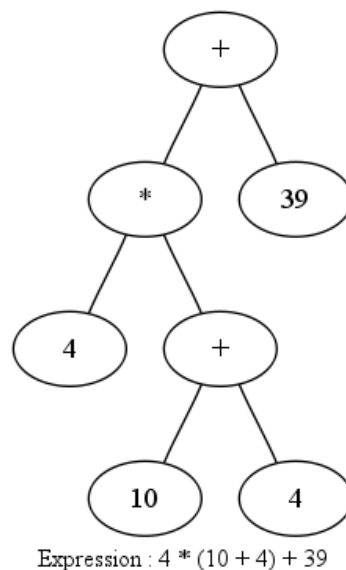
Définissez une classe abstraite *Forme* qui inclut des données membres protégées (*x* et *y*) pour la position de la forme, une méthode publique *deplacer* pour déplacer la forme et une méthode publique *afficher* pour afficher « textuellement » la forme. Dérivez la classe pour créer des *Lignes*, des *Cercles* et des *Rectangles*. Vous pouvez représenter une ligne et un rectangle avec 2 points, un cercle avec un centre et un rayon. Implantez la méthode *toString()* pour toutes les classes.

Ecrivez un programme qui permette de tester toutes les fonctionnalités de vos classes. Les instances de classes devront être stockées dans une liste capable de gérer des éléments de type *Forme*.

2.1.4.3. Exercice : Expression mathématique

Soit l'expression mathématique suivante : $4 * (10 + 4) + 39$

Trouvez les classes et les méthodes nécessaires pour représenter cette expression. Nous rappelons qu'une expression de ce type peut être représentée sous la forme d'un arbre.



Utilisez les notions d'héritage, d'abstraction et de polymorphisme dynamique dans la solution proposée. Justifiez vos choix, si nécessaire.

2.1.4.4. Exercice : Exceptions

Quel est le type d'exception qui sera lancé par chacune des lignes suivantes :

A. `Integer.parseInt("26.2");`

- B. `String s; s.indexOf('a');`
- C. `String s = "hello"; s.charAt(5);`

2.1.4.5. Exercice : Bloc try/catch

Ecrire un bloc `try ... catch` qui lance une exception si la valeur d'une variable *X* est inférieure à zéro. L'exception doit être une instance de la classe *Exception*, et, quand elle est interceptée, le message retourné doit imprimer le texte suivant : « ERREUR : Valeur négative. ».

2.1.5. Solutions

2.1.5.1. Héritage

Solution pour l'exercice « Héritage ».

```
import java.util.Calendar;

public class Date {
    private int annee;
    private int mois;
    private int jour;

    public int getAnnee() {
        return annee;
    }

    public void setAnnee(int annee) {
        if (this.controlerValiditeDate(annee, this.getMois(), this.getJour())) {
            this.annee = annee;
        }
    }

    public int getMois() {
        return mois;
    }

    public void setMois(int mois) {
        if (this.controlerValiditeDate(this.getAnnee(), mois, this.getJour())) {
            this.mois = mois;
        }
    }

    public int getJour() {
        return jour;
    }

    public void setJour(int jour) {
        if (this.controlerValiditeDate(this.getAnnee(), this.getMois(), jour)) {
            this.jour = jour;
        }
    }

    protected Date(int annee, int mois, int jour) {
        super();
        if (this.controlerValiditeDate(annee, mois, jour)) {
            this.annee = annee;
            this.mois = mois;
            this.jour = jour;
        }
    }

    private boolean isAnneeBissextiles (int annee) {
        Calendar cal = Calendar.getInstance();
        cal.set(Calendar.YEAR, annee);
        return cal.getActualMaximum(Calendar.DAY_OF_YEAR) > 365;
    }

    public void afficher() {
        System.out.println(this.getJour() + "/" + this.getMois() + "/" +
this.getAnnee());
    }
}
```

```

private boolean controlerValiditeDate(int annee, int mois, int jour) {
    if (annee < 1970 || annee > 2050) {
        return false;
    }

    if (mois < 1 || mois > 12) {
        return false;
    }

    switch (mois) {
        case 1: // janvier
            return jour >= 1 && jour <= 31;

        case 2: // février
            if (this.isAnneeBissextiles(annee)) {
                return jour >= 1 && jour <= 29;
            }
            else {
                return jour >= 1 && jour <= 28;
            }

        case 3: // mars
            return jour >= 1 && jour <= 31;

        case 4: // avril
            return jour >= 1 && jour <= 30;

        case 5: // mai
            return jour >= 1 && jour <= 31;

        case 6: // juin
            return jour >= 1 && jour <= 30;

        case 7: // juillet
            return jour >= 1 && jour <= 31;

        case 8: // aout
            return jour >= 1 && jour <= 31;

        case 9: // septembre
            return jour >= 1 && jour <= 30;

        case 10: // octobre
            return jour >= 1 && jour <= 31;

        case 11: // novembre
            return jour >= 1 && jour <= 30;

        case 12: // décembre
            return jour >= 1 && jour <= 31;

        default:
            return false;
    }
}

public Date clone() {
    return new Date(this.getAnnee(), this.getMois(), this.getJour());
}

@Override
public boolean equals(Object obj) {
    if (!(obj instanceof Date)) {
        return false;
    }

    Date d2 = (Date) obj;
    return this.getAnnee() == d2.getAnnee() && this.getMois() == d2.getMois() &&
this.getJour() == d2.getJour();
}

public void comparer(Date d) {
    if (this.getAnnee() != d.getAnnee()) {
        System.out.println("Les années sont différentes");
    }
    if (this.getMois() != d.getMois()) {
        System.out.println("Les mois sont différents");
    }
    if (this.getJour() != d.getJour()) {
        System.out.println("Les jours sont différents");
    }
}
}

```

```

public class Evenement extends Date {
    private String evenement;

    protected Evenement(int annee, int mois, int jour, String evenement) {
        super(annee, mois, jour);
        this.evenement = evenement;
    }

    public String getEvenement() {
        return evenement;
    }

    public void setEvenement(String evenement) {
        this.evenement = evenement;
    }

    @Override
    public void afficher() {
        System.out.print("Evenement: " + this.getEvenement() + " Date: ");
        super.afficher();
    }

    @Override
    public Evenement clone() {
        return new Evenement(this.getAnnee(), this.getMois(), this.getJour(),
this.getEvenement());
    }

    @Override
    public boolean equals(Object obj) {
        if (super.equals(obj)) {
            Evenement e = (Evenement) obj;
            return this.getEvenement().equals(e.getEvenement());
        }
        else {
            return false;
        }
    }

    public void comparer(Evenement e) {
        super.comparer(e);
        if (!this.getEvenement().equals(e.getEvenement())) {
            System.out.println("Les événements sont différentes");
        }
    }
}

```

2.1.5.2. Classe abstraite

Solution pour l'exercice « Classe abstraite ».

Note : l'exercice parle de position X et Y. Pour des raisons de simplicité, le but étant de montrer l'utilisation d'une classe abstraite, j'utilise le type « int » pour décrire les points.

```

package forme;
public abstract class Forme {

    protected int x;
    protected int y;

    public abstract void deplacer (int newX, int newY);
    public void afficher () {
        System.out.println (this.getDescr());
    }
    protected abstract String getDescr();
    protected void setX(int x) {
        this.x = x;
    }
    protected void setY(int y) {
        this.y = y;
    }
    protected int getX() {
        return x;
    }
}

```

```

    }
    protected int getY() {
        return y;
    }
    @Override
    public String toString() {
        return "Ceci est une forme de type: ";
    }
}

package forme;
public class Ligne extends Forme {
    public Ligne(int debut, int fin) {
        super();
        this.setX(debut);
        this.setY(fin);
    }

    @Override
    public void deplacer(int debut, int fin) {
        this.setX(debut);
        this.setY(fin);
    }

    @Override
    protected String getDescr ( ) {
        return "Ligne, début: " + this.getX() + " fin: " + this.getY();
    }
    @Override
    public void afficher() {
        super.afficher();
    }
    @Override
    public String toString() {
        return super.toString() + this.getDescr();
    }
}

package forme;
public class Rectangle extends Forme {
    public Rectangle(int coinHaut, int coinBas) {
        super();
        this.setX(coinHaut);
        this.setY(coinBas);
    }

    @Override
    public void deplacer(int coinHaut, int coinBas) {
        this.setX(coinHaut);
        this.setY(coinBas);
    }

    @Override
    protected String getDescr ( ) {
        return "Rectangle, coin haut: " + this.getX() + " coin bas: " + this.getY();
    }
    @Override
    public void afficher() {
        super.afficher();
    }
    @Override
    public String toString() {
        return super.toString() + this.getDescr();
    }
}

package forme;
public class Cercle extends Forme {
    public Cercle(int centre, int rayon) {
        super();
        this.setX(centre);
        this.setY(rayon);
    }
    @Override
    public void deplacer(int centre, int rayon) {
        this.setX(centre);
        this.setY(rayon);
    }
    @Override
    protected String getDescr ( ) {
        return "Cercle, centre: " + this.getX() + " rayon: " + this.getY();
    }
}

```



```

    }

    @Override
    public void afficher() {
        super.afficher();
    }

    @Override
    public String toString() {
        return super.toString() + this.getDescr();
    }
}

package forme;

import java.util.ArrayList;
import java.util.List;

public class Test {

    public static void main(String[] args) {
        List<Forme> formes = new ArrayList<Forme>();
        formes.add(new Ligne(3, 5));
        formes.add(new Rectangle(2, 4));
        formes.add(new Cercle(1, 3));

        for (Forme f : formes) {
            f.afficher();
            f.deplacer(4, 6);
            System.out.println("Forme après le déplacement: " + f.toString());
        }
    }
}

```

2.1.5.3. Expression mathématique

Solution pour l'exercice « Expression mathématique ». Elle fait intervenir l'héritage, les classes abstraites et les méthodes polymorphes.

```

abstract class Noeud {
    abstract public double calculer();
}

class NoeudNumerique extends Noeud
{
    public NoeudNumerique (double num) {
        this.num = num;
    }

    public double calculer () {
        System.out.println("Noeud numérique " + num);
        return num;
    }

    private double num;
};

abstract class NoeudBinaire extends Noeud
{
    public NoeudBinaire (Noeud gauche, Noeud droit) {
        this.gauche = gauche;
        this.droit = droit;
    }

    protected Noeud gauche;
    protected Noeud droit;
};

class NoeudAddition extends NoeudBinaire
{
    public NoeudAddition (Noeud gauche, Noeud droit) {
        super(gauche, droit);
    }

    public double calculer () {
        System.out.println("Addition");
    }
}

```

```

        return gauche.calculer() + droit.calculer();
    }
};

class NoeudMultiplication extends NoeudBinaire
{
    public NoeudMultiplication (Noeud gauche, Noeud droit) {
        super(gauche, droit);
    }

    public double calculer () {
        System.out.println("Multiplication");
        return gauche.calculer() * droit.calculer();
    }
};

public class Calculatrice {
    public static void main(String[] args) {
        Noeud n1 = new NoeudNumerique(10.0);
        Noeud n2 = new NoeudNumerique(5.0);
        Noeud n3 = new NoeudAddition(n1, n2);
        Noeud n4 = new NoeudNumerique(5);
        Noeud n5 = new NoeudMultiplication(n3, n4);

        double resultat = n5.calculer();

        System.out.println("Résultat de ( 10 + 5 ) * 5 : " + resultat);
    }
}

```

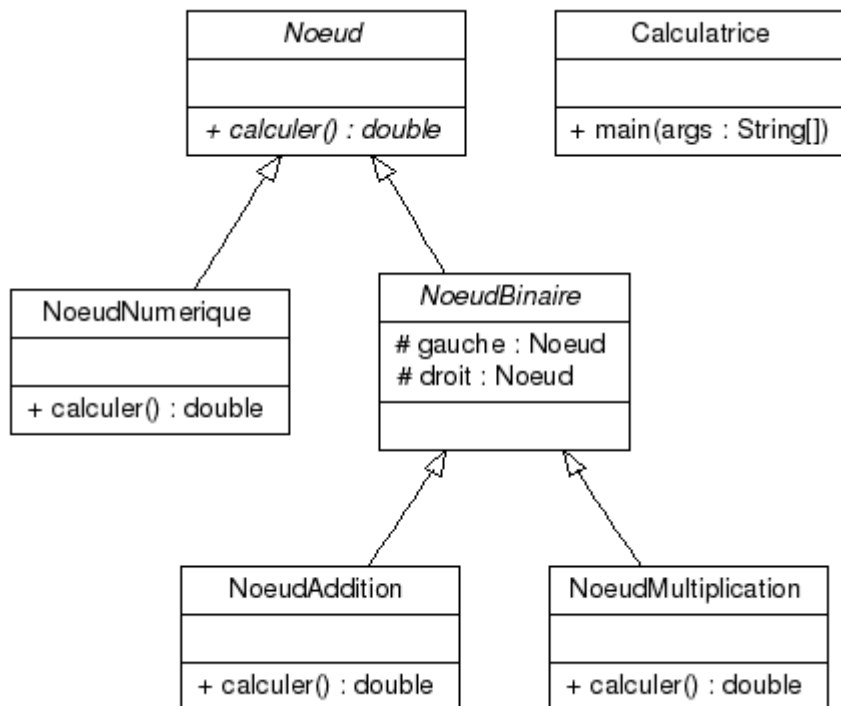


Diagramme UML – Expression

2.1.5.4. Exceptions

- A. java.lang.NumberFormatException: For input string: "26.2"
 ➔ un entier ne comporte pas de partie décimale, que ce soit avec un point ou une virgule.
- B. java.lang.Error: Unresolved compilation problem: The local variable s may not have been initialized
 ➔ La variable s est instanciée mais aucune valeur ne lui a été assignée. String n'étant pas un type primitif cela génère une erreur.

- C. `java.lang.StringIndexOutOfBoundsException`: String index out of range: 5
→ les chaînes de caractères sont indexées à partir de 0. 5 représente donc la 6^e lettre. Hors « hello » n'en compte que 5.

2.1.5.5. Bloc try/catch

Solution pour l'exercice « Bloc try/catch ».

```
package trycatch;

public class TryCatchTest {

    public static void main(String[] args) {
        try {
            int x = -1;
            if (x < 0) {
                throw new ValeurNegativeException (x);
            }
        } catch (ValeurNegativeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

2.2 Les collections

2.2.1. Introduction

Les collections sont des conteneurs dans lesquels nous pouvons ajouter des objets. Toutes les collections possèdent les opérations de bases suivantes :

- insertion d'un objet
- suppression d'un objet
- récupération de la référence sur un objet

2.2.1.1. Les collections supportées par le framework Java

Les tableaux de base ont des limites. L'une d'elles est très contraignante : la taille d'un tableau est fixée à sa création. Les collections permettent de lever cette contrainte en apportant de nombreuses autres facilités. Comme les tableaux de base, les éléments doivent tous être du même type.

Les collections offrent plusieurs types d'interfaces en fonction des caractéristiques attendues.

1. Tableau 4.1. Interfaces du framework des collections

Interface	Description
Collection	C'est l'interface racine de toutes les autres interfaces.
Set	Interface d'une collection qui ne peut pas contenir deux fois le même objet.
List	Interface d'une collection ordonnée. Les éléments peuvent être dupliqués.
Map	Interface d'une collection de type dictionnaire. Les clés ne peuvent pas être dupliquées.

Interface	Description
Queue	Interface d'une collection de type « file », comme les files de type premier entré, premier sorti.

Depuis la version 5 de Java, les collections génériques sont supportées. Nous pouvons donc spécifier le type des objets qu'une collection peut manipuler. Avant les types génériques, les collections ne pouvaient accueillir que des *Object*. Cette limitation forçait le développeur à faire de nombreux transtypages lorsqu'il accédait à un élément.

Ajouter un objet avec un type incorrect à une collection génère maintenant une exception de type *ClassCastException*.

2.2.1.2. L'interface List

L'interface *List* est implémentée par de nombreuses classes dont *ArrayList*, *LinkedList* et *Vector*. Nous pouvons accéder aux éléments en utilisant les indices traditionnels comme avec les tableaux de base. *LinkedList* implémente une séquence avec une liste chaînée alors que *ArrayList* et *Vector* sont très similaires aux tableaux. La différence entre *ArrayList* et *Vector* est que les méthodes de la classe *Vector* sont synchronisées, donc utilisables sans modifications dans des threads, alors que les méthodes de *ArrayList* ne le sont pas. Par contre, *ArrayList* est plus rapide que *Vector*.

Le programme suivant montre quelques possibilités sur les *List*. La classe d'implémentation choisie est *ArrayList*. Nous utilisons un itérateur pour parcourir les éléments de la liste dans la méthode *enleverCouleurs*. Nous observons l'utilisation des types génériques dans la déclaration des variables *liste* et *enleverListe*.

```
import java.util.List;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Iterator;

public class CollectionTest
{
    private static final String[] colors =
        { "MAGENTA", "ROUGE", "BLANC", "BLEU", "CYAN" };
    private static final String[] enleverCouleurs =
        { "ROUGE", "BLANC", "BLEU" };

    // crée un ArrayList, ajoute des couleurs et le manipule
    public CollectionTest()
    {
        List< String > liste = new ArrayList< String >();
        List< String > enleverListe = new ArrayList< String >();

        // ajoute des éléments
        for ( String color : colors )
            liste.add( color );

        // ajoute les éléments de enleverCouleurs à enleverListe
        for ( String color : enleverCouleurs )
            enleverListe.add( color );

        System.out.println( "ArrayList: " );

        // imprime le contenu de liste
        for ( int count = 0; count < liste.size(); count++ )
```

```

        System.out.printf( "%s ", liste.get( count ) );

        // supprime les éléments de liste présents dans enleverListe
        enleverCouleurs( liste, enleverListe );

        System.out.println( "\n\nArrayList après appel de enleverCouleurs: " );

        // imprime le contenu de liste
        for ( String color : liste )
            System.out.printf( "%s ", color );
    }

    // supprime les couleurs spécifiées dans collection2 de collection1
    private void enleverCouleurs(
        Collection< String > collection1, Collection< String > collection2 )
    {
        // récupère un itérateur
        Iterator< String > iterator = collection1.iterator();

        // boucle sur les éléments
        while ( iterator.hasNext() )

            if ( collection2.contains( iterator.next() ) )
                iterator.remove(); // supprimer la couleur
    }

    public static void main( String args[] )
    {
        new CollectionTest();
    }
}

```

Ce programme montre les possibilités fournies par la classe *LinkedList* :

```

import java.util.List;
import java.util.LinkedList;
import java.util.ListIterator;

public class ListTest
{
    private static final String couleurs[] = { "noir", "jaune",
        "vert", "bleu", "violet", "argent" };
    private static final String couleurs2[] = { "or", "blanc",
        "marron", "bleu", "gris", "argent" };

    // construction et manipulation des deux listes.
    public ListTest()
    {
        List< String > liste1 = new LinkedList< String >();
        List< String > liste2 = new LinkedList< String >();

        // ajoute des elements a liste1
        for ( String color : couleurs )
            liste1.add( color );

        // ajoute des elements a liste2
        for ( String color : couleurs2 )
            liste2.add( color );

        liste1.addAll( liste2 ); // concatenation des listes
        liste2 = null;          // libere liste2
        imprimerListe( liste1 ); // imprime liste1

        convertirEnMajuscules( liste1 ); // conversion en majuscules
    }
}

```

```

        imprimerListe( liste1 );          // imprime liste1

        System.out.print( "\nSuppression de certains elements..." );
        enleverElements( liste1, 4, 7 );
        imprimerListe( liste1 );
        imprimerListeInversee( liste1 ); // imprime la liste a l'envers
    }

    public void imprimerListe( List< String > liste )
    {
        System.out.println( "\nliste: " );

        for ( String color : liste )
            System.out.printf( "%s ", color );

        System.out.println();
    }

    private void convertirEnMajuscules( List< String > liste )
    {
        ListIterator< String > iterator = liste.listIterator();

        while ( iterator.hasNext() )
        {
            String color = iterator.next();          // recupere un element
            iterator.set( color.toUpperCase() ); // conversion en majuscules
        }
    }

    private void enleverElements( List< String > list, int start, int end )
    {
        list.subList( start, end ).clear();
    }

    private void imprimerListeInversee( List< String > list )
    {
        ListIterator< String > iterator = list.listIterator( list.size() );

        System.out.println( "\nListe inversee:" );

        while ( iterator.hasPrevious() )
            System.out.printf( "%s ", iterator.previous() );
    }

    public static void main( String args[] )
    {
        new ListTest();
    }
}

```

2.2.1.3. Les algorithmes du framework collections

Le framework Collections fournit de nombreux algorithmes qui agissent sur une collection. Ces algorithmes sont implémentés sous la forme de méthodes statiques appartenant à la classe *Collections*.

2. Tableau 4.2. Quelques algorithmes de la classe Collection

Algorithme	Description
sort	Trie les éléments d'une liste.

Algorithme	Description
binarySearch	Recherche d'un élément dans une collection.
reverse	Inverse la position des éléments d'une liste.
copy	Copie les éléments d'une liste dans une autre.
min	Retourne le plus petit élément de la liste.
max	Retourne le plus grand élément de la liste.
addAll	Ajoute tous les éléments d'un tableau dans une liste.
frequency	Calcule combien d'objets dans une liste sont égaux à celui spécifié.

Nous n'allons pas donner d'exemple pour chaque algorithme, mais nous contenter des algorithmes *sort* et *binarySearch* :

2.2.1.3.1. sort

Le programme *Sort.java* utilise un tableau de base pour remplir une liste, puis trie cette liste en ordre croissant. Le contenu de la liste est imprimé avant et après l'opération de tri.

```
import java.util.Vector;
import java.util.Collections;

public class Sort {
    public static void main( String [] args ) {
        Integer [] tableau = { 10, 5, 6, 2, 8 };
        Vector<Integer> vecteur = new Vector<Integer>();

        Collections.addAll( vecteur, tableau );
        imprimerListe( vecteur );
        Collections.sort( vecteur );
        imprimerListe( vecteur );
    }

    public static void imprimerListe( Vector<Integer> vecteur ) {
        for ( Integer element: vecteur )
            System.out.print( element + " " );

        System.out.println( "" );
    }
}
```

2.2.1.3.2. binarySearch

Le programme *BinarySearch.java* effectue une recherche dans une liste pour retourner la position de l'élément spécifié. Le contenu de la liste et le résultat de la recherche sont imprimés.

```
import java.util.Vector;
import java.util.Collections;
```

```

public class BinarySearch {
    public static void main( String [] args ) {
        String [] tableau = { "VERT", "MARRON", "ROUGE", "JAUNE" };
        Vector<String> vecteur = new Vector<String>();

        Collections.addAll( vecteur, tableau );
        imprimerListe( vecteur );

        // Recherche d'un élément
        int position = Collections.binarySearch( vecteur, "ROUGE" );
        if ( position >= 0 )
            System.out.printf( "\nL'élément \"ROUGE\" a été trouvé à la
position : %d\n", position );
        else
            System.out.printf( "L'élément n'a pas été trouvé dans la liste\n" );
    }

    public static void imprimerListe( Vector<String> vecteur ) {
        for ( String element: vecteur )
            System.out.print( element + " " );

        System.out.println( "" );
    }
}

```

2.2.1.4. L'interface Map

Les classes qui implémentent l'interface *Map* permettent d'associer une clé qui servira d'indice pour accéder aux éléments à un objet à stocker. Ce type de structure est aussi appelé dictionnaire dans d'autres langages. Les clés ne peuvent pas être dupliquées.

Exemple d'utilisation de l'interface *Map* :

```

import java.util.StringTokenizer;
import java.util.Map;
import java.util.HashMap;
import java.util.Set;
import java.util.TreeSet;
import java.util.Scanner;

public class CompteurMotsDifférents
{
    private Map< String, Integer > map;
    private Scanner scanner;

    public CompteurMotsDifférents() {
        map = new HashMap< String, Integer >();
        scanner = new Scanner( System.in );
        creerMap();
        imprimerMap();
    }

    private void creerMap() {
        System.out.println( "Entrez un texte:" );
        String entree = scanner.nextLine();

        StringTokenizer tokenizer = new StringTokenizer( entree );

        while ( tokenizer.hasMoreTokens() ) {
            String mot = tokenizer.nextToken().toLowerCase();

```



```

        if ( map.containsKey( mot ) ) {
            int count = map.get( mot );
            map.put( mot, count + 1 );
        }
        else
            map.put( mot, 1 );
    }
}

private void imprimerMap()
{
    Set< String > cles = map.keySet();
    TreeSet< String > clesTriees = new TreeSet< String >( cles );

    System.out.println( "La Map contient:\nCle\t\tValeur" );

    for ( String cle : clesTriees )
        System.out.printf( "%-10s%-10s\n", cle, map.get( cle ) );

    System.out.printf(
        "\nsize:%d\nisEmpty:%b\n", map.size(), map.isEmpty() );
}

public static void main( String args[] )
{
    new CompteurMotsDiffereents();
}
}

```

L'exemple utilise une collection implémentant l'interface *Map* pour compter les mots différents dans un texte saisi au clavier. Plusieurs nouvelles sont présentes dans le programme. La classe *StringTokenizer* permet de découper un texte en « tokens ». Chaque mot est une clé dans la collection. La valeur associée est le nombre de fois que le mot est apparu dans le texte.

Pour information, les classes suivantes implémentent l'interface *Map* :

- *Hashtable*
- *HashMap*

L'interface *SortedMap* hérite de l'interface *Map*. Cette interface ajoute la notion de tri sur les clés. Les objets qui servent de clés doivent implémenter l'interface *Comparable* afin de pouvoir être triés. *SortedMap* possède les méthodes supplémentaires suivantes par rapport à l'interface *Map* :

- *Comparator< K > comparator ()* : retourne l'objet *Comparator* chargé du tri des clés de la collection.
- *K firstKey()* : Retourne la plus petite clé.
- *K lastKey()* : Retourne la plus grande clé.
- *SortedMap <K,V> subMap(K fromKey, K toKey)* : Retourne une *SortedMap* constituée des éléments compris entre la *fromKey* incluse et la *toKey* exclue.
- *SortedMap <K,V> headMap(K fromKey)* : Retourne tous les éléments dont la clé est strictement inférieure à la clé passée en argument.
- *SortedMap <K,V> tailMap(K fromKey)* : Retourne tous les éléments dont la clé est supérieure ou égale à la clé passée en argument.

La classe *TreeMap* implémente cette interface. Voici un exemple d'utilisation de cette classe :

```

import java.util.*;

public class TestTreeMap {

```

```

public static void main( String [] args ) {
    TreeMap< String, Integer > villes = new TreeMap< String, Integer >();

    villes.put( "PARIS", 75000 );
    villes.put( "BORDEAUX", 33000 );
    villes.put( "BESANCON", 25000 );
    villes.put( "SAONE", 25660 );

    System.out.println( "Toutes les villes :");
    for ( String cle: villes.keySet() )
        System.out.printf( "%10s\t: %d\n", cle, villes.get( cle ) );

    SortedMap< String, Integer> villesEnB = villes.subMap( "B", "C" );

    System.out.println( "\n\nToutes les villes dont le nom commence par B
:");
    for ( String cle: villesEnB.keySet() )
        System.out.printf( "%10s\t: %d\n", cle, villesEnB.get( cle ) );

    }
}

```

2.3 Les entrées-sorties

2.3.1. Introduction

Comme tous les langages objets, Java fournit un ensemble de classes conçues pour manipuler les flux de données. Java est très riche en ce domaine. Les classes concernant les entrées-sorties sont toutes dans le package *java.io*. Pour pouvoir les utiliser, il nous suffit d'importer ce package avec la ligne :

```
import java.io.*;
```

Le framework Java fournit une cinquantaine de classes dédiées à la gestion des flots de données. Deux classes constituent la base du framework :

- *InputStream*
- *OutputStream*

InputStream permet de lire un flot de données en entrée, alors que *OutputStream* permet d'écrire dans un flot de données.

Parmi les méthodes de la classe *InputStream*, nous trouvons *read* et *close*. *read* est présente avec plusieurs signatures. Pour la classe *OutputStream*, nous avons *write* et *close*.

La plupart des classes de gestion des flots de données héritent de l'une de ces deux classes. Toutefois, l'utilisation de ces classes dérivées, capables de lire et d'écrire n'importe quoi est complexe. Pour la lecture de fichiers de texte, Sun a donc créé un ensemble de classes supplémentaires. Les classes de base de ce framework sont *Reader* et *Writer*.

Dans ce chapitre, nous allons présenter uniquement les classes *Reader* et *Writer*.

2.3.2. Lire des fichiers textes

La classe *Reader* permet de lire des fichiers texte. C'est une classe abstraite de bas niveau. Cette classe fournit des méthodes qui doivent être implémentées dans les classes dérivées.

Pour pouvoir travailler, nous allons utiliser la classe concrète *FileReader* qui hérite de *Reader*. Les méthodes *read* sont encore primitives. Ces méthodes lisent le fichier caractère par caractère ou un nombre défini de caractères. L'exemple suivant ouvre un fichier nommé *texte.txt* situé dans le répertoire courant et affiche le contenu à l'écran.

```
import java.io.*;
public class Copie {

    public static void main(String[] args) throws IOException {
        File inputFile = new File( "texte.txt" );
        FileReader in = new FileReader(inputFile);

        int c;

        while ( ( c = in.read() ) != -1 )
            System.out.print( (char) c );

        in.close();
    }
}
```

L'utilisation de la classe *File* n'est pas obligatoire. Cette classe donnée à titre d'exemple permet de manipuler le nom d'un fichier pour en obtenir des informations spécifiques, comme le nom du répertoire, l'extension du fichier, etc.

Nous transtypons le *int* lu avec la méthode *read* en *char* pour que la méthode *print* affiche un caractère. Sans le trantypage, le code ascii du caractère serait imprimé.

Pour pouvoir lire le fichier ligne par ligne, ce qui est souvent le cas pour des fichiers texte, nous allons utiliser une autre classe : *BufferedReader*. Cette classe possède une méthode *read* qui retourne une *String*. A chaque lecture, nous avons à disposition une ligne complète.

```
import java.io.*;
public class Copie2 {

    public static void main( String [] args ) throws IOException {
        BufferedReader in = new BufferedReader( new FileReader( "fichier.txt" )
    );
        String str;

        while ( ( str = in.readLine() ) != null ) {
            // faire quelque chose avec la ligne
            System.out.println( str );
        }

        in.close();
    }
}
```

Nous observons que *BufferedReader* « enveloppe » un objet de type *FileReader*. C'est l'application du design pattern « Décorateur » que nous verrons plus tard dans le module.

BufferedReader appelle en interne autant de fois que nécessaire la méthode *read* de *FileReader* pour construire une chaîne de caractère. Le programme se contente d'imprimer à l'écran le contenu de la chaîne.

2.3.3. Writer

Le programme *Ecriture1.java* ouvre un fichier texte sans tampon. Les écritures sont synchronisées avec le *write*. Un calcul est effectué et le résultat est écrit dans le fichier ouvert.

Exemple 6.1. Ecriture1.java

```
// Ecriture.java
import java.io.*;

public class Ecriture {

    public static void main( String[] args ) throws IOException {
        FileWriter sortie = new FileWriter( new File( "autretexte.txt" ) );

        int s, x, a;
        s = 10;
        x = 10;

        // Un petit calcul avant l'écriture du résultat dans le fichier.
        a = s + x;

        sortie.write( "Résultat de " );
        sortie.write( "s + x = " + a );

        sortie.close();
    }
}
```

Ouvrir un fichier sans tampon peut amener à des performances moindres. Chaque appel à *write* provoque une écriture réelle sur le disque. Il est préférable d'utiliser des ouvertures de fichiers avec tampon.

Le programme *Ecriture2.java* utilise un tampon pour optimiser les écritures dans le fichier. Le programme ouvre un fichier nommé *args.txt* pour y écrire tous les arguments passés à la ligne de commande. La méthode *newLine* est appelée pour insérer un retour chariot en fin de ligne.

Exemple 6.2. Ecriture2.java

```
// Ecriture2.java
import java.io.*;

public class Ecriture2 {

    public static void main( String[] args ) throws IOException {
        BufferedWriter sortie =
            new BufferedWriter(
                new FileWriter( new File( "args.txt" ) ) );

        for( String arg: args ) {
            sortie.write( arg );
            sortie.newLine();
        }
    }
}
```

```
        sortie.close();
    }
}
```

2.3.4. Formater une sortie à l'écran

Dans de nombreuses circonstances, nous devons formater la sortie de certaines valeurs afin d'obtenir un affichage correct. Par exemple, nous pourrions vouloir imprimer une liste de couples de valeurs constituées d'une référence entière et d'une valeur réelle. La liste doit correctement formater, les valeurs bien alignées. Pour la valeur réelle, nous devons prendre en compte uniquement les deux premières décimales. Voici un programme permettant d'obtenir le résultat escompté.

```
public class ImpressionFormatee {
    public static void main( String[] args ) {
        int [] ref = { 100, 2, 1004, 34, 654 };
        double [] val = { 1.3456, 456.3, 13., 1234.678, 86.19 };

        for (int i = 0; i < ref.length; i++)
            System.out.printf("%1$5d %2$10.2f\n", ref[i], val[i]);
    }
}
```

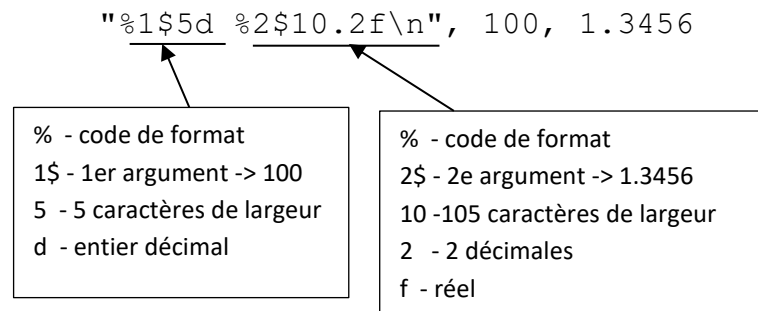
Nous compilons et lançons le programme. Le résultat est correct :

```
# javac ImpressionFormatee.java
# java ImpressionFormatee
Impression avec 2 décimales :
 100      1,35
   2     456,30
1004     13,00
  34    1234,68
 654     86,19
```

Nous utilisons la méthode *printf* de la classe *PrintStream* pour formater notre affichage. La description de la chaîne de formatage est détaillée dans la documentation de la classe *java.util.Formatter*. Résumons un peu le principe. Soit la ligne suivante :

```
System.out.printf("%1$5d %2$10.2f\n", 100, 1.3456);
```

Le schéma montre à quoi chaque élément de la chaîne de formatage correspond.



Fonctionnement de la méthode *printf*

2.3.5. Saisie de valeurs au clavier

La saisie des valeurs simples au clavier n'est pas forcément facile avec Java. Afin de se faciliter l'existence, voici une classe utilitaire qui nous permet de lire différents types de valeurs. Son usage est trivial.

```
import java.io.*;

public class LectureClavier {
    public static String lireString() {
        String ligne = null;

        try {
            InputStreamReader i = new InputStreamReader(System.in);
            BufferedReader b = new BufferedReader(i);
            ligne = b.readLine();
        }
        catch(IOException e) {
            System.err.println(e);
        }
        return ligne;
    }

    public static int lireInt() {
        return Integer.parseInt(lireString());
    }

    public static double lireDouble() {
        return Double.parseDouble(lireString());
    }

    // Nous pouvons imaginer d'autres types de lecture.
}
```

Cette classe est située dans un fichier nommé *LectureClavier.java*¹⁰. Le programme *TestLectureClavier.java* utilise la classe *LectureClavier* afin de lire quelques valeurs et de les imprimer à l'écran.

```
public class TestLectureClavier {
    public static void main(String[] args) {

        System.out.print("Saisir un int : ");
```

¹⁰ Cette classe doit être placée dans le même répertoire que la classe qui l'utilise.

```

        int i = LectureClavier.lireInt();
        System.out.println("i vaut " + i);

        System.out.print("Saisir un double : ");
        double d = LectureClavier.lireDouble();
        System.out.println("d vaut " + d);
    }
}

```

2.3.6. La sérialisation des objets

La sérialisation d'un objet consiste à le transformer de façon à pouvoir l'écrire dans un flot de données. La désérialisation est le mécanisme inverse. Plusieurs raisons peuvent nous inciter à sérialiser un objet :

- Mémoriser l'état d'un objet dans un fichier à la fin d'une application pour pouvoir le reconstruire plus tard.
- Mémoriser l'état d'un objet dans une base de données de type SGBD.
- Pouvoir transférer un objet d'une application à une autre à travers des connexions réseau
- Etc

[Avertissement] La persistance

Un objet qui peut être sérialisé est donc dit persistant : il peut survivre à l'application qui l'a créé.

Pour pouvoir être sérialisé, un objet doit implémenter l'interface *Serializable*. Cette interface ne possède pas de méthode à implémenter. Elle permet juste de typer l'objet afin d'indiquer qu'il peut être sérialisé.

La sérialisation assure la sauvegarde de toutes les données élémentaires de chaque objet, mais également la sauvegarde des références vers d'autres objets mises en œuvre dans les relations de composition et d'association.

La sérialisation est basée sur l'utilisation des classes *ObjectInputStream* et *ObjectOutputStream*.

ObjectInputStream hérite de *InputStream* et fournit la méthode *readObject*. *ObjectOutputStream* hérite de *OutputStream* et fournit la méthode *writeObject*.

L'exemple Sérialisation d'un compte montre l'utilisation de la sérialisation. Le compte bancaire *compte1* est créé puis il est injecté dans un flux. Nous récupérons l'état de *compte1* depuis le fichier *compte.dat* pour initialiser l'état de *compte2*. Nous affichons ensuite les deux comptes à l'écran.

3. Exemple 5.3. Sérialisation d'un compte

```

import java.io.*;

class Compte implements Serializable {
    static final long serialVersionUID = 3129471313357965203L;
    String nom;
    Double solde;

    public Compte() {
    }
}

```

```

    public Compte( String nom, Double solde ) {
        this.nom = nom;
        this.solde = solde;
    }

    public String toString() {
        return this.getClass().getName() + ", " +
            nom + ", " + solde;
    }
}

public class Serialisation {
    public static void main( String [] args ) {
        Compte compte1 = new Compte( "jmcaricand", new Double( 1000.00 ) );
        Compte compte2 = new Compte();

        try {
            FileOutputStream out = new FileOutputStream( "compte.dat" );
            ObjectOutputStream o = new ObjectOutputStream( out );
            o.writeObject(compte1);
        }
        catch( IOException e ) {
            System.out.println( e.getMessage() );
        }

        try {
            FileInputStream in = new FileInputStream( "compte.dat" );
            ObjectInputStream i = new ObjectInputStream( in );
            compte2 = ( Compte ) i.readObject();
        }
        catch( IOException e ) {
            System.out.println( e.getMessage() );
        }
        catch( ClassNotFoundException e ) {
            System.out.println( e.getMessage() );
        }

        System.out.println( "Compte 1 :\n" + compte1 + "\n\n" );
        System.out.println( "Compte 2 :\n" + compte2 + "\n\n" );
    }
}

```

Il est possible d'éviter de sauvegarder certaines variables de notre instance de classe en marquant la variable avec le mot clé *transient*. Certaines données n'ont pas besoin d'être sauvegardées car cela n'aurait guère de sens et aucun intérêt.

```

import java.io.*;

public class Date implements Serializable {
    transient Timer alerteRendezVous;
}

```

[Avertissement] Les variables statiques

Les variables statiques ne peuvent être sérialisées. Java n'intègre aucun mécanisme pour gérer ce type de variables.

Nous pouvons également noter que la désérialisation des objets impose de gérer l'exception *ClassNotFoundException*. En effet, en fonction de l'environnement d'exécution du programme, la classe de l'objet en cours de lecture depuis le flux de données peut ne pas exister si le fichier *class* ou *jar* ne sont pas trouvés.

Il est également utile de gérer l'exception *InvalidateClassException* pour intercepter le fait que la version de la classe en cours de lecture est différente de celle stockée dans le flux de données. Java crée automatiquement à la compilation un numéro de série qui sera stocké comme variable statique dans la classe. Les objets sont sérialisés en stockant le nom de la classe, mais également le numéro de série. Ces deux informations permettent de vérifier si l'objet issu du flux est bien compatible avec la classe en cours d'exécution.

Pour créer un numéro de série unique pour une classe et le fixer définitivement à la compilation, le JDK fournit l'outil *serialver*. Nous avons utilisé cet outil pour créer la variable *serialVersionUID* de la classe *Compte* du programme *Serialisation.java*. La compilation émet d'ailleurs un avertissement lorsqu'une classe sérialisable ne possède pas la variable statique *serialVersionUID*.

```
# serialver Compte
Compte: static final long serialVersionUID = 3129471313357965203L;
```

2.4 JDBC

JDBC est un moyen d'émettre des requêtes SQL vers les serveurs, et de récupérer le résultat de cette requête dans du code Java.

2.4.1. Accéder aux bases de données

On considère aujourd'hui qu'environ 80% des produits logiciels développés ont besoin d'accéder à une base de données. Dès la conception du langage Java, l'accès aux bases de données a donc été une priorité des concepteurs. L'API JDBC (Java Database Connectivity) a été la première API développée après le JDK, et séparément de lui. Elle fut introduite dans l'API standard dès le JDK 1.1. L'idée originale était de donner la possibilité à un programme Java d'accéder à toute base de données ANSI SQL-2.

On distingue plusieurs types de pilotes JDBC, même si cette distinction est de moins en moins d'actualité, tant Java est devenu incontournable pour les vendeurs de base de données.

Tableau 1. Types de pilotes JDBC

Type de pilote JDBC	Description
1	Pont JDBC-ODBC. La solution la moins couteuse. Requier du code natif côté client. Solution peu performante.
2	API Java-native. La façon la plus simple pour le serveur de construire un pilote JDBC. Requier du code natif côté client.
3	Pilote réseau tout en Java. Le type le plus flexible puisqu'il permet de se connecter à une base de données en restant indépendant du middleware.
4	Pilote natif tout en Java. Le type le plus puissant. Nécessite de connaître le protocole d'accès propre à la base de données. C'est ce type de pilote que les vendeurs de base de données fournissent.

Voici la liste des différentes versions de JDBC, ainsi que le JDK correspondant.

Tableau 2. Versions successives de JDBC

Version de JDBC	Version du JDK	Packages
JDBC 1.0 (1998)	JDK 1.1	java.sql
JDBC 2.0 (1999)	JDK 1.2	java.sql
JDBC 2.0 options (1999)	JDK 1.2	java.sql
JDBC 2.1 options (1999)	JDK 1.2	java.sql
JDBC 3.0 API noyau (2001)	JDK 1.4	java.sql, javax.sql
JDBC 4.0 (2006)	JDK 6	java.sql, javax.sql
JDBC 4.1 (2011)	JDK 7	langage, java.sql, javax.sql

Techniquement, JDBC est un ensemble d'interfaces qui définissent le schéma général de connexion et d'interrogation des bases de données. Le pilote d'une base de données fournit des classes qui implémentent ces interfaces. Il est donc propre à chaque base, mais on peut l'utiliser de façon transparente en ne considérant que les interfaces. C'est même de cette façon qu'il faut procéder.

2.4.2. Principe d'accès à une base

Le principe d'utilisation du JDBC est particulièrement simple. Il se compose de quatre actions :

- Chargement du pilote de la base.
- Ouverture d'une connexion à la base de données. Le code doit fournir au minimum trois informations : une URL d'accès, un nom de connexion et un mot de passe.
- Création d'un objet de type Statement, qui va prendre en charge l'acheminement de la requête SQL vers le serveur, et récupérer les résultats.
- Récupération du résultat de la requête dans un objet de type ResultSet, et exploitation de ce résultat.

2.4.3. Exemple

2.4.3.1. Chargement du pilote de la base

Le pilote d'une base de données est tout simplement une classe Java instance de l'interface `java.sql.Driver`. Comme nous l'avons déjà vu, il est possible de charger une classe Java explicitement dans le code, par appel de la méthode `Class.forName()`.

Chaque éditeur de base de données fournit une telle classe, généralement dans un JAR qui constitue le pilote complet. Le nom complet de cette classe varie donc d'un éditeur à l'autre. Pour chaque base, il faut donc trouver ce nom. Dans le cas d'Oracle, il s'agit de `oracle.jdbc.OracleDriver`. L'ouverture d'une connexion sur une base Oracle aura donc la forme suivante :

- Tout d'abord chargement du pilote Oracle, récupération d'un objet de type `Class` : `Class.forName("oracle.jdbc.OracleDriver")` ;

Ensuite ouverture de la connexion proprement dite :

```
Connection con = DriverManager.getConnection(chaineDeConnexion, user, password) ;
```

L'objet `chaineDeConnexion` est de la classe `String`. C'est une concaténation des éléments suivants :

- `jdbc:oracle:thin` : indique le driver utilisé pour accéder à la base de donnée. Ce driver est propre à chaque base.
- `myhost:1521` : indique le nom d'hôte qui héberge la base de données, ainsi que le port d'accès.
- `orcl` : le nom de la base de données à laquelle on souhaite se connecter.

Ces différents éléments se concatènent en une chaîne unique qui a la forme suivante : `jdbc:oracle:thin:@myhost:1521:orcl`

Ces deux méthodes jettent bien sur leurs propres exceptions, notamment si la classe du pilote n'est pas trouvée, si une erreur s'est glissée dans l'URL de connexion, si l'hôte est inaccessible, ou s'il se trouve une erreur dans les identifiants de connexion.

2.4.3.2. Création d'une requête

Pour pouvoir passer une requête à la base, il faut un objet de type `Statement`. Cet objet s'obtient en faisant appel à la méthode `createStatement()` de notre objet de type `Connection`.

```
Statement smt = con.createStatement() ;
```

On peut envoyer ensuite trois types de requêtes :

- Des sélections : `smt.executeQuery(chaineSQL)` ;
- Des mises à jour : `smt.executeUpdate(chaineSQL)` ;
- Des exécutions de procédures stockées : `smt.execute(chaineSQL)`.

2.4.3.3. Récupération d'un résultat

On récupère le résultat dans un objet de type `ResultSet`. Dans le cas d'une mise à jour de la base, ce qui correspond aux commandes SQL `insert`, `update`, `create`, `drop`, `delete`, on utilise la commande :

```
int i = smt.executeUpdate("insert into .... values ....") ;
```

L'entier de retour représente le nombre de lignes affectées dans le cas d'un insert, d'un update ou d'un delete.

Dans le cas d'une requête de sélection, un objet de type ResultSet est renvoyé.

```
ResultSet resultSet = smt.executeQuery("select Nom, Age from Etudiants") ;
```

L'objet resultSet contient les lignes renvoyées par la requête.

Enfin, pour parcourir ces lignes, il faut utiliser les méthodes de l'interface ResultSet. Voici un exemple.

```
while (rs.next()) {  
    String nom = rs.getString("Nom") ;  
    int age = rs.getInt("Age") ;  
}
```

Les champs des lignes sélectionnées sont maintenant disponibles sous forme d'objets Java, ce qui était bien notre objectif.

2.4.3.4. Exemple complet

Voici un exemple complet du code que nous venons de présenter très rapidement.

Exemple : Un premier exemple de connexion

```
import java.sql.DriverManager;  
import java.sql.ResultSet;  
import java.sql.Connection;  
import java.sql.Statement;  
  
public class OracleJDBCExample {  
  
    public static void main(String[] argv) {  
        try {  
            // chargement de la classe par son nom  
            Class.forName("oracle.jdbc.driver.OracleDriver");  
            // Chaîne de connexion  
            Connection connection = DriverManager.getConnection(  
                "jdbc:oracle:thin:@localhost:1521:xe", "system",  
                "password");  
            // Envoi d'un requête générique  
            String sql = "select * from Etudiants" ;  
            Statement smt = connection.createStatement() ;  
            ResultSet rs = smt.executeQuery(sql) ;  
            while (rs.next()) {  
                System.out.println(rs.getString("Nom")) ;  
            }  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Après cette rapide présentation des choses, examinons chacun des points en détails.

2.4.4. Établissement d'une connexion

La première tâche à accomplir pour interroger une base de données est de s'y connecter. L'établissement d'une connexion passe par deux étapes :

- Chargement du pilote de la base de données. Ce pilote est en général une classe Java, qui implémente l'interface Driver.
- Instanciation du pilote, avec passage des paramètres de connexion : URL de la base, ou toute autre façon de l'identifier, nom de connexion et mot de passe.

Il y a des bases de données, par exemple MySQL, pour lesquelles il n'y a qu'un seul pilote. Pour d'autres bases de données, par exemple Oracle, il peut exister plusieurs types de pilotes. Il faut alors choisir celui que l'on veut utiliser.

2.4.4.1. Chargement et déclaration du pilote

2.4.4.1.1. Introduction

Le chargement du pilote consiste à charger la classe de ce pilote, et à en créer une instance. La façon la plus immédiate de procéder est de faire appel à la méthode `forName(String)` de la classe `Class`, puis par appel de la méthode `newInstance()`. Cette procédure est pratique, car il n'y a pas besoin de connaître le pilote au moment de l'écriture du code. Il est parfaitement possible de le passer en paramètre au moment du lancement de l'application.

Ensuite, il faut déclarer le pilote au niveau du `DriverManager`, par appel à la méthode statique `registerDriver(Driver)`. Cet objet se chargera enfin de la construction de l'objet connexion à la base de données. Cet objet peut connaître autant de pilotes qu'on peut lui en donner, et c'est lui qui, au vu de la chaîne de connexion qu'il reçoit, va choisir le bon pilote et créer l'objet de type `Connection`.

Le pilote est une classe qui implémente l'interface `Driver`. Cette interface contient une méthode principale : `connect()`, à laquelle il faut passer comme paramètres l'URL de la base et les identifiants de connexion pour pouvoir établir une connexion à la base de données.

Outre `registerDriver(Driver)` et `deregisterDriver(Driver)`, la classe `DriverManager` possède des méthodes qui permettent de configurer les pilotes qu'elle gère.

2.4.4.1.2. Chargement et instanciation du pilote

La méthode statique `forName(String)` de la classe `Class` nous renvoie une instance de `Class`. On ne lui fournit qu'une chaîne de caractères qui représente le nom complet du pilote. Une fois cette instance créée, le `DriverManager` connaît automatiquement le pilote, et il est possible de demander une connexion sur la base.

L'avantage de cette méthode est que l'on n'a pas besoin du pilote au moment de la compilation. Il suffit juste de connaître son nom complet. La machine Java n'aura besoin d'avoir ce pilote dans son espace de nom que lors de son chargement.

L'instanciation de ce pilote peut se faire par l'appel à la méthode statique `newInstance()` de la classe `Class`. Elle nous renvoie un objet de type `Driver`, interface implémentée par tous les pilotes JDBC.

Cette méthode était la méthode préconisée dans les anciennes versions du JDK et de JDBC. De nouveaux mécanismes ont été apportés à partir de la version Java 6, qui permettent de procéder de façon plus simple et purement déclarative. Notamment, il n'est plus nécessaire de charger explicitement la classe du pilote, ni même de l'instancier.

2.4.4.1.3. Enregistrement du pilote

L'enregistrement du pilote se fait par appel à la méthode statique `registerDriver(Driver)` de la classe `DriverManager`. On passe en paramètre l'instance du pilote que l'on vient de construire.

2.4.4.1.4. Pilotes pour les principales bases de données

Base de données	Classe du pilote	Référence
MySQL	<code>com.mysql.jdbc.Driver</code>	http://www.mysql.com/
Apache Derby	<code>org.apache.derby.jdbc.EmbeddedDriver</code>	http://db.apache.org/derby/
PostgreSQL	<code>org.postgresql.Driver</code>	http://www.postgresql.org/
SQLite	<code>org.sqlite.JDBC</code>	http://www.sqlite.org/
HSQLDB	<code>org.hsqldb.jdbcDriver</code>	http://hsqldb.org/
Oracle	<code>oracle.jdbc.driver.OracleDriver</code>	http://www.oracle.com/
SQLServer	<code>com.microsoft.jdbc.sqlserver.SQLServerDriver</code>	http://www.microsoft.com/sqlserver/

2.4.4.2. Connexion à la base

2.4.4.2.1. Établissement de la connexion

La connexion à la base consiste à demander au `DriverManager` un objet de type `Connection`, par l'appel de la méthode `getConnection()`. La il n'y a qu'une seule façon de le faire : passer l'URL de la base, un nom de connexion et un mot de passe valides. L'appel de cette méthode renvoie un objet instance d'une classe qui implémente l'interface `Connection`.

Exemple : Obtenir une connexion du `DriverManager`

```
Connection con = DriverManager.getConnection(
    "jdbc:oracle:thin:@localhost:1521:xe", "system", "password") ;
```

2.4.4.2.2. Interface `Connection`

Comme nous l'avons déjà dit, `Connection` est une interface. L'objet que nous obtenons est donc une implémentation de cette interface par une classe que nous ne connaissons pas, et que nous n'avons pas besoin de connaître. Cette classe est propre à chaque serveur de base de données, et en général écrite par les développeurs de cette base de données.

L'ouverture et la fermeture d'une connexion a une base de données sont deux choses qui doivent être traitées avec attention. L'ouverture est un processus coûteux. Sur certains serveurs, elle peut prendre plusieurs secondes. Une connexion ouverte sur une base de données est une ressource consommée.

Comme toutes les ressources d'un système d'exploitation, elle est « rare », en tout cas, il existe un nombre maximal de connexions ouvertes à un instant donné. Cela rend indispensable la fermeture de toute connexion ouverte.

L'interface Connection comporte environ 45 méthodes, que l'on peut regrouper en trois catégories :

- les méthodes de gestion de transaction ;
- les méthodes de création de déclaration
- les méthodes de gestion des objets.

2.4.4.2.2.1. Gestion des transactions

Par défaut, le mode de fonctionnement d'un pilote JDBC est auto-commit. Ce qui signifie qu'une demande de validation de transaction est passée à l'issue de chaque requête SQL exécutée.

L'interface Connection propose deux méthodes :

- `getAutoCommit()` : permet de tester dans quelle mode la connexion se trouve
- `setAutoCommit(boolean)` : permet de changer ce mode.

Parmi les autres méthodes nous retiendrons les 2 principales :

- `commit()` : termine une transaction en validant les modifications.
- `rollback()` : termine une transaction en annulant les modifications.

2.4.4.2.2.2. Construction d'un `Statement`

Un objet de type `Statement` est un objet qui prend en charge une requête SQL, l'adresse au serveur et récupère un résultat. Le résultat est lui-même stocké dans un objet de type `ResultSet`.

Un tel objet est obtenu par appel d'une des méthodes du type `createStatement()` de l'interface Connection.

2.4.5. Exécution de requêtes SQL

Un objet de type `Statement` permet d'envoyer une requête SQL à la base de données, et de récupérer un résultat de requête, stocké dans un objet de type `ResultSet`.

Il existe trois types de déclarations :

- les instances de `Statement` : permettent d'envoyer une requête SQL de sélection ou de mise à jour ;
- les instances de `PreparedStatement` : permettent d'envoyer des requêtes paramétrées.
- les instances de `CallableStatement` : permettent d'appeler une procédure stockée ;

2.4.5.1. Création d'un objet `Statement`

Un objet de type `Statement` s'obtient en appelant la méthode `createStatement()` de l'interface Connection.

Exemple 5. Création d'un objet `Statement`

```
Statement smt = connexion.createStatement();
```

On peut créer autant d'objets de ce type par connexion, et tous ces objets peuvent être utilisés de façon concurrente.

Il est important de fermer un Statement lorsqu'il n'est plus utilisé, par appel à sa méthode `close()`.

2.4.5.2. Exécution d'un Statement

Il existe trois méthodes pour exécuter un Statement. Chacune de ces méthodes doit être utilisée dans son propre contexte.

- `executeQuery(String)` doit être utilisée si l'exécution du Statement retourne une liste d'objets (cas d'un select) ;
- `executeUpdate(String)` doit être utilisée si l'exécution du Statement retourne un nombre d'objets modifiés (cas des requête de création, d'effacement ou de mise à jour) ;
- `execute(String)` doit être utilisée si le type de Statement exécuté n'est pas connu.

Ces trois méthodes prennent une chaîne de caractères en paramètres. Cette chaîne est la commande SQL qui va être exécutée sur le serveur.

2.4.5.2.1. Méthode `executeQuery(String)`

Cette méthode retourne un objet de type `ResultSet`. Si la requête SQL passée en paramètre ne correspond pas (par exemple, si elle contient un insert), alors une exception de type `SQLException` est jetée.

Exemple 6. Méthode `Statement.executeQuery(String)`

```
Statement smt = connection.createStatement() ;
ResultSet rs = smt.executeQuery("select Nom, Prenom from Etudiants") ;
// exploitation du resultat
while (rs.hasNext()) {
    ...
}
```

2.4.5.2.2. Méthode `executeUpdate(String)`

Toutes les requêtes de mise à jour de la base retournent un entier qui correspond aux nombres d'éléments créés, mis à jour ou effacés. Voyons comment on utilise une telle méthode.

Exemple : Méthode `Statement.executeUpdate(String)`

```
Statement smt = connection.createStatement() ;
int count = smt.executeUpdate("insert into Etudiants(Nom, Prenom) " +
                             "values ('Robert', 'Surcouf')") ;
// exploitation du resultat
if (count > 0) {
    ...
}
```

2.4.5.2.3. Méthode `execute(String)`

Cette méthode doit être utilisée si l'on ne connaît pas la nature de la requête SQL qui va être effectivement exécutée. Elle retourne true si l'objet retourné est de type ResultSet et false s'il s'agit d'un entier. On peut ensuite obtenir ces objets en appelant les méthodes `getResultSet()` et `getUpdateCount()`.

De plus, si l'exécution de cette requête a généré plusieurs résultats, la méthode `getMoreResult()` doit être appelée. Voyons l'utilisation de tout ceci.

Exemple : Méthode `Statement.execute(String)`

```
Statement stmt = conn.createStatement() ;
boolean returnedValue = stmt.execute(sql) ; // on ne connaît pas la nature
                                           // de cette requête

ResultSet rs ;
int count ;

do {
    if (returnedValue) { // le résultat est un result set
        rs = stmt.getResultSet();
        // exploitation du result set
        ...
    } else { // le résultat est un entier
        count = stmt.getUpdateCount() ;
        if (count == -1) {
            // une valeur -1 indique qu'il n'y a plus de résultat à exploiter
            break ; // sortie du while
        } else {
            // exploitation du count
            ...
        }
    }
    returnedValue = stmt.getMoreResult() ;
} while (true) ;
```

Notons que par défaut, l'ouverture d'un nouveau ResultSet par appel à la méthode `getResultSet()` ferme automatiquement l'ancien. Ce comportement peut être changé, si le pilote utilisé le permet, comme nous le verrons dans le paragraphe sur les ResultSet.

2.4.5.3. Fermeture d'un objet `Statement`

Tous les objets de type `Statement` doivent être fermés par un appel à la méthode `close()` une fois qu'ils ne sont plus utilisés. La fermeture d'un `Statement` entraîne automatiquement la fermeture de tous les objets `ResultSet` qui ont été ouverts sur ce `Statement`.

Notons que même si la fermeture d'une connexion (par appel à sa méthode `close()`) ferme automatiquement tous les `Statement` ouverts dessus, il est recommandé de le faire à la main. De même que pour la fermeture des `ResultSet`.

Une fois qu'un objet `Statement` ou `ResultSet` a été fermé, il est illégal d'invoquer une de ses méthodes. L'appel d'une telle méthode peut jeter systématiquement une exception de type `SQLException`.

2.4.5.4. Création d'un objet `PreparedStatement`

L'interface `PreparedStatement` étend l'interface `Statement`, donc tout ce qui précède et qui concerne l'interface `Statement` reste valide pour l'interface `PreparedStatement`.

L'interface `PreparedStatement` ajoute la possibilité de paramétrer des requêtes SQL. Les instances de `PreparedStatement` s'utilisent quand une même requête doit être exécutée plusieurs fois, avec des

paramètres différents. La chaîne de caractères SQL contient donc des marqueurs, qui seront remplacés par des valeurs à chaque exécution.

On préférera l'utilisation de PreparedStatement aux Statement classiques, en particulier pour les requêtes de mise à jour. Ces requêtes comportent le plus souvent des paramètres, qu'il faut ajouter à la chaîne de caractères SQL à exécuter. Utiliser des PreparedStatement permet de passer par les méthodes de paramétrage du pilote de base, ce qui nous décharge de la délicate gestion des erreurs, et évitera les attaques par injection de code SQL.

2.4.5.4.1. Création d'un PreparedStatement

Une instance de PreparedStatement se crée de la même façon qu'un Statement normal.

Exemple 9. Création d'un PreparedStatement

```
PreparedStatement ps = conn.prepareStatement(  
    "insert into Marins (nom, prenom, age) values (?, ?, ?)" ;
```

De même que dans le cas d'un Statement normal, on peut passer des options lors de la création d'un PreparedStatement.

On remarque la présence de trois caractères ? dans la chaîne SQL passée en paramètre. Ce sont ces caractères qui doivent être remplacés par des valeurs afin de pouvoir exécuter la requête SQL proprement dite.

2.4.5.4.2. Fixer les paramètres d'un PreparedStatement

L'interface PreparedStatement propose un jeu de méthode set<Type>(int, Type) : setInt(int, int), setFloat(int, float), setString(int, String), etc... Chacune de ces méthodes peut être appelée pour fixer la valeur d'un paramètre donné.

Exemple : Fixer les paramètres d'un PreparedStatement

```
PreparedStatement ps = conn.prepareStatement(  
    "insert into Etudiants (Nom, Prenom, Age) values (?, ?, ?)" ;  
  
ps.setString(1, "Surcouf") ;  
ps.setString(2, "Robert") ;  
ps.setInt(3, 32) ;
```

On notera que la numérotation des paramètres commence à 1. Tous les paramètres doivent avoir une valeur fixée avant que la requête soit exécutée, sous peine de SQLException.

On peut effacer la valeur de tous les paramètres d'un PreparedStatement en appelant la méthode clearParameter().

2.4.5.4.3. Positionner un paramètre à la valeur null

Positionner un paramètre à la valeur null nécessite l'utilisation d'une méthode particulière : setNull(int, int). Cette méthode prend deux paramètres :

- le premier est le numéro d'ordre du paramètre ;
- le second est le type du paramètre, il s'agit de l'une des constantes définies dans la classe java.sql.Types

Exemple : Positionner un paramètre a null

```
PreparedStatement ps = conn.prepareStatement(
    "insert into Marins (nom, prenom, age) values (?, ?, ?)" ;

ps.setString(1, "Surcouf") ;
ps.setString(2, "Robert") ;
ps.setNull(3, java.sql.Types.INTEGER) ;
```

2.4.5.5. Exécution d'un PreparedStatement

L'exécution d'un PreparedStatement se déroule de la même manière que pour un Statement, à l'aide des trois méthodes executeQuery(), executeUpdate() et execute().

2.4.5.5.1. Méthode getMetaData() et getParameterMetaData()

L'interface PreparedStatement propose deux méthodes :

- getMetaData() : retourne un objet ResultSetMetaData, qui contient des informations sur les colonnes retournées par la requête SQL ;
- getParameterMetaData() : retourne un objet ParameterMetaData, qui contient des informations sur les paramètres déclarés dans ce PreparedStatement.

La méthode getMetaData() retourne un objet de type ResultSetMetaData qui propose un jeu d'une vingtaine de méthodes pour déterminer les types de colonne du résultat de la requête. Voyons ceci sur un exemple.

Exemple : Utilisation d'un objet ResultSetMetaData

```
// on ne connaît pas les colonnes du résultat de cette requête
PreparedStatement pstmt = conn.prepareStatement("select * from Etudiants where
id = ?") ;
ResultSetMetaData rsmd = pstmt.getMetaData() ;

// lecture du nombre de colonne
int columnCount = rsmd.getColumnCount() ;
for (int i = 1 ; i <= columnCount ; i++) {
    int columnType = rsmd.getColumnType(i) ;
    // label de la colonne
    String columnLabel = rsmd.getColumnLabel(i) ;
    // nom de la colonne
    String columnName = rsmd.getColumnName(i) ;
}
```

On remarquera qu'ici aussi les paramètres sont numérotés à partir de 1.

La méthode getParameterMetaData() retourne un objet de type ParameterMetaData, qui lui-même propose quelques méthodes pour obtenir des informations sur les paramètres d'un PreparedStatement. Voyons un exemple d'utilisation.

Exemple : Utilisation d'un objet ParameterMetaData

```
// on ne connaît pas les colonnes du résultat de cette requête
```

```

PreparedStatement pstmt = conn.prepareStatement("select * from Etudiants where
id = ?") ;
ParameterMetaData pmd = pstmt.getParameterMetaData() ;

// lecture du nombre de colonne
int parameterCount = pmd.getParameterCount() ;
for (int i = 1 ; i <= parameterCount ; i++) {
    // type du paramètre (une constante de java.sql.Types)
    int parameterType = pmd.getColumnType(i) ;
    // mode du paramètre
    int parameterMode = pmd.getParameterMode(i) ;
}

```

Notons que le **mode** d'un paramètre, qui peut prendre l'une des valeurs IN, OUT ou INOUT est utilisé pour les CallableStatements.

2.4.5.6. Exécution de mises à jour en batch

Il est également possible de grouper des requêtes SQL dans un Statement et de les exécuter en une seule fois. On appelle ce genre d'exécution ***l'exécution en batch***. En général cela signifie que l'on s'attend à ce que l'exécution totale soit assez longue, et que pendant ce temps, le système puisse faire autre chose.

L'exécution de commandes SQL en batch n'est possible que pour des requêtes de modification de la base, celles que l'on exécute avec la méthode `executeUpdate(String)`. Toute requête SQL qui peut être exécutée de la sorte, peut également être passée en paramètre de la méthode `addBatch(String)`. L'appel de cette méthode ne fait que stocker la requête SQL, sans l'exécuter. C'est l'appel à la méthode `executeBatch()` qui déclenche l'exécution.

L'exécution en batch peut représenter des gains substantiels en performances. Le coût d'un aller-retour avec la base de données est important, et l'on tentera le plus possible de minimiser ces allers-retours.

2.4.5.6.1. Cas des Statement

Voyons un exemple d'exécution en batch pour les objets Statement.

Exemple : Exécution d'un Statement en batch

```
// l'exécution en batch doit se faire en mode non auto-commit
connection.setAutoCommit(false) ;
Statement smt = connection.createStatement() ;
smt.addBatch("insert into Etudiants(Nom, Prenom) values ('Surcouf', 'Robert'))
;
smt.addBatch("insert into Etudiants (nom, prenom) values ('Tabarly', 'Eric')) ;
...
// lancement de l'exécution de toutes nos insertions
int [] updateCounts = smt.executeBatch() ;
```

Comme le mode **auto-commit** a été désactivé, il nous est possible, en cas d'erreur sur l'appel à `executeBatch()` d'annuler la transaction et de remettre la base dans l'état initial.

2.4.5.6.2. Cas des PreparedStatement

Il est aussi possible de lancer des exécutions de mise à jour en batch lorsque l'on travaille avec des PreparedStatement. Le mécanisme est toutefois légèrement différent, puisqu'un PreparedStatement est construit sur une unique requête SQL paramétrée.

PreparedStatement propose une méthode `addBatch()` qui ne prend aucun paramètre. Au moment où cette méthode est appelée, JDBC construit une requête SQL avec la requête du PreparedStatement, et le jeu de paramètres qu'il a à sa disposition. Si l'un des paramètres n'a pas été fixé, une erreur est générée. On peut ainsi ajouter autant de requêtes que l'on veut.

L'exécution des requêtes se fait de la même manière et sous les mêmes contraintes que pour un Statement classique.

Exemple 16. Exécution d'un PreparedStatement en batch

```
// l'exécution en batch doit se faire en mode non auto-commit
connection.setAutoCommit(false) ;
PreparedStatement psmt =
    connection.prepareStatement("insert into Marins(nom, prenom) " +
                                "values (?, ?)" ) ;

psmt.setParameter(1, "Surcouf") ;
psmt.setParameter(2, "Robert") ;
// ajout d'une requête avec les paramètres ('Surcouf', 'Robert')
psmt.addBatch() ;

psmt.setParameter(2, "Pierre") ;
// ajout d'une requête avec les paramètres ('Surcouf', 'Pierre')
psmt.addBatch() ;

psmt.setParameter(1, "Tabarly") ;
psmt.setParameter(2, "Eric") ;
// ajout d'une requête avec les paramètres ('Tabarly', 'Eric')
psmt.addBatch() ;
...
// lancement de l'exécution de toutes nos insertions
int [] updateCounts = smt.executeBatch() ;
```

2.4.6. Exploitation des résultats

Le résultat d'une requête est stocké dans un objet de type `ResultSet`. La encore, ce type est une interface, et le pilote de la base de données nous renvoie une instance de classe qui implémente cette interface.

Les objets `ResultSet` peuvent paraître assez simples à manipuler de prime abord, et de fait ils le sont. Toutefois, cette simplicité masque certaines subtilités de leur fonctionnement que nous verrons en détails.

Les objets `ResultSet` peuvent exposer différents comportements vis à vis de leurs fonctionnalités. Ces comportements sont de trois ordres :

- `ResultSet type` : indique la façon dont on peut manipuler les données portées par ce `ResultSet` ;
- `ResultSet concurrency` : la capacité d'un `ResultSet` de mettre à jour ou non les données qu'il porte ;
- `ResultSet holdability` : le comportement d'un `ResultSet` lorsque le `Statement` auquel il est attaché est fermé ou validé.

Notons que l'interface `Statement` expose des *getters* pour les trois propriétés `resultSetType`, `resultSetConcurrency` et `resultSetHoldability`, de sorte qu'il est possible d'interroger un objet `Statement` construit avec les valeurs par défaut.

2.4.6.1. Types de ResultSet

Pour comprendre ce qui suit, il faut s'imaginer qu'un `ResultSet` est en fait un tableau dont les colonnes sont celles qui ont été extraites par notre requête SQL, et dont les lignes sont les résultats de cette requête. Examiner les résultats de notre requête consiste donc à examiner le contenu de ce tableau, ligne par ligne. La ligne courante est désignée par un **curseur**.

Le type d'un `ResultSet` recouvre deux choses : les mouvements possibles pour le curseur, et la sensibilité du résultat à la base de données sous-jacente. Effectivement, la spécification JDBC ne précise pas si le tableau de résultat doit être lu en une fois, au moment de l'exécution de la requête, ou par paquets, au fur et à mesure que l'on balaye ses lignes. Dans le deuxième cas, la question peut se poser : est-on en train de lire la base telle qu'elle était au moment de la requête, ou bien ce que l'on voit prend-il en compte des modifications qui pourraient avoir eu lieu par ailleurs ?

Il y a trois types de `ResultSet`, définis par des constantes de cette interface :

- `TYPE_FORWARD_ONLY` : c'est le mode par défaut. Le curseur ne peut se déplacer que d'une ligne à la fois, et uniquement vers l'avant. Le mode de sensibilité n'est pas défini, dépend de la base de données que l'on utilise, et de son pilote.
- `TYPE_SCROLL_INSENSITIVE` : signifie que tous les mouvements du curseur sont possibles, y compris son positionnement sur une ligne directement. Le mode **insensible** signifie que des modifications faites à la base ne sont pas transmises au `ResultSet`, ce qui signifie que la lecture plusieurs fois d'une même ligne renverra toujours le même résultat. En revanche, on ne sait pas si les données sont lues au moment de l'exécution de la commande SQL, ou au moment de la lecture d'une ligne.
- `TYPE_SCROLL_SENSITIVE` : de même, tous les mouvements du curseur sont possibles. Les changements faits sur la base de données sont reflétés dans le `ResultSet`. Donc deux lectures successives d'une même ligne peuvent donner des résultats différents.

Le type de ResultSet que l'on veut peut être imposé lors de la création d'un Statement, en fixant son paramètre resultSetType. On peut tester notre connexion afin de savoir si elle supporte le type de ResultSet que l'on veut, comme dans l'exemple suivant.

Exemple 17. Fixer le type d'un ResultSet, utilisation de DatabaseMetaData

```
// récupération des informations sur notre base
DataBaseMetaData dbdm = connection.getMetaData() ;

// notre base supporte-t-elle le mode TYPE_SCROLL_SENSITIVE ?
if (dbdm.supportsResultSetType(ResultSet.TYPE_SCROLL_SENSITIVE)) {

    // traitement
    PreparedStatement psmt = connection.prepareStatement(
        "select Marins where nom like ?", ResultSet.TYPE_SCROLL_SENSITIVE) ;
    ...
}
```

2.4.6.2. Mise à jour au travers d'un ResultSet : concurrency

Deux valeurs sont disponibles sur resultSetConcurrency :

- CONCUR_READ_ONLY : aucune mise à jour des données n'est possible au travers de ce ResultSet. Il s'agit du mode par défaut.
- CONCUR_UPDATABLE : il est possible de mettre à jour les données de ce ResultSet.

La encore, tous les pilotes ne supportent pas nécessairement la mise à jour des données au travers du ResultSet. On peut interroger les méta-données de la base de la même façon que pour le type de la façon suivante.

Exemple 18. Fixer la concurrence d'un ResultSet

```
// récupération des informations sur notre base
DataBaseMetaData dbdm = connection.getMetaData() ;

// notre base supporte-t-elle le mode CONCUR_UPDATABLE ?
if (dbdm.supportsResultSetConcurrency(ResultSet.CONCUR_UPDATABLE)) {

    // traitement
    PreparedStatement psmt = connection.prepareStatement(
        "select Marins where nom like ?",
        ResultSet.TYPE_SCROLL_SENSITIVE,
        ResultSet.CONCUR_UPDATABLE) ;
    ...
}
```

2.4.6.3. Comportement lors de la fermeture du statement : holdability

Lorsque la méthode commit() est appelée sur la transaction, manuellement ou automatiquement, il se peut que les ResultSet ouverts sur cette connexion soient également fermés, ce qui n'est pas toujours le résultat désiré. Après tout, si ces ResultSet sont ouverts en lecture seule, rien ne devrait pouvoir s'opposer à ce que l'on puisse continuer à lire les données qu'ils contiennent.

Deux valeurs sont possibles pour le paramètre `resultSetHoldability` :

- `HOLD_CURSORS_OVER_COMMIT` : les curseurs des `ResultSet` sont conservés lors d'un `commit()` ;
- `CLOSE_CURSORS_AT_COMMIT` : les curseurs sont fermés lors d'un `commit()`.

Ici la norme JDBC ne définit pas de comportement par défaut, il faut donc interroger la connexion afin de savoir dans quel mode on se trouve.

Exemple 19. Fixer la concurrence d'un `ResultSet`

```
// récupération des informations sur notre base
DataBaseMetaData dbdm = connection.getMetaData() ;

// notre base supporte-t-elle le mode CONCUR_UPDATABLE ?
if (dbdm.getResultSetHoldability(ResultSet.HOLD_CURSORS_OVER_COMMIT)) {

    // traitement
    PreparedStatement psmt = connection.prepareStatement(
        "select Marins where nom like ?",
        ResultSet.TYPE_SCROLL_SENSITIVE,
        ResultSet.CONCUR_UPDATABLE,
        ResultSet.HOLD_CURSORS_OVER_COMMIT) ;

    ...

}
```

2.4.6.4. Lecture du résultat

On obtient un objet `ResultSet` le plus souvent en invoquant la méthode `executeQuery(String)` d'un objet `Statement`.

Un `ResultSet` peut être vu comme un tableau de résultats, dont chaque colonne est un champ, et chaque ligne un enregistrement. La lecture des lignes d'un `ResultSet` se fait au travers d'un curseur, que l'on peut déplacer ligne par ligne. Lorsque l'on obtient un objet `ResultSet`, par exécution de la méthode `executeQuery(String)`, ce curseur est positionné sur une ligne virtuelle, qui se trouve avant la première ligne du tableau.

La première catégorie de méthodes proposées par l'interface `ResultSet` va donc nous permettre de déplacer le curseur dans ce tableau.

2.4.6.4.1. Déplacement du curseur d'un `ResultSet`

Voici les méthodes :

- `next()` : déplace le curseur sur la ligne suivante. Elle retourne `true` si le curseur est positionné sur une ligne, `false` si le curseur a dépassé la fin du tableau. La valeur du retour de cette méthode doit obligatoirement être testée avant la lecture de la ligne courante.
- `previous()` : déplace le curseur sur la ligne précédente. Retourne `true` ou `false`, suivant que le curseur se positionne sur une ligne, ou en dehors du tableau.
- `first()`, `last()` : positionne le curseur sur la première ligne ou la dernière ligne du tableau. Retourne `true` ou `false` suivant que cette ligne existe ou pas.
- `beforeFirst()`, `afterLast()` : positionne le curseur sur l'une des lignes virtuelles se trouvant avant la première ligne, ou après la dernière. Ce mouvement est toujours possible, même sur un tableau vide.

- `relative(int rows)` : déplace le curseur du nombre de lignes indiqué. Le déplacement a lieu vers le bas du tableau si ce nombre est positif, vers le haut s'il est négatif. Si `rows` vaut 0, alors le curseur ne bouge pas. Si le tableau ne comporte pas assez de ligne, vers le haut ou vers le bas, alors le curseur se positionne sur l'une des lignes virtuelles avant la première ligne, ou après la dernière. Cette méthode retourne `true` si le curseur a été positionné sur une ligne, `false` dans le cas contraire.
- `absolute(int rows)` : positionne le curseur en absolu dans le tableau. La première ligne du tableau est numérotée 1, donc `absolute(1)` positionne le curseur sur cette première ligne. On peut passer un paramètre négatif à cette méthode. Dans ce cas, les lignes sont numérotées à partir de la dernière, et en commençant par -1. Ainsi `absolute(-1)` positionne le curseur sur la dernière ligne du tableau, `absolute(-2)` sur l'avant-dernière etc... Si `rows` est plus grand que le nombre de lignes, alors le curseur se positionne sur la ligne virtuelle qui se trouve après le tableau si `rows` est positif, et sur la ligne virtuelle avant le tableau s'il est négatif. L'appel à `absolute(0)` positionne le curseur avant la première ligne du tableau.

Si notre `ResultSet` est de type `FORWARD_ONLY`, alors la seule méthode possible est `next()`.

2.4.6.4.2. Lecture des valeurs

Une fois le curseur positionné sur une ligne valide, il est possible d'accéder aux valeurs de chaque cellule. Pour cela, `ResultSet` expose une collection de méthode de type `get<Type>()`, qui permet de lire la valeur de chaque cellule dans son bon type.

Ces méthodes peuvent prendre deux types de paramètre :

- Un entier de type `int`, qui doit correspondre au numéro de la colonne que l'on cherche. Les colonnes sont numérotées à partir de 1.
- Une chaîne de caractères `String`, qui doit correspondre au nom de la colonne.

Notons que l'on peut obtenir le numéro d'une colonne par la méthode `findColumn(String)` de `ResultSet`.

Exemple 20. Lecture du résultat d'une requête

```
// création d'un prepared statement sur une requête simple
PreparedStatement psmt = connection.prepareStatement("select * from Marins
where id = ?") ;

// exécution de la requête
psmt.setInt(1, 15) ;
ResultSet rs = psmt.executeQuery() ;

// analyse du résultat
ResultSetMetaData md = rs.getMetaData() ;
int columnCount = md.getColumnCount() ;

// lecture du résultat
if (rs.next()) {

    for (int columnIndex = 1 ; columnIndex <= columnCount ; columnIndex++)
    {

        // lecture du type de notre colonne
        int columnType = md.getColumnType(columnIndex) ;

        if (columnType == java.sql.Type.INT) {

            String columnName = md.getColumnName(columnIndex) ;
            int value = rs.getInt(columnIndex) ;
```

```

    } else if (columnType == java.sql.Type.STRING) {

        String columnName = md.getColumnName(columnIndex) ;
        String value = rs.getString(columnIndex) ;

    } // on peut ainsi balayer tous les types de notre table
}

```

2.4.6.4.3. Cas des valeurs nulles

Lors de la lecture des valeurs de notre ligne, une conversion est effectuée du type SQL vers le type Java associé. Un problème se pose pour les int (par exemple), puisqu'un int en Java est un type de base, qui ne peut pas être nul. En fait, un entier nul en SQL vaudra 0 en Java, et un booléen vaudra false.

Dans ce cas, on peut appeler la méthode `wasNull()`, qui ne prend pas d'argument, et qui nous retourne true si la dernière lecture qui a été faite, était en fait une valeur nulle.

Exemple 21. Utilisation de `rs.wasNull()`

```

// reprenons le morceau de code de l'exemple précédent
if (columnType == java.sql.Type.INT) {

    String columnName = md.getColumnName(columnIndex) ;
    int value = rs.getInt(columnIndex) ;
    if (rs.wasNull()) {
        // dans ce cas la valeur SQL de value est null
    }
}

```

2.4.6.4.5. Remarque sur les dates

Il existe trois types de date en SQL : DATE, TIME et TIMESTAMP. Ces trois types sont associés à trois types Java : `java.sql.Date`, `java.sql.Time` et `java.sql.Timestamp`. Le piège est qu'il existe aussi une classe Java `java.util.Date`, étendue par `java.sql.Date`, et qu'il ne faut pas utiliser dans le contexte de JDBC.

2.4.6.5. Mise à jour du résultat

La mise à jour d'une base de données directement au travers d'un `ResultSet` est possible si le `resultSetConcurrency` est de type `CONCUR_UPDATABLE`. Dans ce cas, trois types d'opérations sont possibles :

- la modification des champs d'une ligne ;
- l'ajout d'une ligne ;
- la suppression d'une ligne.

2.4.6.5.1. Modification d'une ligne

Il est possible de modifier la ligne courante d'un `ResultSet` en utilisant une des méthodes `update<Type>(int, Type)`, qui prend en paramètre le numéro de la colonne et la nouvelle valeur à attribuer à cette colonne pour cette ligne.

On peut de cette façon modifier autant de colonnes que l'on souhaite sur la ligne courante. La modification sera prise en compte sur l'appel à la méthode `updateRow()` du `ResultSet`. On peut également annuler ces modifications par l'appel à la méthode `cancelRowUpdates()`.

Exemple 22. Mise à jour d'un `ResultSet`

```
// création d'un statement
PreparedStatement psmt = connection.prepareStatement(
    "select nom, prenom, ddnaissance from Marins where nom = ?" ) ;

// exécution de la requête
psmt.setString(1, "Tabarly") ;
ResultSet rs = psmt.executeQuery() ;

// mise à jour du résultat
if (rs.next()) {

    // modification du resultset
    rs.updateString("prenom", "Eric") ;
    rs.updateInt("ddnaissance", 1931) ;

    // prise en compte
    rs.updateRow() ;
}
```

Notons qu'un `ResultSet` ne "voit" pas nécessairement les modifications que l'on fait sur lui. Cela signifie que si l'on lit la valeur d'une cellule que l'on vient de modifier, il n'est pas garanti que l'on lise la valeur modifiée, quand bien même elle a été correctement prise en compte.

On peut interroger la méthode `DatabaseMetaData.ownUpdatesAreVisible(int type)` afin de savoir si un `ResultSet` voit les modifications qu'on lui applique, pour un type (`java.sql.Types`) donné.

Exemple 23. Tester si une modification est visible dans un `ResultSet`

```
DatabaseMetaData md = connection.getMetaData() ;

// le type du result set peut valoir TYPE_FORWARD_ONLY,
// TYPE_SCROLL_INSENSITIVE ou TYPE_SCROLL_SENSITIVE
if (md.ownUpdatesAreVisible(resultSet.getType())) {

    // les modifications de ce result set sont visibles
}
```

De même, un `ResultSet` peut voir ou non les modifications faites dans d'autres `ResultSet` (et donc éventuellement dans d'autres transactions), et l'on peut tester ce comportement grâce à la méthode `DatabaseMetaData.otherUpdatesAreVisible(int type)`

2.4.6.5.2. Suppression d'une ligne

La suppression de la ligne courante se fait simplement par appel à la méthode `deleteRow()`.

On peut tester si notre `ResultSet` voit ses propres suppressions ou pas, par appel à la méthode `DatabaseMetaData.ownDeletesAreVisible(int type)`. Si elles le sont, alors la méthode `rowDeleted()` retourne `true` si la ligne a été effacée, `false` sinon. Il existe aussi une méthode

DatabaseMetaData.otherDeletesAreVisible(int type) qui permet de savoir si les effacements de lignes menées dans d'autres ResultSet (et donc éventuellement dans d'autres transactions) sont visibles ou non.

Il est important de savoir si un ResultSet est sensible ou non aux lignes que l'on efface dedans, car dans certaines implémentations, les lignes effacées ne sont pas retirées, mais remplacées par des lignes vides.

Dans ce cas, il faut prendre garde à bien vérifier que l'on n'est pas sur une ligne fantôme en appelant rowDeleted() systématiquement sur chaque ligne avant d'analyser son contenu.

On peut savoir si l'implémentation de JDBC que l'on utilise efface réellement les lignes d'un ResultSet ou les remplace par des lignes vides en appelant la méthode deletesAreDetected() de ResultSet. Cette méthode renvoie true si une ligne effacée est remplacée par une ligne vide ou invalide.

2.4.6.5.3. Création d'une ligne

La création d'une ligne dans un ResultSet est une option du standard, qui n'est pas supportée par tous les pilotes.

L'insertion d'une ligne est un processus qui se déroule en trois temps :

1. Placer le curseur sur une ligne virtuelle particulière par appel à la méthode moveToInsertRow().
2. Renseigner les valeurs de cette ligne pour toutes les colonnes, par les méthodes update<Type>(Type).
3. Insérer cette nouvelle ligne dans le tableau par appel à la méthode insertRow().

Une fois notre nouvelle ligne insérée, on peut positionner le curseur dessus en appelant la méthode moveToCurrentRow(). Voyons ceci sur un exemple.

Exemple 24. Création d'une ligne dans un ResultSet

```
// création d'un result set
ResultSet rs = psmt.executeQuery() ;

// positionnement sur la ligne d'insertion
rs.moveToInsertRow() ;

// préciser la valeur de toutes les colonnes
rs.updateString("nom", "Tabarly") ;
rs.updateString("prenom", "Eric") ;
rs.updateInt("dnnaissance", 1931) ;

// valider la création de la ligne
rs.insertRow() ;

// positionnement du curseur sur la ligne nouvellement créée
rs.moveToCurrentRow() ;
```

La encore, c'est le résultat de l'appel aux méthodes DatabaseMetaData.ownInsertsAreVisible(int type) et DatabaseMetaData.otherInsertsAreVisible(int type) qui nous dira si le ResultSet voit les modifications qu'il a lui-même faite sur la base.

2.4.6.6. Fermeture d'un ResultSet

Un ResultSet peut être (et doit être !) explicitement fermé par un appel à sa méthode close(). Une fois fermé, on ne peut plus accéder à son contenu.

Chapitre 3. Design Patterns

3.1. Introduction

Un design pattern consiste en une solution éprouvée à un problème connu et récurrent. Un ensemble de vingt-trois design patterns sont disponibles dans un ouvrage de référence : « DESIGN PATTERNS - Catalogue de modèles de conception réutilisables ».

La programmation orientée objet soulève des problèmes de conception assez retords. En effet, l'objectif de la POO est de construire des logiciels qui soient souples et facilement maintenables. Concevoir de bonnes classes d'objets métiers n'est pas suffisant. Il nous faut encore assembler tous ces objets de manière correcte afin de construire des applications logicielles évolutives. Des contraintes très fortes font que certaines classes non présentes sur les diagrammes UML issus de l'analyse du domaine de l'application vont apparaître. Ces sont des classes purement de conception.

Seule l'expérience peut nous apporter la connaissance nécessaire pour prendre de bonnes décisions de conception. Le problème, c'est que le développement coûte de l'argent. Le droit à l'erreur est donc difficilement compris par les clients. Heureusement, comme dans tous les domaines, l'expérience est quelque chose qui peut être partagée. Les problèmes que nous allons rencontrer dans le développement de nos programmes sont souvent toujours les mêmes. D'autres développeurs ont certainement buté sur les mêmes difficultés. Certains développeurs ont compris qu'il fallait regrouper une liste de modèles de conception à des problèmes connus dans un catalogue afin de partager leurs solutions avec d'autres. Ce partage permet de tester ces solutions à travers de milliers de programmes et de mettre rapidement en évidence les « bonnes » solutions à appliquer dans tel ou tel cas.

Un design pattern n'est pas quelque chose de théorique, mais plutôt du bon sens. Nous avons dans certains de nos problèmes appliqués tel ou tel design pattern sans le savoir. L'intérêt des catalogues de design patterns est qu'ils nomment des solutions. Ils facilitent donc la communication entre les développeurs. Le fait qu'un design pattern est un nom permet de le reconnaître lorsqu'on examine le code d'un collègue tout comme nous pouvons reconnaître un algorithme de tri comme le « tri à bulles » ou le « tri quick sort ».

Nous allons voir que les designs patterns sont basés sur les interfaces et la délégation des actions. Ils vont donc compliquer légèrement notre code dans un premier temps. Nous nous apercevrons ensuite qu'ils simplifient en fait le problème à résoudre avec des solutions évidentes chargées de « bon sens ».

Un design pattern est composé de :

- Un nom
- Une description du pattern
- Un schéma UML avec des notes si nécessaires

Dans les sections suivantes, nous présenterons certains diagrammes UML dans le cadre de l'étude de patterns. Ces diagrammes sont là pour capter l'idée que véhicule le pattern considéré. Les exemples fournis peuvent implémenter un ensemble de classes qui ne se superposera pas forcément au diagramme. Ce n'est pas anormal, car un pattern doit être adapté à chaque situation pour lui correspondre au mieux.

3.2. Les patterns

Nous allons voir les patterns suivants :

- Composite

- Décorateur
- Etat
- Fabrique Abstraite
- Observateur
- Singleton
- Stratégie
- Visiteur
- MVC (Modèle-Vue-Contrôleur)

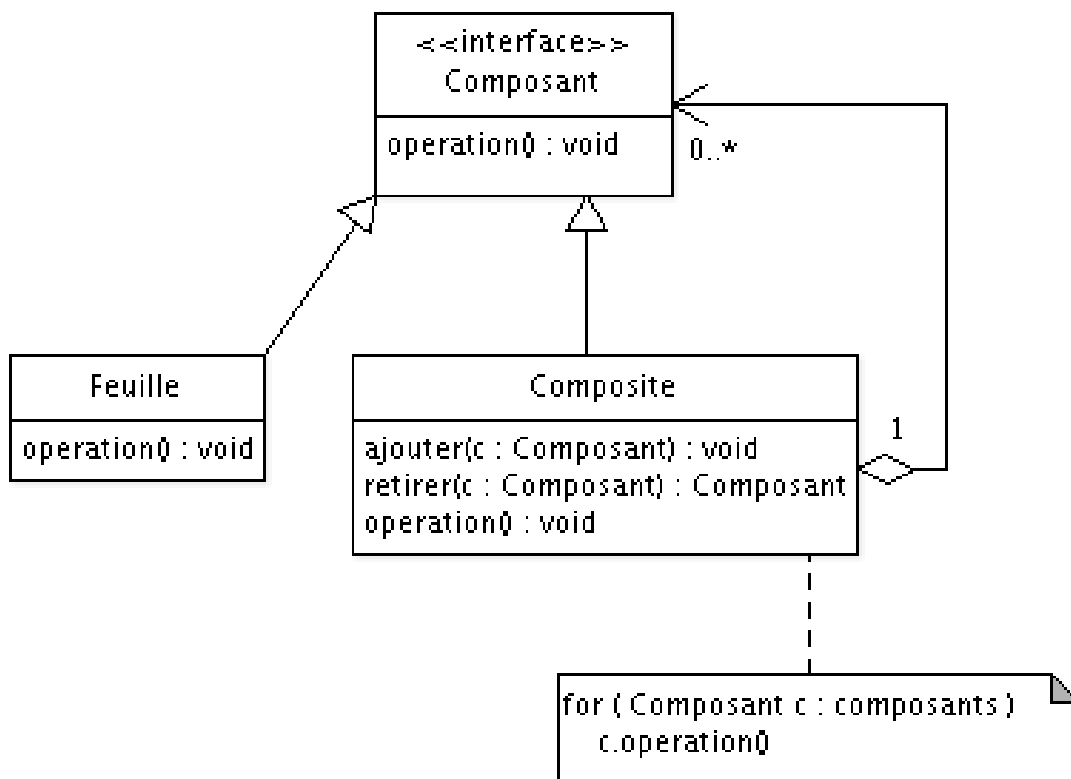
3.2.1 Composite

Le modèle *Composite* permet de composer des hiérarchies composant/composé. Il offre la possibilité au client de traiter de façon uniforme des objets individuels ou des combinaisons de ceux-ci. La profondeur de l'imbrication de la composition n'a pas d'importance.

Un cas d'école est la manipulation d'objets graphiques par une application. Une application de ce type permet de les déplacer, les supprimer, etc. Ces objets peuvent être groupés pour que la même action puisse être appliquée sur un ensemble d'objets. Un groupe d'objets est lui aussi un objet graphique. C'est un objet graphique composé. Les objets qu'il contient en sont les composants. Le groupe d'objets peut être manipulé comme l'un de ces composants (ou presque). L'utilisateur de la classe *Groupe* utilise donc l'instance du groupe comme un objet graphique simple.

Le grand avantage du modèle *Composite* est de grandement faciliter le travail avec les objets, car nous faisons abstraction du fait qu'ils soient des composants et/ou des composés. Le secret du modèle *Composite* est que les composants et les composés partagent la même interface de base.

Le schéma UML du modèle Composite est le suivant :



Le modèle composite

Nous pouvons constater que toutes les classes de ce modèle héritent de la classe *Composant*. Elles possèdent donc toutes la méthode *operation()* et sont toutes du type *Composant*. La classe *Composite* est également un « Composant ».

Une instance de cette classe peut donc être manipulée comme un composant individuel. La méthode *operation()* itère sur tous les composants et appelle leur méthode *operation()*.

3.2.1.2. Application du modèle composite pour un calcul

Le modèle composite est utilisable dans de nombreuses situations. Prenons un exemple concernant un programme de gestion d'un parc de véhicules de location. Nous devons entretenir ce parc. La société est composée d'un siège social et de plusieurs agences à travers le pays. Ces agences peuvent posséder des filiales.

Nous voulons calculer le coût d'entretien de la même façon à tous les niveaux :

- Siège social
- Agence

Grâce au modèle composite, nous pouvons utiliser les classes *SocieteMere* et *SocieteSansFiliale* de manière identique, car les deux classes héritent de la même classe abstraite *Societe*.

En utilisant le modèle composite, le calcul du coût d'entretien est homogène, quel que soit le niveau à partir duquel nous voulons le lancer. Nous pouvons également constater que nous pourrions facilement créer des filiales de filiales sans rien changer à la conception de nos classes.

```
import java.util.*;

abstract class Societe {

    protected static double coutUnitaireVehicule = 5.0;
    protected int nbrVehicules;

    public void ajouterVehicule() {
        nbrVehicules = nbrVehicules + 1;
    }

    public abstract double calculerCoutEntretien();
    public abstract boolean ajouterFiliale( Societe filiale );
}

class SocieteMere extends Societe {

    List<Societe> filiales = new ArrayList<Societe>();

    public boolean ajouterFiliale( Societe filiale ) {
        return filiales.add( filiale );
    }

    public double calculerCoutEntretien() {
        double cout = 0.0;

        for( Societe filiale : filiales )
            cout = cout + filiale.calculerCoutEntretien();

        return cout + nbrVehicules * coutUnitaireVehicule;
    }
}
```

```

class SocieteSansFiliale extends Societe {

    public boolean ajouterFiliale( Societe filiale ) {
        return false;
    }

    public double calculerCoutEntretien() {
        return nbrVehicules * coutUnitaireVehicule;
    }
}

public class Utilisateur {
    public static void main( String[] args ) {
        Societe societel = new SocieteSansFiliale();
        societel.ajouterVehicule();
        Societe societ2 = new SocieteSansFiliale();
        societ2.ajouterVehicule();
        societ2.ajouterVehicule();

        Societe groupe = new SocieteMere();
        groupe.ajouterFiliale( societel );
        groupe.ajouterFiliale( societ2 );
        groupe.ajouterVehicule();

        System.out.println( "Cout d'entretien total du groupe : " +
                             groupe.calculerCoutEntretien() );
    }
}

```

3.2.2. Décorateur

3.2.2.1. Description

Le problème de conception le plus important en programmation orientée objet est l'adaptation au changement. Le changement est inévitable. Il nous faut donc minimiser l'impact de ce changement sur notre application. Les designs patterns sont conçus pour prévoir ces modifications et en amortir le coût.

Le pattern décorateur ressemble au pattern stratégie dans le sens où il permet également de modifier le comportement d'un objet. Ces deux patterns nous aident à respecter un précepte important en conception orientée objet : nous devons créer des classes qui n'ont pas besoin d'être modifiées, car elles intègrent des mécanismes permettant de leur ajouter des extensions, sans avoir à toucher le code d'implémentation.

Note **Principe de conception**

Les classes doivent être ouvertes à l'extension, mais fermées à la modification.

Le pattern stratégie est un exemple du but à atteindre.

Le pattern décorateur propose d'attacher dynamiquement des comportements à une classe. L'objectif n'est pas de sélectionner un comportement parmi plusieurs, mais bien d'y ajouter des responsabilités supplémentaires au comportement initial. Le décorateur fournit une alternative à l'héritage de classe, car les responsabilités sont ajoutées dynamiquement au gré des besoins.

3.2.2.2. Diagramme de classes

Voici le schéma UML de Décorateur :

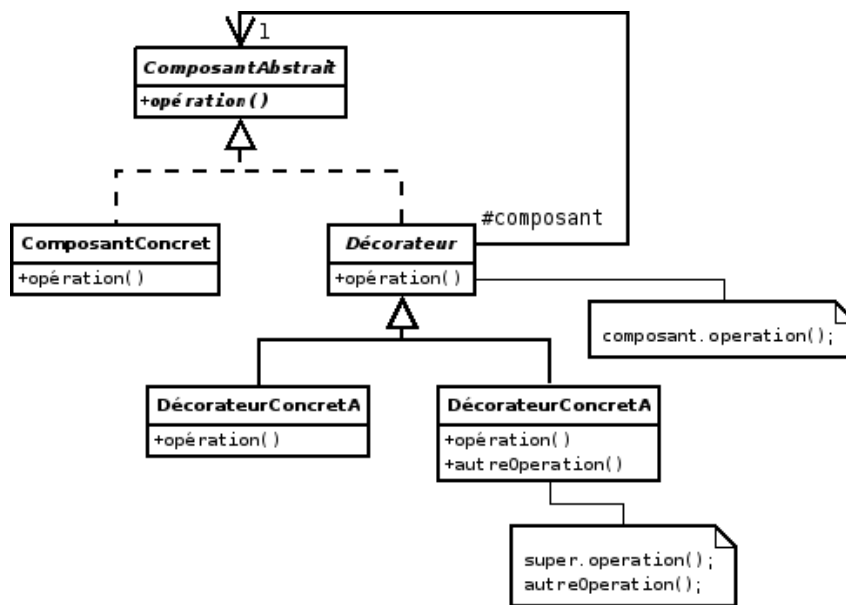


Figure 1. Pattern Décorateur

3.2.2.3. Classes participantes

- *ComposantAbstrait* est l'interface commune aux composants concrets et aux décorateurs.
- *ComposantConcret* est un composant qui peut être instancié. C'est ce type de composant qui doit recevoir de nouvelles responsabilités.
- *Décorateur* est une classe abstraite qui implémente l'interface *ComposantAbstrait*. Elle possède une référence vers un composant.
- *DécorateurConcretA* et *DécorateurConcretB* sont des classes concrètes qui héritent de *Décorateur*. Elles contiennent les responsabilités supplémentaires à attacher aux composants décorés.

3.2.2.4. Domaine d'application

- L'application doit pouvoir réaliser des assemblages de responsabilités inconnus lors de la création des classes d'objets. L'héritage fige les assemblages à la compilation. Cette contrainte n'est pas souhaitable dans certains cas.
- Utiliser l'héritage pour étendre les responsabilités des objets amène à la multiplication des classes. Le décorateur est préférable.

3.2.2. 5. Exemple

Nous allons prendre comme exemple un composant graphique¹¹ *ChampTexte*. Ce composant gère un champ de texte. Par défaut, il ne possède pas de bordure et pas d'ascenseur. Nous pouvons ajouter ces fonctionnalités dynamiquement en cours d'exécution.

```
interface Composant {
```

¹¹ Dans l'exemple, nous illustrons seulement le pattern. L'aspect graphique n'est pas abordé.

```

        void dessiner();
    }

    class ChampTexte implements Composant {
        private int largeur, hauteur;

        public ChampTexte( int w, int h ) {
            largeur = w;
            hauteur = h;
        }

        public void dessiner() {
            System.out.println( "ChampTexte: " + largeur + ", " + hauteur );
        }
    }

    abstract class Decorateur implements Composant {
        private Composant composant;
        public Decorateur( Composant w ) {
            composant = w;
        }
        public void dessiner() {
            composant.dessiner();
        }
    }

    class DecorateurBordure extends Decorateur {
        public DecorateurBordure( Composant w ) {
            super( w );
        }
        public void dessiner() {
            super.dessiner();
            System.out.println( "    DecorateurBordure" );
        }
    }

    class DecorateurAscenseur extends Decorateur {
        public DecorateurAscenseur( Composant w ) {
            super( w );
        }

        public void dessiner() {
            super.dessiner();
            System.out.println( "    DecorateurAscenseur" );
        }
    }

    public class DecorateurComposant {
        public static void main( String[] args ) {
            Composant unComposant = new DecorateurBordure( new DecorateurAscenseur(
            new ChampTexte( 80, 24 ) ) );
            unComposant.dessiner();
        }
    }
}

```

3.2.2.6. Conclusion

Le pattern décorateur n'a pas comme objectif de se substituer complètement au composant qu'il enveloppe, mais seulement d'attacher des responsabilités à certaines méthodes. Il offre plus de souplesse que l'héritage statique, car il évite d'attacher « en dur » des responsabilités aux classes existantes. Néanmoins, un décorateur ne peut être considéré comme complètement apparenté au composant enveloppé, car l'interface implémentée par le décorateur est beaucoup plus restreinte.

3.2.3. Etat

Nous savons qu'un objet possède toujours un état. Cet état est en général mémorisé dans les attributs de l'instance. Le comportement d'un objet dépend de son état. Prenons comme exemple un moteur de voiture. Nous nous limiterons aux aspects qui nous intéressent dans ce cas particulier.

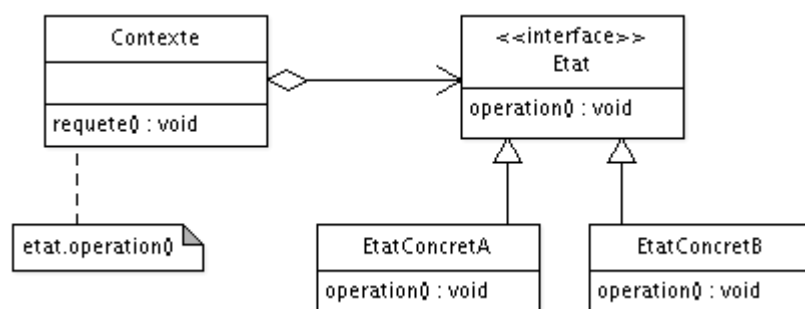
Notre classe *Moteur* possède un seul attribut « enMarche » de type booléen (nous faisons simple dans notre exemple). Elle définit les méthodes suivantes :

- demarrer
- arreter
- appuyerAccelerateur
- relacherAccelerateur

Il paraît évident que les méthodes précédentes auront des comportements différents selon que le moteur est en marche ou non. La solution qui vient à l'esprit immédiatement est de gérer les comportements dans les méthodes à coups de *if* ... :

```
...  
if (this.enMarche) {  
    ...  
} else {  
    ...  
}
```

Si les états ne sont pas nombreux, cette solution est viable. Elle est toutefois assez figée. Ajouter un nouvel état impose de modifier de manière importante les méthodes existantes en risquant d'introduire des erreurs dans un code correct. Une autre méthode est d'utiliser le pattern « Etat ». Ce modèle nous propose d'extraire les comportements d'un objet et de les placer dans des classes séparées. Observons le diagramme UML de ce pattern sur le schéma ci-dessous :



Le modèle état

Figure 1. UML - Modèle Etat

Dans le schéma, nous pouvons voir qu'une instance de la classe *Contexte* possède une référence sur des classes de type *Etat*. Deux classes concrètes *EtatConcret1* et *EtatConcret2* implémentent cette interface. Nous constatons que l'objet délègue l'exécution de la méthode *operation()* à la référence de type *Etat* active. Ce n'est donc plus notre objet qui réalise l'action, mais une instance de type *EtatConcret1* ou *EtatConcret2*. Nous n'avons plus qu'à basculer correctement d'un état à l'autre et le tour est joué. Il faut penser à passer une référence sur l'objet aux instances représentant les états, car elles devront pouvoir accéder aux attributs de l'objet. Afin de présenter un exemple complet, voici le code pour notre moteur :

```

interface EtatMoteur {
    public void demarrer();
    public void arreter();
    public void appuyerAccelérateur();
    public void relacherAccelérateur();
}

class EtatMoteurArrete implements EtatMoteur {
    public EtatMoteurArrete(Moteur moteur) {
        this.moteur = moteur;
    }

    public void demarrer() {
        System.out.println("Mise en marche du moteur");
        this.moteur.etat = new EtatMoteurEnMarche(this.moteur);
    }

    public void arreter() {
        System.out.println("Impossible de stopper un moteur non démarré");
    }

    public void appuyerAccelérateur() {
        System.out.println("Accélération Impossible : moteur non démarré");
    }

    public void relacherAccelérateur() {
        System.out.println("Décélération sans effet : moteur non démarré");
    }

    protected Moteur moteur;
}

class EtatMoteurEnMarche implements EtatMoteur {
    public EtatMoteurEnMarche(Moteur moteur) {
        this.moteur = moteur;
    }

    public void demarrer() {
        System.out.println("Moteur déjà en marche");
    }

    public void arreter() {
        System.out.println("Arrêt du moteur");
        this.moteur.etat = new EtatMoteurArrete(this.moteur);
    }

    public void appuyerAccelérateur() {
        System.out.println("Accélération");
    }

    public void relacherAccelérateur() {
        System.out.println("Décélération");
    }

    protected Moteur moteur;
}

class Moteur {
    public Moteur() {
        etat = new EtatMoteurArrete(this);
    }

    public void demarrer() {
        etat.demarrer();
    }
}

```

```

    }

    public void arreter() {
        etat.arreter();
    }

    public void appuyerAccelerateur() {
        etat.appuyerAccelerateur();
    }

    public void relacherAccelerateur() {
        etat.relacherAccelerateur();
    }

    protected EtatMoteur etat;
}

public class Etat {

    public static void main(String[] args) {
        Moteur moteur = new Moteur();

        moteur.demarrer();
        moteur.appuyerAccelerateur();
        moteur.relacherAccelerateur();

        System.out.println("");

        moteur.arreter();
        moteur.arreter();
        moteur.appuyerAccelerateur();
        moteur.relacherAccelerateur();

        System.out.println("");

        moteur.demarrer();
        moteur.demarrer();
    }
}

```

Nous utilisons le modèle état quand :

- le comportement d'un objet dépend fortement de son état
- les instructions conditionnelles permettant de coder les états d'un objet rendent l'implémentation trop complexe

3.2.4. Builder¹²

Le [Gang Of Four](#) nous donne la définition suivante du design pattern « builder » :

« Le pattern Builder est utilisé pour la création d'objets complexes dont les différentes parties doivent être créées suivant un certain ordre ou algorithme spécifique. Une classe externe contrôle l'algorithme de construction »

¹² Source : Publicis Sapient

On a également le rôle de ce design pattern :

« Separate the construction of a complex object from its representation so that the same construction process can create different representations. »

« dissocier la construction d'un objet de sa représentation, afin que le même processus de construction ait la possibilité de créer des représentations différentes »

Typiquement, ce design pattern sera préconisé dans les situations où au moins :

- L'objet final est imposant, et sa création complexe ;
- Beaucoup d'arguments doivent être passés à la construction de l'objet, afin d'avoir un design lisible ;
- Certains de ces arguments sont optionnels (par ex. : un bateau n'a pas forcément de capitaine, mais toujours une taille), ou ont plusieurs variations (la taille peut par exemple être passée en mètres ou en pieds).

3.2.4.1 Le builder fluent : beaucoup de méthodes chaînées, un seul build

C'est l'image que l'on se fait le plus souvent du builder : une suite de méthodes chaînées, suivie d'un **build** final qui agrège les données, généralement dans une **innerClass**

```
Boat littleboat = new BoatBuilder().withSize(2).build();
class Boat{
    int size;
    Boat(int size){ ...}

    class BoatBuilder(){
        int size;
        BoatBuilder withSize(int size){
            this.size=size;
            return this;
        }
        Boat build(){
            return new Boat(size);
        }
    }
}
```

Ce builder permet ainsi facilement d'avoir des paramètres optionnels, à la différence d'un constructeur de base. Mais les paramètres passés sont injectés tels quels dans les attributs de classes, sans traitements.

3.2.4.2 Le fluent interface

Ce n'est pas un builder à proprement parler, mais le principe est le même : il s'agit en fait de l'utilisation du design pattern fluent lors de la construction d'un objet mutable (également appelée désignation chaînée : les setters préfixés par '**with**' renvoient également l'instance à la place de void)

```
Boat erika = new Boat().withCaptain(haddock).withSize(2);
```

Utilisation : le fluent interface permet d'éviter efficacement l'anti-pattern « **Telescoping Constructor Pattern** », en particulier sur les instances immutables :

```
Boat(int size) { ... }
Boat(int size, Human captain) { ... }
Boat(int size, Human captain, boolean inWood, boolean motor) { ... }
```

```
Boat(int size, Human captain, boolean inWood, boolean motor, boolean sail) {
... }

Boat aurore = new Boat(haddock , false, true, false);
```

3.2.4.3 Le builder Command : beaucoup de setter, un seul build() (mais qui fait beaucoup)

Relativement similaire au builder-fluent décrit ci-dessus, la principale différence est qu'un traitement est effectué au sein de la méthode **build**.(transformer un nom en Human)

```
class Boat(){
    Human captain;
    ...
}

class BoatBuilder(){
    String ownerName;
    BoatBuilder withOwnerName(String ownerName){
        ...
    }

    Boat build(){
        Boat boat = new Boat();
        boat.setCaptain(new Human(ownerName));
        return boat;
    }
}

Boat boat = new BoatBuilder().withOwnerName("Haddock").build();
```

Ce pattern permet ainsi des traitements plus complexes lors de la conception: l'objet généré est plus que la somme des paramètres passés en entrée.

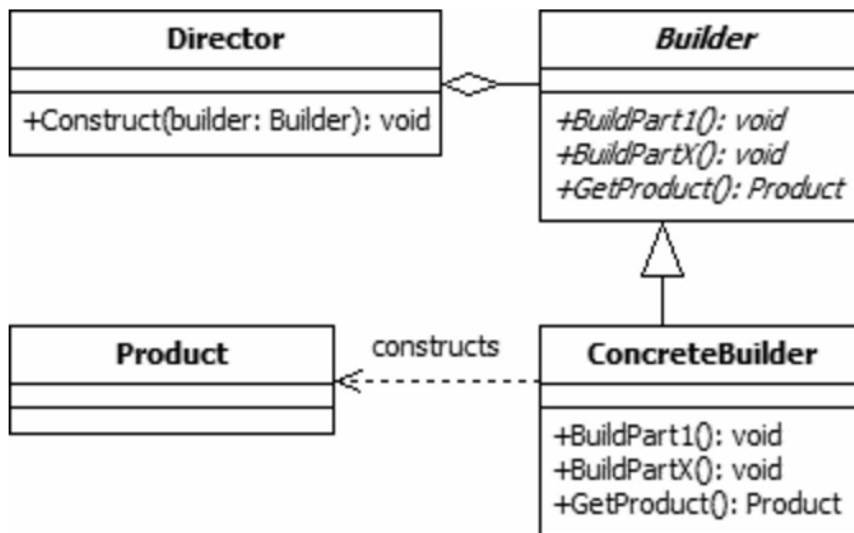
L'avantage principal est que nous n'avons pas à construire d'objets complexes avant de les passer en entrée : ces traitements peuvent être faits au sein du build.

Le design pattern command

Il s'agit en fait d'un design pattern command, dont la méthode **void execute()** a été sournoisement transformée en **build()**. Le rôle du design pattern command est en effet très similaire à celui du pattern builder : « Encapsuler toutes les informations nécessaires pour effectuer une action séparément à une date ultérieure »

3.2.4.4 Le builder officiel : beaucoup de build, une seule class Director.

Lorsque le Gang Of Four a défini le builder, il a défini en fait l'implémentation d'un monteur, qui repose sur un duo de classes **builder-director** :



Étape par étape, le **Director** va demander au builder de construire les différentes parties d'un objet composé (voir design pattern composite), la récupération des données et l'ordonnancement des différentes étapes de construction étant gérée par une class « Director ».

Idéalement, la classe **Builder** est abstraite, afin de permettre différentes versions du montage/implémentation de la classe finale.

Cela correspond à cette implémentation:

```

class LicenceBuilder {
    License buildPartLicense(Boat boat, String ownerName) {
        boat.setLicenseOwner(new License(ownerName));
    }
}

abstract class AbstractBoatBuilder<TBoat extends Boat> {
    abstract TBoat buildPartHull();

    abstract void buildPartPropulsion(TBoat boat);

    void buildAdministration(TBoat boat) {
        if (boat.getOwnerLicense() != null) {
            boat.setCaptain(boat.getOwnerLicense().getNavigator());
            boat.setAuthorizedToNavigate(true);
        }
    }
}

class BoatDirector {
    AbstractBoatBuilder boatBuilder;
    LicenceBuilder licenseBuilder;

    BoatDirector(AbstractBoatBuilder boatBuilder, LicenceBuilder
licenseBuilder) {
        this.builder = boatBuilder;
        this.licenseBuilder = licenseBuilder;
    }

    public Boat build(){
        Boat boat = builder.buildPartHull();
        builder.buildPartPropulsion(boat);
    }
}
  
```



```

        licenseBuilder.buildPartLicense(boat, "Rackam LeRouge");
        boatBuilder.buildPartAdministration(boat);
        return boat;
    }
}

Boat boat = new BoatDirector(boatBuilder, licenseBuilder).build();

```

L'objectif du builder monteur est double :

- D'une part, décomposer le build en phases distinctes,
- D'autre part, autoriser différentes implémentations du monteur (à l'image des **abstract Factory**).

```

class Gallion extends Boat {...}

class GallionBuilder extends AbstractBuilder<Gallion> {
    Boat buildPartHull() {
        return new Gallion();
    }

    void buildPartPropulsion(Gallion boat) {
        boat.setPropulsion(new Sail());
    }
}

class SubMarine extends Boat {...}

class SubMarineBuilder extends AbstractBuilder<SubMarine> {
    Boat buildPartHull() {
        return new SubMarine();
    }

    void buildPartPropulsion(SubMarine boat) {
        boat.setPropulsion(new AtomicMotor());
    }

    @Override
    void buildAdministration(Submarine boat, String ownerName) {
        super.buildAdministration(boat, ownerName);
        boat.setNuclearCode("0000");
    }
}

```

La manière dont les méthodes **builder.buildPart** incorporent les modifications à l'objet composite est entièrement au choix du développeur. On aurait également pu écrire : de même, pour le passage des arguments: la définition du Gang of Four ne précise pas s'ils doivent être passés aux **builders** ou au **director**.

Quels sont les avantages du builder-monteur ?

Ce pattern permet une séparation claire entre l'ordonnancement de la création, et la construction des différentes parties de l'objet.

Séparation du builder et du director

Dans la pratique, on en arrive souvent à fusionner la classe **director** et la classe **builder** (par exemple dans la class java.lang.StringBuilder)

3.2.4.5 Pourquoi tant d'implémentations différentes du builder?

Il existe un gap important entre la définition donnée par le Gang of Four, qui précise une implémentation, et le rôle auquel répond le pattern builder : « dissocier la construction d'un objet de sa représentation. » De plus, le design UML « officiel » ne répond pas à la problématique du passage des arguments.

A voile et à moteur: le builder-fluent-command-monteur

En théorie, si l'on voulait réaliser un **builder** remplissant à la fois le rôle et la définition du Gang Of Four, on aboutirait à ceci :

```
class Boat {
    Human captain;
    Propulsion propulsion;
    License licenseOwner;
}

class LicenceBuilder {
    License buildPartLicense(Boat boat, String ownerName) {
        boat.setLicenseOwner(new License(ownerName));
    }
}

abstract class AbstractBoatBuilder<TBoat extends Boat> {
    abstract TBoat buildPartHull();

    abstract void buildPartPropulsion(TBoat boat);

    void buildAdministration(TBoat boat) {
        if (boat.getOwnerLicense() != null) {
            boat.setCaptain(boat.getOwnerLicense().getNavigator());
            boat.setAuthorizedToNavigate(true);
        }
    }
}

class BoatDirector {
    AbstractBoatBuilder boatBuilder;
    LicenseBuilder licenseBuilder;

    BoatDirector(AbstractBoatBuilder boatBuilder, LicenseBuilder
licenseBuilder) {
        this.builder = boatBuilder;
        this.licenseBuilder = licenseBuilder;
    }

    String ownerName;

    BoatDirector withOwnerName(String ownerName) {
        this.ownerName = ownerName;
        return this;
    }

    public Boat build(){
        Boat boat = builder.buildPartHull();
        builder.buildPartPropulsion(boat);
        licenseBuilder.buildPartLicense(boat, ownerName);
        boatBuilder.buildPartAdministation(boat)
        return boat;
    }
}
```

```

}

class Kayak extends Boat {...}

class KayakBuilder extends AbstractBuilder<Kayak> {

    Boat buildPartHull() {
        return new Kayak();
    }

    void buildPartPropulsion(Boat boat) {
        boat.setPropulsion(new Paddle());
    }
}

Boat boat = new Director(new KayakBuilder(), new
LicenseBuilder()).withOwnerName("Tintin").build();

```

Pourquoi ne l'utilise-t-on pas toujours (voir pas du tout) ?

L'inconvénient de ce design est qu'il devient rapidement très volumineux, et rajoute des complexités pas toujours justifiées: en effet suivant les cas, on n'aura pas besoin d'une construction ordonnée, de traitements complexes, de différentes implémentations du(des) builder(s), voir même d'injecter des paramètres. Il peut alors être pertinent d'utiliser une version « plus souple », allégée du design pattern **builder**.

3.2.4.6 Choisir le bon patron de conception

Complexité des paramètres vs Complexité du code	Beaucoup de paramètres	Peu de paramètres, mais beaucoup de combinaisons possibles ou présence de paramètres optionnels	Peu de paramètres, Peu de combinaisons possibles et pas d'optionnels
Beaucoup d'étapes, construction complexe	Builder-Monteur-command-fluent, avec les classes <i>Director</i> et le(les) <i>Builder</i> distinctes dans des fichiers à part	Builder-Monteur-command-fluent, avec le <i>Builder</i> et le <i>Director</i> fusionnés, mais conserver plusieurs méthodes <i>partBuild</i> distinctes	▪ Si Plusieurs implémentations différentes ou inconnues : <i>AbstractFactory</i> ▪ Sinon: <i>Factory</i> Ne pas hésiter à recourir aux méthodes privées si le besoin se fait sentir
Appels à des classes tiers.	<i>Inner Builder-Command</i> en classe interne de la classe tiers		
Peu de complexité, pas d'appels à des classes tiers	<i>Builder-Command</i> normal		<i>Constructor</i>
Aucune complexité (Recopie exacte des attributs du builder dans l'objet produit)	<i>Builder-fluent</i>		

3.2.5. Observateur

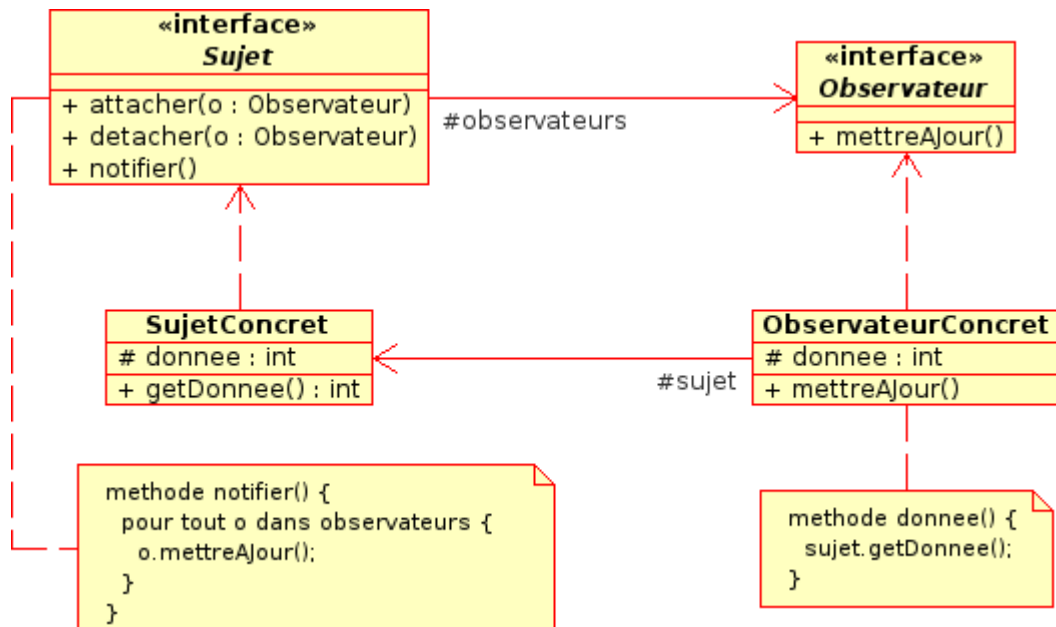
3.2.5.1. Motivation

Dans de nombreuses applications, les données manipulées doivent être affichées sous des formes différentes. Une application comme le tableur Excel en est un exemple. Les données d'un document

peuvent être visualisées comme une feuille de calcul ou comme un graphique. Lorsque l'utilisateur modifie les données, les deux vues (la feuille de calcul et le graphique) affichent aussitôt les changements.

Dans la terminologie des modèles, nous parlerons de l'objet manipulant les données comme étant le « Sujet » et des objets affichant les données comme les « Observateurs ».

3.2.5.2. Diagramme de classes



Le modèle observateur

Comme nous pouvons le voir sur le diagramme de classes, le sujet connaît les observateurs auxquels il devra envoyer une notification de modification. Il devra donc mémoriser la liste des observateurs. Chaque observateur devra garder une référence vers le sujet dont il devra afficher une vue.

3.2.5.3. Exemple

Voici un exemple simple mais intéressant. Nous demandons à un utilisateur d'entrer un entier au clavier. Lorsque nous modifions l'état du sujet, celui-ci notifie le changement à tous les observateurs enregistrés. Chaque observateur affiche les nouvelles informations en appliquant le formatage nécessaire.

La gestion des erreurs n'est pas active dans ce petit programme. Une remarque : le code ne reflète pas exactement le diagramme de classes du modèle observateur. C'est un phénomène assez courant. Nous n'avons pas cherché à compliquer inutilement le code pour adhérer parfaitement au modèle.

```
import java.util.*;

class Sujet {
    private Observateur[] observateurs = new Observateur[9];
    private int totalObservateurs = 0;
    private int etat;

    public void attacher( Observateur o ) {
        observateurs[totalObservateurs++] = o;
    }

    public int getEtat() {
        return etat;
    }
}
```

```

    }

    public void setEtat( int in ) {
        etat = in;
        notifier();
    }

    private void notifier() {
        for (int i=0; i < totalObservateurs; i++)
            observateurs[i].mettreAJour();
    }
}

abstract class Observateur {
    protected Sujet sujet;

    public abstract void mettreAJour();
}

class ObservateurHexadecimal extends Observateur {
    public ObservateurHexadecimal( Sujet s ) {
        sujet = s;  sujet.attacher( this );
    }

    public void mettreAJour() {
        System.out.print( " " + Integer.toHexString( sujet.getEtat() ) );
    }
}

class ObservateurOctal extends Observateur {
    public ObservateurOctal( Sujet s ) {
        sujet = s;  sujet.attacher( this );
    }

    public void mettreAJour() {
        System.out.print( " " + Integer.toOctalString( sujet.getEtat() ) );
    }
}

class ObservateurBinaire extends Observateur {
    public ObservateurBinaire( Sujet s ) {
        sujet = s;  sujet.attacher( this );
    }

    public void mettreAJour() {
        System.out.print( " " + Integer.toBinaryString( sujet.getEtat() ) );
    }
}

public class TestObservateur {
    public static void main( String[] args ) {
        Scanner sc = new Scanner(System.in);
        Sujet sub = new Sujet();
        int nombre;

        new ObservateurHexadecimal( sub );
        new ObservateurOctal( sub );
        new ObservateurBinaire( sub );

        while (true) {
            System.out.print( "\nEntrez un entier: " );
            nombre = sc.nextInt();
            sub.setEtat( nombre );
        }
    }
}

```

```
}  
}
```

Dans un programme utilisant une interface utilisateur, l'utilisateur agit sur le sujet à travers l'un des observateurs. Dans ce contexte, un observateur est appelé une vue. L'architecture mise en action est souvent nommée « Document/Vue ». Le document est matérialisé par le sujet et la ou les vues par des observateurs.

3.2.5.4. Cas d'utilisation

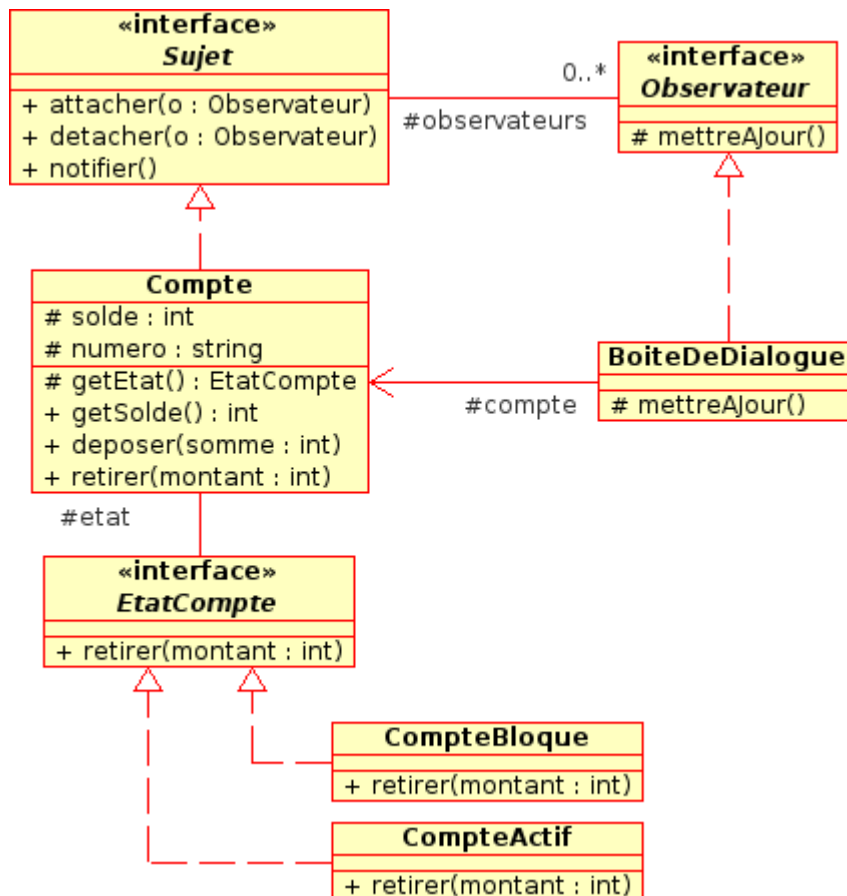
- Certaines informations manipulées par l'application doivent être affichées sous des formats différents
- Dans une interface graphique, plusieurs contrôles (zones de texte, listes déroulantes) pointent sur les mêmes données. Dès que ces données sont modifiées, tous les contrôles doivent prendre en compte les changements

3.2.5.5. Exercice - 1

En reprenant le code ci-dessus, créez un programme utilisant une interface graphique. Dans ce programme, nous aurons une boîte de dialogue permettant de saisir les valeurs et trois fenêtres chargées d'afficher le nombre sous un format adéquat. Tout comme le programme en ligne de commande, le sujet doit notifier les changements aux vues enregistrées.

3.2.5.6. Exercice - 2

En utilisant le diagramme ci-dessous, écrivez un programme graphique dont l'objectif est d'afficher les informations d'un compte bancaire. Un compte peut être actif (le solde est nul ou positif) ou bloqué temporairement (le solde est négatif). Lorsque le compte est bloqué, seule l'opération de retrait (retirer) doit générer une exception indiquant l'impossibilité d'effectuer cette action. Les autres opérations restent inchangées.



Le modèle Compte

L'état du compte, actif ou bloqué, est dynamique. Il est associé au solde. Un compte bloqué deviendra actif dès que le solde sera à nouveau positif ou nul.

Pour le programme, nous aurons deux vues (boîtes de dialogue). Une vue permettra de modifier les informations du compte (le solde) et une autre vue affichera l'état du compte, ainsi que le numéro et le montant en euros.

3.2.6. Singleton

3.2.6.1. Motivation

S'assurer qu'une seule instance d'une classe peut être créée et fournir un point d'accès global à un objet. Le développeur doit donc empêcher la possibilité d'écrire la ligne suivante au niveau de l'application :

```
MaClasse maClasse = new MaClasse();
```

Ce type de problème se rencontre fréquemment. Prenons un exemple : nous voulons concevoir une classe qui retourne un identifiant unique comme un numéro de référence. Il serait malencontreux de pouvoir créer plusieurs instances à partir de cette classe, car chacune d'entre-elles retournerait un identifiant qui pourrait entrer en collision avec un identifiant déjà existant.

Pour résoudre ce problème, nous avons le modèle Singleton. Le constructeur d'une classe implémentant ce modèle est privé. Impossible donc d'utiliser le mot clé *new*. Par contre, la classe expose une méthode statique qui renverra une instance de cette classe, instance créée au premier appel de la méthode.

3.2.6.2. Exemple

Le code ci-dessous illustre l'utilisation du modèle singleton. Nous devons générer des nombres tous différents. La génération doit donc être réalisée en un point unique de l'application. La classe *Singleton* chargée de créer ces nombres ne peut pas être instanciée dans le programme par le mot clé *new*. En effet, le constructeur est marqué comme *private*. Cette classe fournit par contre une méthode *getInstance()*. C'est cette méthode qui retourne une instance de la classe. L'instance retournée n'est créée que si nécessaire.

```
class Singleton {
    private static Singleton instance = null;
    private Integer counter = 0;

    private Singleton() { }

    public static Singleton getInstance() {
        if(Singleton.instance == null) {
            Singleton.instance = new Singleton();
        }
        return Singleton.instance;
    }

    public Integer getCounter() { return ++this.counter; }
}

public class TestSingleton {
    public static void main(String [] args) {
        // Singleton s1 = new Singleton(); // Erreur ! Ne compile pas !

        Integer i1 = Singleton.getInstance().getCounter();
        Integer i2 = Singleton.getInstance().getCounter();

        System.out.println("Valeur du compteur : " + i1);
        System.out.println("Valeur du compteur : " + i2);
    }
}
```

3.2.6.3. Cas d'utilisation

- L'application ne doit utiliser qu'une instance d'une classe et cette instance ne doit être accessible aux clients qu'à partir d'un point d'entrée unique

3.2.7. Stratégie

3.2.7.1. Description

Le pattern Stratégie définit une famille d'algorithmes, encapsule chacun d'autres eux pour les rendre interchangeables. Les algorithmes sont donc externalisés et peuvent évoluer indépendamment des objets qui les utilisent. Dans le vocabulaire des Designs Patterns, le terme algorithme est un alias à implémentation, à ne pas confondre avec l'algorithme procédural.

Lorsqu'un objet peut effectuer une action en utilisant différents algorithmes, il est souhaitable de découpler ces algorithmes de l'objet lui-même. Avec ce pattern, les algorithmes peuvent être améliorés ou changés sans que cela n'impacte la classe qui les utilise. Il est même possible de changer dynamiquement d'algorithme en fonction du contexte dans lequel évolue l'objet.

3.2.7.2. Diagramme de classes

Voici le schéma UML de Stratégie :

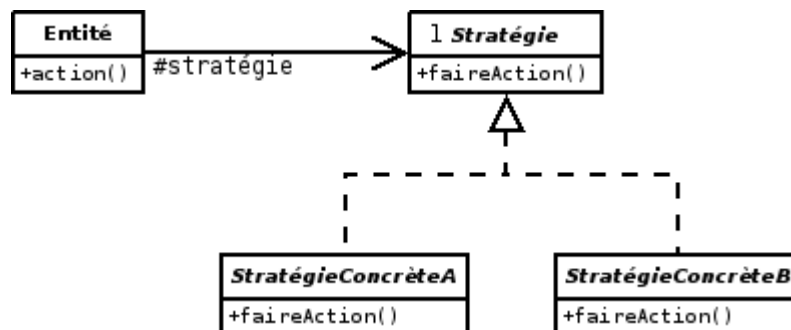


Figure 1. Pattern Stratégie

3.2.7.3. Classes participantes

Nous avons une interface *Stratégie*. Cette interface est l'interface commune à toutes les stratégies concrètes. *StratégieConcrèteA* et *StratégieConcrèteB* sont des classes d'implémentation. Elles fournissent donc une implémentation pour la méthode *faireAction*. *Entité* est la classe qui utilise l'un des algorithmes de la famille *Stratégie*. *Entité* possède une référence sur un objet de type *Stratégie*. *Entité* doit pouvoir transmettre les données à *Stratégie*. Le passage des informations est en général réalisé à travers des arguments transmis à la méthode *faireAction*. *Entité* délègue le travail à faire à *Stratégie*.

3.2.7.4. Domaines d'utilisation

Nous appliquons le pattern stratégie dans les cas suivants :

- Nous devons implémenter de nombreuses classes similaires qui diffèrent par un seul comportement. Plutôt que de créer plusieurs classes, nous en créons une seule et nous utilisons le pattern stratégie pour implémenter les différents algorithmes.
- Une classe implémente des nombreux algorithmes pour traiter un calcul. Intégrer ces algorithmes dans l'implémentation de la classe limite sa capacité à évoluer sans une modification profonde du code source. Le choix de découpler les algorithmes de la classe elle-même implique l'utilisation du pattern stratégie.

- Le classe doit choisir un algorithme pour effectuer un traitement en fonction du contexte en cours.

3.2.7.5. Exemple

Nous voulons modéliser un ensemble de véhicules sur un écran de simulation. Les types de véhicules sont nombreux. Les véhicules possèdent une méthode *deplacer* pour réaliser le déplacement. En classifiant rapidement les véhicules, nous obtenons les classes suivantes :

- *VehiculeTourisme*
- *VehiculeSport*

Une classe *Vehicule* est créée comme classe de base. C'est une classe dont la méthode *deplacer* est abstraite.

La première tentation est de créer une hiérarchie de classe et de surcharger la méthode *deplacer* pour l'adapter au type de véhicule. Les véhicules de tourisme rouleront normalement et les véhicules de sports rouleront vite.

Notre simulateur est développé dans l'optique de fonctionner une vingtaines d'années. La notion de véhicule de tourisme et véhicule de sport est somme toute subjective. Surtout dans la façon de se déplacer. Les véhicules de tourisme d'aujourd'hui sont plus rapides et plus performants que les véhicules de sports d'hier. Associer la notion de tourisme avec vitesse normale et sport avec vitesse rapide dans le code des classes n'est pas forcément judicieux.

Une possibilité est de créer tous les types de véhicules possibles et de leurs associer notre vision de la vitesse. Cette possibilité est guère réaliste, car nous ne pourrions jamais créer tous les classes nécessaires et, comme nous avons déjà dit, la notion de vitesse évolue au cours des années.

Afin de rendre notre simulateur résistant au changement, le comportement de la méthode *deplacer* doit donc être découplé des classes *VehiculeTourisme* et *VehiculeSport*. Le comportement de la méthode *deplacer* sera affecté dynamiquement au véhicule. Nous n'utiliserons pas le mécanisme de l'héritage, mais plutôt la composition d'objets.

```
interface DeplacerAlgorithme {
    public void deplacer();
}

class DeplacerVitesseModeree implements DeplacerAlgorithme {
    public void deplacer() {
        System.out.println( "Vitesse moderee." );
    }
}

class DeplacerVitesseRapide implements DeplacerAlgorithme {
    public void deplacer() {
        System.out.println( "Vitesse rapide." );
    }
}

abstract class Vehicule {
    private DeplacerAlgorithme deplacerAlgorithme;
```

```

    public Vehicule( DeplacerAlgorithme algo ) {
        setDeplacerAlgorithme( algo );
    }

    public void setDeplacerAlgorithme( DeplacerAlgorithme algo ) {
        deplacerAlgorithme = algo;
    }

    public void deplacer() {
        deplacerAlgorithme.deplacer();
    }
}

class VehiculeTourisme extends Vehicule {

    public VehiculeTourisme( DeplacerAlgorithme algo ) {
        super( algo );
    }

}

class VehiculeSport extends Vehicule {

    public VehiculeSport( DeplacerAlgorithme algo ) {
        super( algo );
    }

}

public class VehiculeStrategie {

    public static void main( String [] args ) {
        Vehicule v1 = new VehiculeTourisme( new DeplacerVitesseModeree() );
        Vehicule v2 = new VehiculeSport( new DeplacerVitesseRapide() );

        System.out.println( "\nComportements initiaux :" );

        System.out.print( "v1 : " );
        v1.deplacer();
        System.out.print( "v2 : " );
        v2.deplacer();

        v1.setDeplacerAlgorithme( new DeplacerVitesseRapide() );
        v2.setDeplacerAlgorithme( new DeplacerVitesseModeree() );

        System.out.println( "\nComportements modifies :" );

        System.out.print( "v1 : " );
        v1.deplacer();
        System.out.print( "v2 : " );
        v2.deplacer();

    }

}

```

3.2.7.6. Conclusion

Le pattern stratégie est un pattern important. Il nous permet :

- de séparer le code volatile (sujet au changement) du cœur de notre application pour en faciliter la maintenance.
- d'éviter la prolifération de classes dont le seul objectif est de modifier un comportement sans autre valeur ajoutée.
- de sélectionner un algorithme en cours d'exécution.

3.2.8. Visiteur¹³

3.2.8.1 Définition

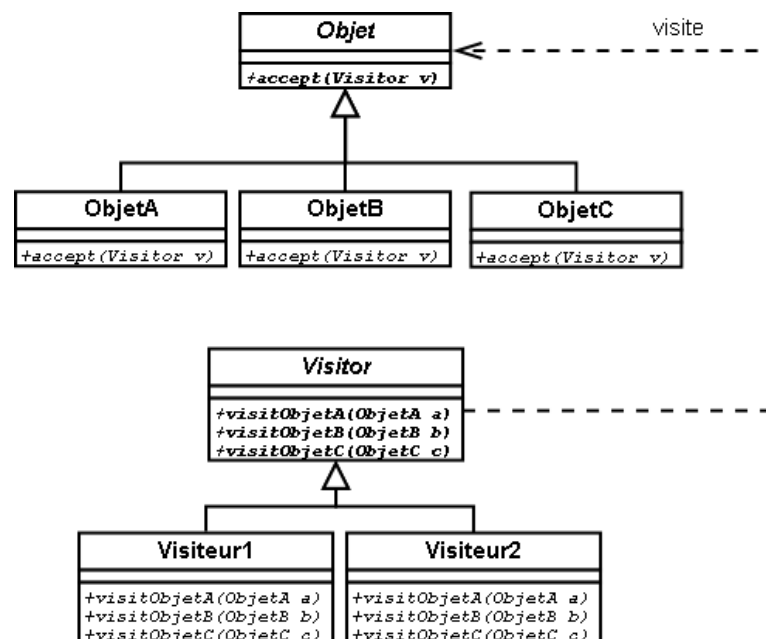
Le pattern Visiteur permet une séparation précise entre données et traitements. Il est le compagnon idéal du pattern Composite.

Le visiteur est une manière de séparer un algorithme d'une structure de données. Un visiteur possède une méthode par type d'objet traité. Pour ajouter un nouveau traitement, il suffit de créer une nouvelle classe dérivée de la classe Visiteur. On n'a donc pas besoin de modifier la structure des objets traités, contrairement à ce qu'il aurait été obligatoire de faire si on avait implémenté les traitements comme des méthodes de ces objets.

L'avantage du patron visiteur est qu'un visiteur peut avoir un état. Ce qui signifie que le traitement d'un type d'objet peut différer en fonction de traitements précédents. Par exemple, un visiteur affichant une structure arborescente peut présenter les nœuds de l'arbre de manière lisible en utilisant une indentation dont le niveau est stocké comme valeur d'état du visiteur.

Il y a 2 hiérarchies distinctes :

- Hiérarchie des objets support de données
- Hiérarchie des Visiteurs (contenant les traitements sur les données)



¹³ Source : developpez.com

3.2.8.2 Implémentation

Objet visité

```
void accept( Visitor v ) {  
    v.visitObjetDeTypeA( this ) ;  
}
```

Visiteur

```
void visitObjetDeTypeA( ObjetDeTypeA objet ) {  
    // Traitement d'un objet de type A  
}  
  
void visitObjetDeTypeB( ObjetDeTypeB objet ) {  
    // Traitement d'un objet de type B  
}  
  
void visitObjetDeTypeC( ObjetDeTypeC objet ) {  
    // Traitement d'un objet de type C  
}
```

Pour appliquer un visiteur, on procède de la sorte :

```
unObjet.accept( unVisiteur ) ;
```

Voici alors ce qu'il se passe (admettons que unObjet soit de type ObjetDeTypeA) :

1. La méthode accept de ObjetDeTypeA est appelée
2. La méthode visitObjetDeTypeA du visiteur v est alors appelée avec comme paramètre this (notre objet de type ObjetDeTypeA)
3. On entre alors dans le corps de la méthode visitObjetDeTypeA du visiteur. Le paramètre obj fait référence à l'objet de type ObjetDeTypeA sur lequel on applique notre traitement

3.2.8.3 Intérêt

Pour ajouter un traitement à notre application, il suffit donc de créer un nouveau Visiteur avant de surcharger les méthodes permettant de visiter les différents objets de la hiérarchie. Ensuite, pour utiliser le visiteur, on fait:

```
monObjet.accept( monVisiteur ) ;
```

De cette manière, on peut facilement ajouter de nouveaux traitements sans toucher à la hiérarchie de nos objets (en POO "classique", on aurait implémenté de nouvelles méthodes pour ajouter de nouvelles fonctionnalités). Grâce aux Visiteurs :

- Le code est plus clair (des fonctionnalités différentes se trouvent dans des Visiteurs différents)

- Des équipes différentes peuvent travailler sur des fonctionnalités différentes sans gêner les autres équipes
- On n'est pas obligé de tout recompiler à chaque ajout d'une fonctionnalité (seul le code du Visiteur est recompilé)

3.2.8.4 Exemple

Première étape : On définit une interface qui représente un élément.

```
interface CarElement {
    void accept(CarElementVisitor visitor);
    // Méthode à définir par les classes implémentant CarElements
}
```

Deuxième étape : On crée les classes qui étendent de cette interface.

```
class Wheel implements CarElement {
    private String name;

    Wheel(String name) {
        this.name = name;
    }

    String getName() {
        return this.name;
    }

    public void accept(CarElementVisitor visitor) {
        visitor.visit(this);
    }
}

class Engine implements CarElement {
    public void accept(CarElementVisitor visitor) {
        visitor.visit(this);
    }
}

class Body implements CarElement {
    public void accept(CarElementVisitor visitor) {
        visitor.visit(this);
    }
}

class Car {
    CarElement[] elements;

    public CarElement[] getElements() {
        return elements.clone(); // Retourne une copie du tableau de références
    }

    public Car() {
        this.elements = new CarElement[] {
            new Wheel("front left"),
            new Wheel("front right"),
            new Wheel("back left"),
            new Wheel("back right"),
            new Body(),
            new Engine()
        };
    }
}
```

```
}  
}
```

Troisième étape : On crée une interface qui représente Visitor.

```
interface CarElementVisitor {  
    void visit(Wheel wheel);  
    void visit(Engine engine);  
    void visit(Body body);  
    void visitCar(Car car);  
}
```

Quatrième étape : On crée deux classes visitor qui étendent l'interface précédente.

```
class CarElementPrintVisitor implements CarElementVisitor {  
    public void visit(Wheel wheel) {  
        System.out.println("Visiting " + wheel.getName() + " wheel");  
    }  
  
    public void visit(Engine engine) {  
        System.out.println("Visiting engine");  
    }  
  
    public void visit(Body body) {  
        System.out.println("Visiting body");  
    }  
  
    public void visitCar(Car car) {  
        System.out.println("\nVisiting car");  
        for(CarElement element : car.getElements()) {  
            element.accept(this);  
        }  
        System.out.println("Visited car");  
    }  
}  
  
class CarElementDoVisitor implements CarElementVisitor {  
    public void visit(Wheel wheel) {  
        System.out.println("Kicking my " + wheel.getName());  
    }  
  
    public void visit(Engine engine) {  
        System.out.println("Starting my engine");  
    }  
  
    public void visit(Body body) {  
        System.out.println("Moving my body");  
    }  
  
    public void visitCar(Car car) {  
        System.out.println("\nStarting my car");  
        for(CarElement carElement : car.getElements()) {  
            carElement.accept(this);  
        }  
        System.out.println("Started car");  
    }  
}
```

Cinquième étape : On crée une classe de test pour la démonstration grâce aux deux classes précédentes.

```

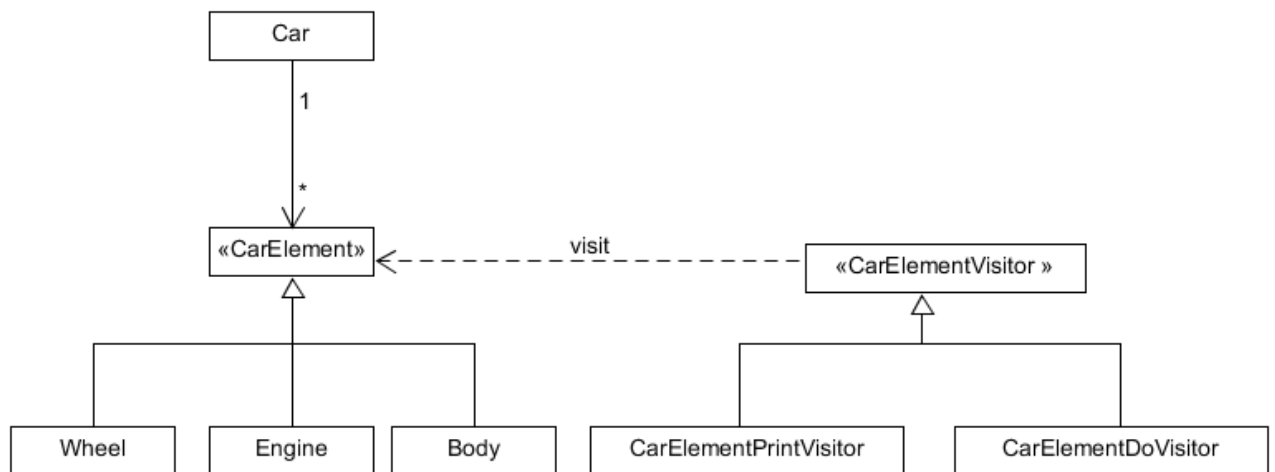
public class TestVisitorDemo {
    static public void main(String[] args) {
        Car car = new Car();

        CarElementVisitor printVisitor = new CarElementPrintVisitor();
        CarElementVisitor doVisitor = new CarElementDoVisitor();

        printVisitor.visitCar(car);
        doVisitor.visitCar(car);
    }
}

```

Voilà à quoi ressemble le diagramme de classe de notre exemple :



Sixième étape : On vérifie que tout cela fonctionne.

```

public class Main{//On remplace ici le nom de la classe TestVisitorDemo par
Main pour être exécuter sur tech.io
    static public void main(String[] args) {

        Car car = new Car();

        CarElementVisitor printVisitor = new CarElementPrintVisitor();
        CarElementVisitor doVisitor = new CarElementDoVisitor();

        printVisitor.visitCar(car);
        doVisitor.visitCar(car);
    }
}

```

Sortie à la console

```

Visiting car
Visiting front left wheel
Visiting front right wheel
Visiting back left wheel
Visiting back right wheel
Visiting body
Visiting engine
Visited car

Starting my car

```



```
Kicking my front left  
Kicking my front right  
Kicking my back left  
Kicking my back right  
Moving my body  
Starting my engine  
Started car
```

3.2.9. MVC

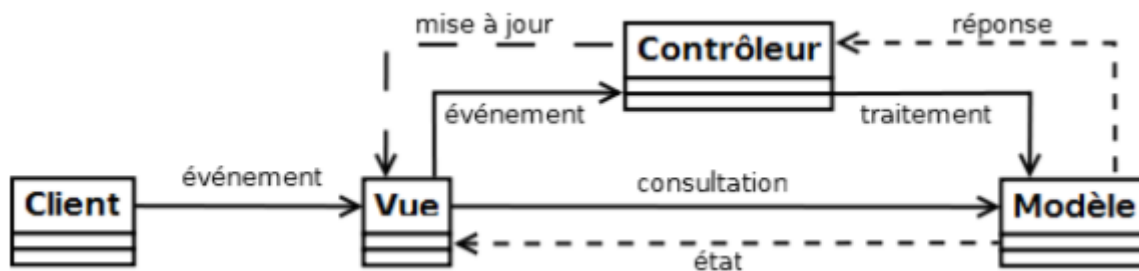
3.2.9.1 Définition

L'architecture Model-View-Controller (ou Modèle-Vue-Contrôleur) est un modèle d'architecture créé en 1979, et proposant de structurer l'application en 3 couches qui communiquent entre elles. Ce pattern facilite la maintenance des applications par le découplage vue-modèle. Il est principalement utilisé pour les interfaces graphiques.

3.2.9.2 Objectif

Le patron de conception vise à séparer la présentation des informations (la vue) de leur représentation interne (le modèle). Une tierce partie (le contrôleur) est en charge d'assurer la cohérence entre ces deux couches.

Ce patron est principalement utilisé dans les interface graphiques (Java Swing).



3.2.9.3 La couche modèle

La couche modèle est la couche métier, le coeur de l'application. Elle représente le comportement de l'application : contient les données de l'application, et effectue des traitements sur ces données. L'interface du modèle propose de consulter ou de mettre à jour les données, pas de les présenter.

3.2.9.4 La couche vue

La couche vue est l'interface avec l'utilisateur. D'une part, elle est en charge de présenter les résultats du modèle en interrogeant celui-ci (lecture seule). D'autre part, elle permet de capturer les actions de l'utilisateur, soit pour mettre à jour la vue (si elle est la seule concernée par l'action), soit pour transmettre une requête spécifique au contrôleur (elle n'effectue aucun traitement sur les données de l'application).

Il est à noter que plusieurs vues sont possibles pour les mêmes données.

3.2.9.5 La couche contrôleur

La couche contrôleur gère les événements de synchronisation. Elle reçoit les événements de l'utilisateur, via la vue. Elle déclenche les actions à effectuer : transmet des requêtes de mise à jour au modèle, et informe la vue de se mettre à jour. Le contrôleur n'effectue aucun traitement, et ne modifie aucune donnée.

3.2.9.6 Exemple de MVC : les Swing Java

L'utilisation du MVC est native en Swing. La plupart des composants Swing (hormis les conteneurs) utilisent une classe spécifique pour contenir leurs données. Les listes (JList) utilisent un objet de classe ListModel pour les données et un objet de classe ListSelectionModel pour gérer les sélections. Les arbres (JTree) utilisent un objet de classe TreeModel pour les données et un objet de classe TreeSelectionModel pour gérer les sélections. Les tables (JTable) utilisent des objets de classe TableModel et TableColumnModel pour les données et un objet de classe ListSelectionModel pour gérer les sélections.

3.2.9.7 Exemple de MVC : un programme Java

Voici un exemple très très simplifié d'un MVC en java.

```
===== Modele.java =====
import java.util.Observable;

class Modele extends Observable {
    private String value;

    void setValue(String value) {
        this.value = value;
        setChanged();
        notifyObservers();
    }

    String getValue() {
        return value;
    }
}

===== Vue.java =====
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.ArrayList;
import java.util.List;
import java.util.Observable;
import java.util.Observer;

class Vue implements Observer {
    Modele modele;
    List<ActionListener> listeners = new ArrayList<ActionListener>();

    Vue(Modele modele) {
        this.modele = modele;
        modele.addObserver(this);
    }

    public void update(Observable o, Object arg) {
```

```

        System.out.println("La valeur saisie est: " + modele.getValue()
+ "\n");
        lireClavier();
    }

    public void lireClavier() {
        System.out.print("Saisissez une valeur: ");
        String s = LectureClavier.lireString();
        for (ActionListener listener : listeners) {
            listener.actionPerformed(new ActionEvent(this, 0, s));
        }
    }
}

===== Controleur.java =====
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

class Controleur implements ActionListener {
    Modele modele;
    Vue vue;

    Controleur(Modele modele, Vue vue) {
        this.modele = modele;
        this.vue = vue;
    }

    public void actionPerformed(ActionEvent e) {
        modele.setValue(e.getActionCommand());
    }
}

===== LectureClavier.java =====
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

public class LectureClavier {
    public static String lireString() {
        String ligne = null;

        try {
            InputStreamReader i = new InputStreamReader(System.in);
            BufferedReader b = new BufferedReader(i);
            ligne = b.readLine();
        }
        catch(IOException e) {
            System.err.println(e);
        }
        return ligne;
    }
}

===== Essai.java =====
class Essai {
    public static void main(String[] arg) {
        Modele modele = new Modele();
        Vue vue = new Vue(modele);
        Controleur controleur = new Controleur(modele, vue);

        vue.listeners.add(controleur);
        vue.lireClavier();
    }
}

```

1. Observateur - Exercice 1

Solution pour l'exercice 1 du pattern Observateur. Chaque observateur a la possibilité de modifier la valeur du sujet. L'erreur de conversion, toujours possible, est gérée correctement dans la classe servant de base aux dialogues. Dès que le sujet est modifié, il notifie aux observateurs enregistrés qu'il est temps de se réafficher.

Quelques remarques :

- Afin de simplifier l'exemple, le sujet est une variable statique déclarée dans la classe *ObservateurExercice1*.
- Le sujet stocke le nombre sous une forme décimale. De ce fait, les observateurs doivent convertir le nombre avant de modifier le sujet.

```
import java.util.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

interface Sujet {
    void attacher(Observateur o);
    void detacher(Observateur o);
    void notifier();
    Integer getNombre();
    void setNombre(Integer nombre);
}

class SujetNombre implements Sujet {
    private ArrayList<Observateur> observateurs = new ArrayList<Observateur>();
    private Integer nombre;

    public void attacher(Observateur o) {
        observateurs.add(o);
    }

    public void detacher(Observateur o) {
        observateurs.remove(observateurs.indexOf(o));
    }

    public void notifier() {
        for (Observateur o: observateurs)
            o.mettreAJour();
    }

    public Integer getNombre() {
        return nombre;
    }

    public void setNombre(Integer nombre) {
        this.nombre = nombre;
        notifier();
    }
}

interface Observateur {
    public void soumettre();
    public void mettreAJour();
}
```

```

abstract class Dialogue extends JDialog implements Observateur {
    protected Sujet sujet;
    protected JTextField editeur;
    protected JButton bouton;

    public Dialogue(Frame parent, Sujet sujet) {
        super(parent);
        setTitle(getClass().getName());

        this.sujet = sujet;

        sujet.attacher(this);

        setLayout(new GridLayout(2, 1));
        editeur = new JTextField();
        bouton = new JButton("Soumettre");

        bouton.addActionListener( new ActionListener() {
            public void actionPerformed( ActionEvent evt) {
                try {
                    soumettre();
                }
                catch(Exception e) {
                    JOptionPane.showMessageDialog(
                        null,
                        e.getMessage(),
                        "Erreur de conversion",
                        JOptionPane.INFORMATION_MESSAGE);
                }
            }
        });

        add(editeur);
        add(bouton);
    }
}

class DialogueHex extends Dialogue {

    DialogueHex(Frame parent, Sujet sujet) {
        super(parent, sujet);
        setBounds(300, 200, 200, 90);
    }

    public void mettreAJour() {
        editeur.setText(Integer.toHexString(sujet.getNombre()));
    }

    public void soumettre() {
        sujet.setNombre(Integer.parseInt(editeur.getText(), 16));
    }
}

class DialogueOct extends Dialogue {

    DialogueOct(Frame parent, Sujet sujet) {
        super(parent, sujet);
        setBounds(300, 300, 200, 90);
    }

    public void mettreAJour() {
        editeur.setText(Integer.toOctalString(sujet.getNombre()));
    }
}

```

```

    }

    public void soumettre() {
        sujet.setNombre(Integer.parseInt(editeur.getText(), 8));
    }
}

class DialogueDec extends Dialogue {

    DialogueDec(Frame parent, Sujet sujet) {
        super(parent, sujet);
        setBounds(300, 100, 200, 90);
    }

    public void mettreAJour() {
        editeur.setText(Integer.toString(sujet.getNombre(), 10));
    }

    public void soumettre() {
        sujet.setNombre(Integer.parseInt(editeur.getText(), 10));
    }
}

class Fenetre extends JFrame {
    private DialogueHex dialogueHex;
    private DialogueOct dialogueOct;
    private DialogueDec dialogueDec;

    public Fenetre() {
        super();

        dialogueHex = new DialogueHex(this, ObservateurExercice1.sujet);
        dialogueHex.setVisible(true);
        dialogueOct = new DialogueOct(this, ObservateurExercice1.sujet);
        dialogueOct.setVisible(true);
        dialogueDec = new DialogueDec(this, ObservateurExercice1.sujet);
        dialogueDec.setVisible(true);
    }
}

public class ObservateurExercice1 {
    static Sujet sujet = new SujetNombre();

    public static void main(String[] args) {
        Fenetre f = new Fenetre();
        f.setTitle("Observateurs");
        f.setBounds(80, 100, 200, 90);
        f.setVisible(true);
    }
}

```

Solution proposée par Christophe Enderlin pour l'exercice 1 sur le pattern *Observateur*.

2. Observateur - Exercice 2

Une autre solution possible pour cet exercice :

```

import java.util.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

interface EtatCompte {
    void deposer(Double montant);
    void retirer(Double montant) throws Exception;
}

class CompteBloque implements EtatCompte {
    protected Compte compte;

    public CompteBloque(Compte compte) {
        this.compte = compte;
    }

    public void deposer(Double montant) {
        compte.solde += montant;

        if( compte.solde >= 0 ) {
            compte.etat = new CompteActif(compte);
        }
    }

    public void retirer(Double montant) throws Exception{
        throw new Exception("Compte bloqué");
    }
}

class CompteActif implements EtatCompte {
    protected Compte compte;

    public CompteActif(Compte compte) {
        this.compte = compte;
    }

    public void deposer(Double montant) {
        compte.solde += montant;
    }

    public void retirer(Double montant) throws Exception{
        compte.solde -= montant;

        if (compte.solde < 0 ) {
            compte.etat = new CompteBloque( compte );
        }
    }
}

interface Sujet {
    void attacher(Observateur o);
    void detacher(Observateur o);
    void notifier();
}

class Compte implements Sujet {
    protected ArrayList<Observateur> observateurs = new
ArrayList<Observateur>();
    protected EtatCompte etat;
    protected Double solde;
    protected String numero;
}

```

```

    public Compte(String numero, Double solde) {
        this.numero = numero;
        this.solde = solde;

        if( solde >= 0) {
            etat = new CompteActif(this);
        }
        else {
            etat = new CompteBloque(this);
        }
    }

    public void attacher(Observateur o) {
        observateurs.add(o);
    }

    public void detacher(Observateur o) {
        observateurs.remove(observateurs.indexOf(o));
    }

    public void notifier() {
        for (Observateur o: observateurs)
            o.mettreAJour();
    }

    public Double getSolde() {
        return solde;
    }

    public void deposer(Double montant) {
        etat.deposer(montant);
        notifier();
    }

    public void retirer(Double montant) throws Exception {
        try {
            etat.retirer(montant);
            notifier();
        }
        catch(Exception e) {
            System.out.println(e.getMessage());
        }
    }

    public String getEtat() {
        return etat.getClass().getName();
    }
}

interface Observateur {
    public void mettreAJour();
}

class DialogueSaisie extends JDialog implements Observateur, ActionListener {
    protected Compte compte;
    protected JTextField editeur;
    protected JButton boutonDeposer;
    protected JButton boutonRetirer;

    public DialogueSaisie(Frame parent, Compte compte) {
        super(parent);
        setTitle("Saisie");
        setBounds(300, 100, 200, 90);
        this.compte = compte;
    }
}

```



```

        compte.attacher(this);
        setLayout(new GridLayout(3, 1));
        editeur = new JTextField();
        boutonDeposer = new JButton("DÃ©poser");
        boutonRetirer = new JButton("Retirer");
        add(editeur);
        add(boutonDeposer);
        add(boutonRetirer);
        boutonDeposer.addActionListener(this);
        boutonRetirer.addActionListener(this);
    }

    public void actionPerformed(ActionEvent evt) {
        if (evt.getSource() == boutonDeposer) {
            try {
                compte.deposer(Double.parseDouble(editeur.getText()));
            }
            catch (Exception e) {
                JOptionPane.showMessageDialog(
                    null,
                    e.getMessage(),
                    "Erreur de conversion",
                    JOptionPane.INFORMATION_MESSAGE);
            }
        }
        else if (evt.getSource() == boutonRetirer) {
            try {
                compte.retirer(Double.parseDouble(editeur.getText()));
            }
            catch (Exception e) {
                JOptionPane.showMessageDialog(
                    null,
                    e.getMessage(),
                    "Erreur de conversion",
                    JOptionPane.INFORMATION_MESSAGE);
            }
        }
    }

    public void mettreAJour() {
        if (compte.getEtat() == "CompteBloque") {
            boutonRetirer.setEnabled(false);
        }
        else {
            boutonRetirer.setEnabled(true);
        }
    }
}

class DialogueVue extends JDialog implements Observateur {
    protected Compte compte;
    protected JLabel labelSolde;
    protected JLabel labelEtat;

    public DialogueVue(Frame parent, Compte compte) {
        super(parent);

        setTitle("Vue");
        setBounds(300, 200, 200, 90);

        this.compte = compte;
        compte.attacher(this);
    }
}

```

```

        setLayout(new GridLayout(2, 1));
        labelSolde = new JLabel("Solde :");
        labelEtat = new JLabel("Etat :");

        add(labelSolde);
        add(labelEtat);

        mettreAJour();
    }

    public void mettreAJour() {
        labelSolde.setText("Solde : " + compte.getSolde());
        labelEtat.setText("Etat : " + compte.getEtat());
    }
}

class Fenetre extends JFrame {
    private DialogueSaisie dialogueSaisie;
    private DialogueVue dialogueVue;

    public Fenetre() {
        super();

        Compte compte = new Compte( "NUMERO1", 10000.00 );
        dialogueSaisie = new DialogueSaisie(this, compte);
        dialogueSaisie.setVisible(true);
        dialogueVue = new DialogueVue(this, compte);
        dialogueVue.setVisible(true);
    }
}

public class TestCompte {
    public static void main(String[] args) {
        Fenetre f = new Fenetre();
        f.setTitle("Compte");
        f.setBounds(80, 100, 200, 90);
        f.setVisible(true);
    }
}

```